《数字信号分析理论与实践》实验案例集

何岭松
华中科技大学机械学院仪器系

第一章：波形分析

采样定理与频率混叠实验

采样信号的量化误差实验

正弦波信号与电子琴实验

标准函数与信号发生器实验

典型信号波形分析实验

音频文件信号波形分析实验

典型信号的微分与积分实验

李沙育图实验

旋转机械轴心轨迹模拟实验

第二章：频谱分析

方波信号的合成与吉普斯现象实验

FFT 谱的能量泄漏与栅栏效应实验

窗函数的作用实验

FFT 谱栅栏效应误差的插值修正实验

音频文件信号波形和频谱分析实验

麦克风信号波形和频谱分析实验

线性谱和对数谱显示实验

第三章：幅值域分析

典型信号的概率密度和概率分布曲线实验

设备故障的概率密度曲线诊断实验

图象的直方图分析实验

第四章：相关分析

典型信号的相关分析实验

互相关分析测距离实验

自相关分析回声时延测量实验

第五章：数字滤波

信号滤波概念实验

FIR 滤波器脉冲响应设计法和吉普斯效应

FIR 滤波器窗函数设计法

音频信号谱减法降噪实验

双线性变换的频带压缩实验

常用 IIR 滤波器的特点实验

第六章：信号的时频分析

典型时变信号的谱阵分析和语谱图分析实验

音频信号的语谱图分析实验

工业信号的谱阵分析实验

第七章：信号的小波分析

小波函数的时移和频移实验

时变信号的连续小波变换实验

从连续小波变换到离散小波变换实验

离散小波变换的数字滤波算法实验

离散小波变换的多尺度分解与重构实验

从离散小波变换到小波包变换的变迁实验

从小波滤波器算法到提升小波算法实验

第八章：希尔伯特-黄变换

希尔伯特变换和解析信号实验

希尔伯特-黄变换实验

第九章：信号的盲源分离

中心极限定理实验

读懂 FastICA 盲源分离算法实验

音频信号的盲源分离实验

数据的 PCA 降维和可视化处理实验

信号的 PCA 降噪实验

信号的 PCA 多尺度分解与重构

第十章：信号的时间序列谱分析

时间序列仿真与建模实验

典型时间序列 ARIMA 建模实验

典型时间序列的高阶 AR 建模谱估计实验

采样定理与频率混叠实验

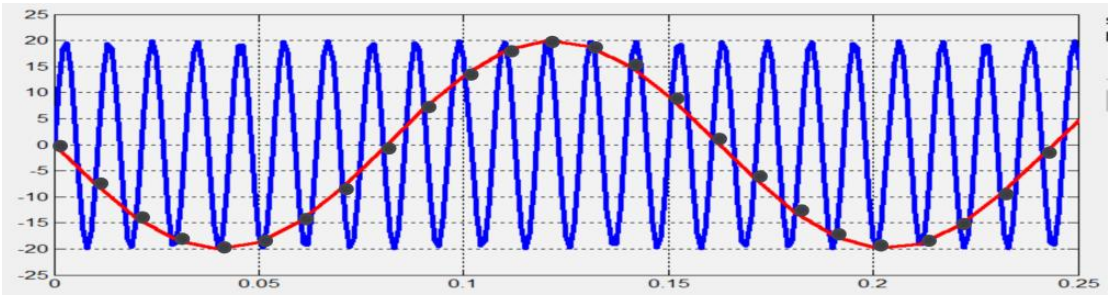
采样定理(Nyquist–Shannon sampling theorem)

为保证采样后信号能真实地保留原始模拟信号信息，信号采样频率必须至少为原信号中最高频率成分的 2 倍。这称为采样定理。

$$Fs \quad > \quad 2\ Fmax$$

频率混叠现象

信号采样时，当采样频率不满足采样定理，信号中的高频成分会被错误的采样成低频成分，这一现象称为频率混叠。频率混叠是信号离散采样所引起的一个特有现象和重要概念。



实验设计

用 sin 函数产生一个 1-100Hz 频率可调的正弦波信号，然后分别用 Fs1=1000Hz 和 Fs2=100Hz 进行采样并显示它们的波形。在正弦波信号频率从 0 到 100Hz 的变化过程中，曲线 1 始终满足采样定理，曲线 2 则从信号频率接近 50Hz 开始发生频率混叠现象；同时，当信号频率大于 Fs2=50Hz 时，用曲线 2 信号计算的频谱以 Fs2=50Hz 为轴折迭，在频谱上显示出虚假混叠频率的位置。

编程实现上用 arange 函数生成 0,1,2,…,N-1 的数字序列，乘以采样间隔 dt，就形成采样脉冲序列 k*dt:

```
t=np.arange(N)*dt
```

然后用采样脉冲序列 t 对 sin 函数进行采样即可：

```
x=A*sin(2pi*F*t)
```

其中 dt=1/Fs，若 Fs 小于 2*F，则产生频率混叠。

采样定理与频率混叠实验样例代码

```
import tkinter as tk
import numpy as np
from scipy.fftpack import fft,ifft
import matplotlib.pyplot as plt
import drvi.drviDSP as dsp
import drvi.drviControlls as dr

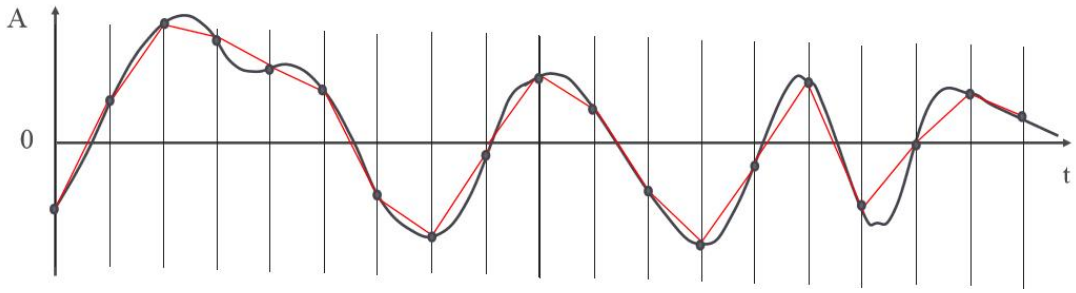
#==CallBack Function==============
def myWorking(v):
 Fs1=1000; dt1=1.0/Fs1; N1=int(1/dt1)
```

```
 t1=np.arange(N1)*dt1
 x1=0.8*np.sin(2*np.pi*v*t1)
 mPlotWave1.setValue2D(t1,x1)
 Fs2=100; dt2=1.0/Fs2; N2=int(1/dt2)
 t2=np.arange(N2)*dt2
 x2=0.8*np.sin(2*np.pi*v*t2)
 mPlotWave2.setValue2D(t2,x2)
 if v<Fs2/2: mDigital2.setValueSingle(v)
 if v>=Fs2/2: mDigital2.setValueSingle(Fs2-v)
 Fs2=100; dt2=1.0/Fs2; N2=2048
 t2=np.arange(N2)*dt2
 x2=0.8*np.sin(2*np.pi*v*t2)
 [mFre,mAmp]=dsp.AmpSpetrumX(N2,Fs2,x2)
 mPlotAmp.setValue2D(mFre,mAmp)
 return 1

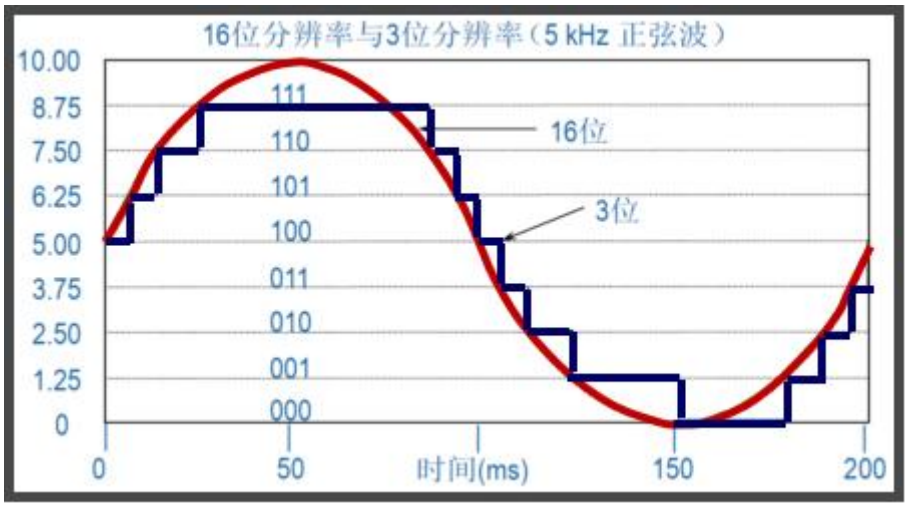
#==Main Window====================
win= tk.Tk()
winWidthCur=1100
winHeightCur=670
win.geometry("%dx%d" %(winWidthCur,winHeightCur))
win.config(bg="#ddeeee")
win.wm_title('信号的采样定理与频率混叠现象')
#==Layout of controls====================
mL1=dr.DRLabel(win,20,25,500,40,'#440000','#ffffff','信号的采样定理与频率混叠现象',20)
mPlotWave1=dr.DRPlot(win,20,70,900,187,'','#113344','#000000','#ffffff','#00eeff','#ffff00',0,1,-1,1,0,0)
mPlotWave2=dr.DRPlot(win,20,255,900,187,'','#113344','#000000','#ffffff','#00eeff','#ffff00',0,1,-1,1,0,0)
mPlotAmp=dr.DRPlot(win,20,440,900,187,'','#113344','#000000','#ffffff','#00eeff','#ffff00',0,50,0,1,0,0)
mL2=dr.DRLabel(win,800,70,120,30,'#663300','#ffffff','Fs = 1000Hz',12)
mL3=dr.DRLabel(win,800,260,120,30,'#663300','#ffffff','Fs = 100Hz',12)
mKnobA=dr.DRGKnob(win,930,100,150,150,'#001122','#226688','#222222','#aaaaaa','0,100',0,100,5,11)
mL4=dr.DRLabel(win,930,360,140,30,'#ddeeee','#000000','信号频率：',18)
mDigital1=dr.DRDigital(win,940,400,120,40,'#000000','#00ffaa',5,2,22)
mL5=dr.DRLabel(win,930,460,140,30,'#ddeeee','#000000','混叠频率：',18)
mDigital2=dr.DRDigital(win,940,500,120,40,'#000000','#00ffaa',5,2,22)
mSpin1=dr.DRSpinBox(win,940,260,140,26,'#ffffff','#000000',5,1,0,100,11)
#==DataStream Binding====================
mKnobA.addCallBackSingle(mDigital1.setValueSingle)
mKnobA.addCallBackSingle(mSpin1.setValueSingle)
mKnobA.addCallBackSingle(myWorking)
mSpin1.addCallBackSingle(mDigital1.setValueSingle)
mSpin1.addCallBackSingle(mKnobA.setValueSingle)
mSpin1.addCallBackSingle(myWorking)
myWorking(5)
#==Main Loop====================
win.mainloop()
```

采样信号的量化误差实验

时间连续的模拟信号经过采样后变为有限个数据点的离散信号，数据点间用直线进行插值逼近，所造成的误差称为离散采样误差，采样频率越高，误差越小。



而采样过程中幅值连续的信号经 A/D 器件量化变为只有有限个有效数字的数字量，这一过程所产生的误差称为量化误差。



实验设计

用 44100Hz 的采样频率和 sin 函数产生一个 800Hz 的音频信号，然后分别进行 2 / 4 / 8 / 16 /32 / 64 / 128 / 256 级的量化，最后再作为化为音频信号播出，感受由于量化带来音频中的颗粒噪声。

采样信号的量化误差实验样例代码

```
import tkinter as tk
import numpy as np
from scipy.fftpack import fft,ifft
import matplotlib.pyplot as plt
import drvi.drviDSP as dsp
import drvi.drviControlls as dr
import drvi.drviSimpleAudio as sa
#==CallBack Function==============
```

```
def myWorking(v):
 global Fs,N,tData,xData,xData1
 if v==0: xData1=np.round(128*xData)/128
 if v==1: xData1=np.round(64*xData)/64
 if v==2: xData1=np.round(32*xData)/32
 if v==3: xData1=np.round(16*xData)/16
 if v==4: xData1=np.round(8*xData)/8
 if v==5: xData1=np.round(4*xData)/4
 if v==6: xData1=0.8*np.round(2*xData)/2
 if v==7: xData1=0.8*np.round(xData)
 mPlotWave2.setValue2D(tData,xData1)
 return 1

def myPlay(v):
 global Fs,N,tData,xData1
 sa.play(1,Fs,xData1,0)
 return 1

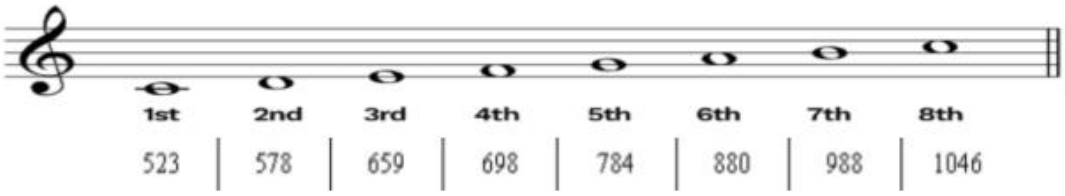
#==Main Window====================
win= tk.Tk()
winWidthCur=1100
winHeightCur=670
win.geometry("%dx%d" %(winWidthCur,winHeightCur))
win.config(bg="#ddeeee")
#==Layout of controls====================
mL1=dr.DRLabel(win,20,25,380,40,'#440000','#ffffff','采样信号的量化误差',20)
mPlotWave1=dr.DRPlot(win,20,90,1000,267,'','#002f5e','#000000','#ffffff','#00eeff','#ffff00',0,0.01,-1,1,0,0)
mPlotWave2=dr.DRPlot(win,20,355,1000,267,'','#002f5e','#000000','#ffffff','#00eeff','#ffff00',0,0.01,-1,1,0,0)
mBut=dr.DRGButton(win,930,50,90,36,'#003300','#00aa00','#ffffff',12,'放音',12,101)
mHBut1=dr.DRHButtonGroup(win,510,50,400,36,'#114433','#ffffff','256,128,64,32,16,8,4,2',11,0)
mHBut1.addCallBackSingle(myWorking)
mBut.addCallBackSingle(myPlay)
#================================
Fs=44100;
dt=1.0/Fs;
N=40000;
tData=np.arange(N)*dt
xData=0.8*np.sin(2*np.pi*800*tData)
xData1=xData
mPlotWave1.setValue2D(tData,xData)
myWorking(0)
#==Main Loop====================
win.mainloop()
```

正弦波信号与电子琴实验

正弦波因信号波形是数学上的正弦曲线而得名。正弦波是一种单一频率成分的基本信号，在后续信号的频谱分析理论中将会学到，任何复杂信号都可以看成是一组不同频率、幅值、相位的正弦波信号的组合。

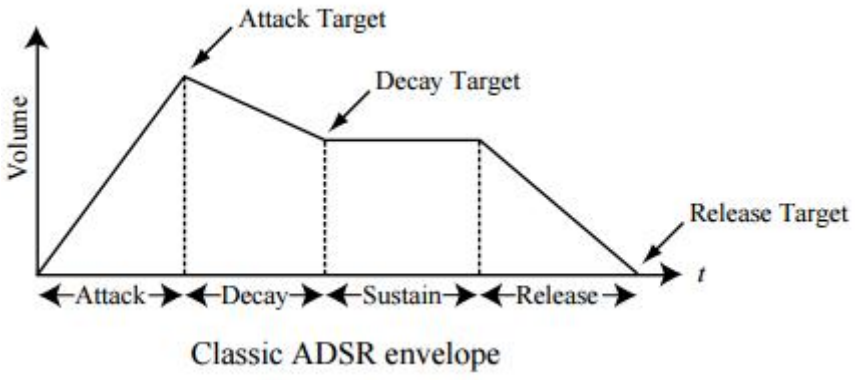
$$y(t) = A\sin(2\pi ft + \varphi)$$

许多实际信号波形，如音叉产生的纯音信号、交流电信号等，其波形都是正弦信号。因此，我们可以用正弦波信号来生成哆、来、咪的声音，也就是可以设计实现一个简单的电子琴。



实验设计

用 44100Hz 的采样频率和 sin 函数产生按哆、来、咪音高变化的不同频率的正弦波信号，然后作为音频信号播出。为增加真实感，对信号加入了 ADSR 幅值包络。



Classic ADSR envelope

正弦波信号与电子琴实验样例代码

```
import tkinter as tk
import numpy as np
from scipy.fftpack import fft,ifft
import matplotlib.pyplot as plt
from scipy import signal
import drvi.drviDSP as dsp
import drvi.drviControlls as dr
import drvi.drviSimpleAudio as sa
#=======================================
def setValue1(v):
 Fs=44100;
 dt=1.0/Fs;
 N=40000;
```

```
 mF=500*1.13**v
 tData=np.arange(N)*dt
 xData=0.8*np.sin(2*np.pi*mF*tData)
 mPlotWave.setValue2D(tData,xData)
 [mFre,mAmp]=dsp.AmpSpetrumX(N,Fs,xData)
 mPlotAmp.setValue2D(mFre,mAmp)
 mDigital1.setValueSingle(mF)

def setValue(v):
 Fs=44100;
 dt=1.0/Fs;
 N=40000;
 mF=500*1.13**v
 tData=np.arange(N)*dt
 xData=0.8*np.sin(2*np.pi*mF*tData)
 mPlotWave.setValue2DFast(tData,xData)
 [mFre,mAmp]=dsp.AmpSpetrumX(N,Fs,xData)
 mPlotAmp.setValue2DFast(mFre,mAmp)
 mDigital1.setValueSingle(mF)
 y=xData
 e=mRadio1.getValueSingle()
 if e==0:
     w=dsp.ADSREnvelop(N)
     y=xData*w
 sa.play(1,Fs,y,0)

#==Main Window=====================
win= tk.Tk()
winWidthCur=1100
winHeightCur=670
win.geometry("%dx%d" %(winWidthCur,winHeightCur))
win.config(bg="#ddeeee")
win.wm_title('正弦波纯音信号波形和频谱')
#==Layout of controls=====================
mLab1=dr.DRLabel(win,20,30,400,40,'#003355','#ffffff','正弦波纯音信号波形和频谱',20)
mPlotWave=dr.DRPlot(win,20,80,1000,240,'','#003355','#000000','#ffffff','#00eeff','#ffff00',0,0.01,-1,1,0,0)
mPlotAmp=dr.DRPlot(win,20,320,1000,240,'','#003355','#000000','#ffffff','#00eeff','#ffff00',0,2500,0,1,0,0)
mHBut2=dr.DRHButtonGroup(win,20,570,1000,50,'#000000','#ffffff','1,2,3,4,5,6,7,8,9',12,0)
mDigital1=dr.DRDigital(win,920,40,100,36,'#000000','#00ffaa',0,0,22)
mRadio1=dr.DRRadioBut(win,600,40,200,30,txt='ADSR,直接',d=1)
#==DataStream Binding=====================
mHBut2.addCallBackSingle(setValue)
#==Main Loop=====================
win.mainloop()
```

标准函数与信号发生器实验

信号发生器是实验室中常见的仪器设备，可以生成正弦波、方波、三角波等典型信号,在生产实践和科技领域中有着广泛的应用。



编程实现信号发生器的原理并不复杂，先对标准函数进行采样，得到数字信号，然后通过 D/A，将数字信号转换为模拟信号进行输出。下面信号发生器设计中采用的标准函数和对应的典型信号：

Sin 函数与正弦波信号

Square 函数与方波信号

Sawtooth 函数与三角波信号

Randn 函数与白噪声信号

实验设计

Numpy 库和 Scipy 库中集成了常用的信号函数，生成采样脉冲数组后，直接调用上述函数进行离散采样即可。注意，采样频率应满足采样频率要求。此处采用 DRVI 库中的信号发生器控件，更简单一些。

标准函数与信号发生器实验样例代码

```
import tkinter as tk
import drvi.drviDSP as dsp
import drvi.drviControlls as dr
#==Main Window=====================
win= tk.Tk()
win.geometry("1100x670")
win.config(bg="#ddeeee")
#==Layout of controls=====================
mLab1=dr.DRLabel(win,20,30,400,40,'#003355','#ffffff','信号发生器',20)
mPlotWave=dr.DRPlot(win,20,80,900,480,'','#003355','#000000','#ffffff','#00eeff','#ffff00',0,0.05,-1,1,0,0)
mVButType=dr.DRRadioBut(win,940,80,120,140,'#003355','#005577',"#ffff00",'正弦波,方波,三角波,白噪声',12,0,0)
mKnobA=dr.DRKnob(win,930,230,130,130,'#004466','#222222','#aaaaaa','0,1',0,1,0.5,11)
mKnobFre=dr.DRKnob(win,930,350,130,130,'#004466','#222222','#aaaaaa','1,1000',1,1000,50,11)
mSwitch1=dr.DRSwitch(win,960,520,80,36,'#004466','#aaddee')
mType=0; Fs=44100; N=4096; A=0.6; Fre=100; Pha=0
mSignal=dsp.DRGenerator(mType,Fs,N,A,Fre,Pha)
#==DataStream Binding=====================
mSwitch1.addCallBackSingle(mSignal.setValueSingle)
mSignal.addCallBack2D(mPlotWave.setValue2DFast)
```

```
mVButType.addCallBackSingle(mSignal.setVSType)
mKnobA.addCallBackSingle(mSignal.setVSAmp)
mKnobFre.addCallBackSingle(mSignal.setVSFre)
#==Main Loop=====================
win.mainloop()
```

典型信号波形分析实验

波形分析就是直接对所记录的信号幅值随时间变化曲线进行分析。因在时间域进行，也称为时域分析。示波器是最常用的信号波形分析工具，可以直接读取信号的幅值、周期等特征参数。



实验设计

从当前显示的信号波形中估计信号的周期/频率(如果是周期信号)、峰值、均值、有效值、标准差；以及无量纲的波形参数，如波形因子、峰值因子、脉冲因子、裕度因子、峭度和偏度，以及反映信号复杂性的近似熵等。

典型信号波形分析实验样例代码

```
import tkinter as tk
import numpy as np
from scipy.fftpack import fft,ifft
from scipy import signal
from scipy.stats import kurtosis
import matplotlib.pyplot as plt
import drvi.drviDSP as dsp
import drvi.drviControlls as dr
#=========================================
def setValue2D(t,x):
  Fs=1/(t[1]-t[0]);
  N=len(x)
  mPlotWave.setValue2D(t,x)
  T=dsp.findPeriod(Fs,N,x)
  if T==0: s='频率: ----'
  if T>0:    s='频率:'+format(1/T,'.2f')
  mTxt.setValueString(s)
  pp=np.max(x)-np.min(x)
  s='峰峰值:'+format(pp,'.2f')
  mTxt1.setValueString(s)
  R=np.sqrt(np.sum(x**2)/N)
  s='有效值:'+format(R,'.2f')
  mTxt2.setValueString(s)
  std=np.std(x)
  s='方差:'+format(std,'.2f')
  mTxt3.setValueString(s)
  M=np.mean(x)
  s='均值:'+format(M,'.2f')
  mTxt4.setValueString(s)
```

```
  kk=(np.sum(x**4)/N)/pow(R,4)
  s='峭度:'+format(kk,'.2f')
  mTxt5.setValueString(s)

#==Main Window=====================
win= tk.Tk()
winWidthCur=1100
winHeightCur=670
win.geometry("%dx%d" %(winWidthCur,winHeightCur))
win.config(bg="#ddeeee")
#==Layout of controls=====================
mLab1=dr.DRLabel(win,20,30,400,40,'#003355','#ffffff','典型信号波形分析',20)
mPlotWave=dr.DRPlot(win,20,80,900,510,'','#003355','#000000','#ffffff','#00eeff','#ffff00',0,0.05,-1,1,0,0)
mLab2=dr.DRLabel(win,20,587,900,50,'#003355','#ffffff','',20)
mVButType=dr.DRRadioBut(win,940,80,120,140,'#003355','#005577',"#ffff00",'正弦波,方波,三角波,白噪声',12,0,0)
mKnobA=dr.DRKnob(win,930,280,130,130,'#004466','#222222','#aaaaaa','0,1',0,1,0.5,11)
mKnobFre=dr.DRKnob(win,930,400,130,130,'#004466','#222222','#aaaaaa','1,1000',1,1000,50,11)
mSwitch1=dr.DRSwitch(win,960,600,80,36,'#004466','#aaddee')
mTxt=dr.DRLabel(win,60,595,130,30,'#000000','#00ffaa','0',14)
mTxt1=dr.DRLabel(win,200,595,130,30,'#000000','#00ffaa','0',14)
mTxt2=dr.DRLabel(win,350,595,130,30,'#000000','#00ffaa','0',14)
mTxt3=dr.DRLabel(win,490,595,130,30,'#000000','#00ffaa','0',14)
mTxt4=dr.DRLabel(win,630,595,130,30,'#000000','#00ffaa','0',14)
mTxt5=dr.DRLabel(win,770,595,130,30,'#000000','#00ffaa','0',14)
#============================================================
mType=0; Fs=44100; N=4096; A=0.6; Fre=100; Pha=0
mSignal=dsp.DRGenerator(mType,Fs,N,A,Fre,Pha)
#==DataStream Binding=====================
mSwitch1.addCallBackSingle(mSignal.setValueSingle)
mSignal.addCallBack2D(setValue2D)
mVButType.addCallBackSingle(mSignal.setVSType)
mKnobA.addCallBackSingle(mSignal.setVSAmp)
mKnobFre.addCallBackSingle(mSignal.setVSFre)
mSignal.setVSPha(0)
#==Main Loop=====================
win.mainloop()
```

音频文件信号波形分析实验

机器工作时产生的振动噪声信号，人的语音信号，以及自然界的鸟鸣声等等，是常用的信号分析数据源；而 WAV/MP3 音频文件是最常用保存音频信号的文件格式。因此，掌握从音频文件中获取音频信号数据是非常必要的。

实验设计

利用 Librosa 音频信号分析库的读文件函数，打开 WAV/MP3 音频文件，获取采样频率和音频信号数组，为简化分析，建议采用强制单通道转换模式读取音频数据。

data,Fs=librosa.load(mFileName,mono=True);

读取完整个音频文件后，再用滚动条从中选取一段信号进行音频信号的波形分析。因为一个音频文件中不同时段的音频信号特征通常是不一样的。

音频文件信号波形分析实验样例代码

```
import tkinter as tk
from tkinter import filedialog
from scipy.fftpack import fft,ifft
import matplotlib.pyplot as plt
from pylab import mpl
from matplotlib import cm
import numpy as np
import librosa
import drvi.drviDSP as dsp
import drvi.drviControlls as dr
import drvi.drviSimpleAudio as sa
import drvi.drviAppendPath as mPath

def setValueSingle(v):
    global Fs,N,CH,data
    mfile=filedialog.askopenfilename(title="选择文件",filetypes=(("MP3 files", "*.mp3"),("WAV files", "*.wav"),("all files", "*.*")))
    if not mfile: return 1
    openFile(mfile)

def openFile(mfile):
    global Fs,N,data
    data,Fs=librosa.load(mfile,mono=True,sr=None)
    N=len(data);T=N/Fs
    mPlotWave.setValue2DX(1/Fs,data)
    mPlotWave.setXlim(0,T)
    mTxt1.setValueString('')
    myWorking()

def setLength(v):
    global mLength
    mLength==2**v*1024
```

```
    myWorking()

def setPos(v):
    global mPosition
    mPosition=v
    myWorking()

def setMove(v):
    global N,mLength,mPosition,mHSlider1
    if N==0: return;
    if v==201: d=-mLength/6
    if v==202: d=mLength/6
    d=100*d/N
    mPosition=mPosition+d
    if mPosition<0: mPosition=0
    if mPosition>100: mPosition=100
    mHSlider1.setValueSingle(mPosition)
    myWorking()

def myWorking():
    global Fs,N,CH,data,xdata,mPosition,mLength
    if N==0: return;
    K=int(mPosition*0.01*N)
    if K<0: K=0
    xdata=data[K:K+mLength]
    mPlotWave1.setValue2DX(1/Fs,xdata)
    T=mLength/Fs
    mPlotWave1.setXlim(0,T)

def setPlay(v):
    global Fs,N,data,pObj;
    if N==0: return;
    if v==0: pObj=sa.play(1,Fs,data,0)
    if v==1: pObj.stop()

def setAmplify(v):
    global Fs,N,CH,data,xdata,mPosition,mLength
    s=''
    T=dsp.findPeriod(Fs,len(xdata),xdata)
    if T>0: s=s+'频率='+format(1/T,'.2f')+'Hz'
    pp=np.max(xdata)-np.min(xdata)
    s=s+',  峰峰值:'+format(pp,'.2f')
    R=np.sqrt(np.sum(xdata**2)/len(xdata))
    s=s+',  有效值:'+format(R,'.2f')
    std=np.std(xdata)
    s=s+',  方差:'+format(std,'.2f')
    M=np.mean(xdata)
    s=s+',  均值:'+format(M,'.2f')
    kk=(np.sum(xdata**4)/len(xdata))/pow(R,4)
    s=s+',  峭度:'+format(kk,'.2f')
```

```
    mTxt1.setValueString(s)

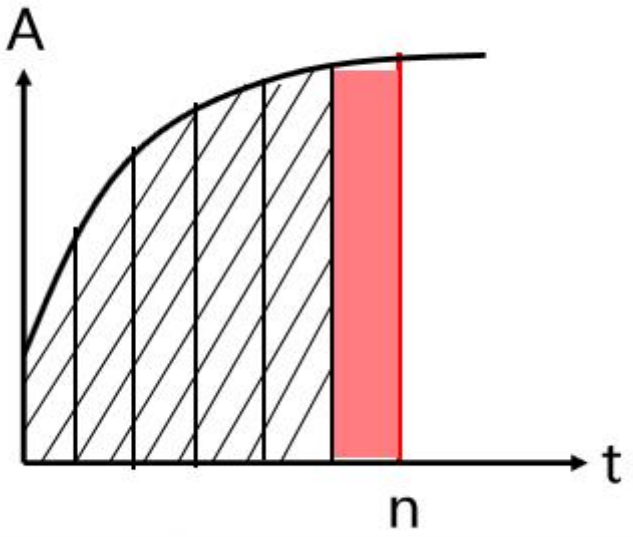
def setTime(v):
    global Fs,N,CH,data,xdata,mPosition,mLength,pObj;
    if Fs==0:
        mTimer2.setValueSingle(1)
        return;
    T=N/Fs
    v=v/1000
    if v>T:
        pObj.stop()
        mTimer2.setValueSingle(1)
    mPosition=100*v/T
    mHSlider1.setValueSingle(mPosition)
    myWorking()

#==Main Window=======================
win= tk.Tk()
win.geometry("1100x670")
win.config(bg="#ddeeee")
#==Layout of controls=======================
mL1=dr.DRLabel(win,20,35,500,40,'#440000','#ffffff','WAV 文件信号波形浏览与分析',20)
mBut1=dr.DRButton(win,660,40,100,36,'#003355','#eeee00','打开文件',12,101)
mOMenu1=dr.DROptionMenu(win,770,40,100,36,'#003355','#eeee00','1024,2048,4096,8192,16384,32768,64535',11,3)
mPlotWave=dr.DRPlot(win,20,130,1060,250,ylow=-1,yhigh=1)
mPlotWave1=dr.DRPlot(win,20,380,1060,250,ylow=-1,yhigh=1)
mHSlider1=dr.DRHSlider(win,20,90,1080,30,'#444444','#004466','#008822',8,20,0,100,0)
mHBut5=dr.DRHButtonGroup(win,880,40,200,36,'#003355','#ffffff','播放,停止',12,0)
mHBut5.addCallBackSingle(setPlay)
mBut9=dr.DRButton(win,900,370,60,26,'#005533','#ffffff','计算',12,400)
mTxt1=dr.DRLabel(win,80,400,700,30,'#000000','#00ffaa','',12)
mTimer2=dr.DRTimer(win,580,40,50,36,'#000000','#ffff00',100,11)
mTimer2.addCallBackSingle(setTime)
mHBut5.addCallBackSingle(mTimer2.setValueSingle)
mOMenu1.addCallBackSingle(setLength)
mBut1.addCallBackSingle(setValueSingle)
mHSlider1.addCallBackSingle(setPos)
mBut9.addCallBackSingle(setAmplify);
#====================================
mPosition=0
mLength=8192
openFile(mPath.getDRVIPath()+'\\dataset\\BGBird.mp3')
#==Main Loop=======================
win.mainloop()
```

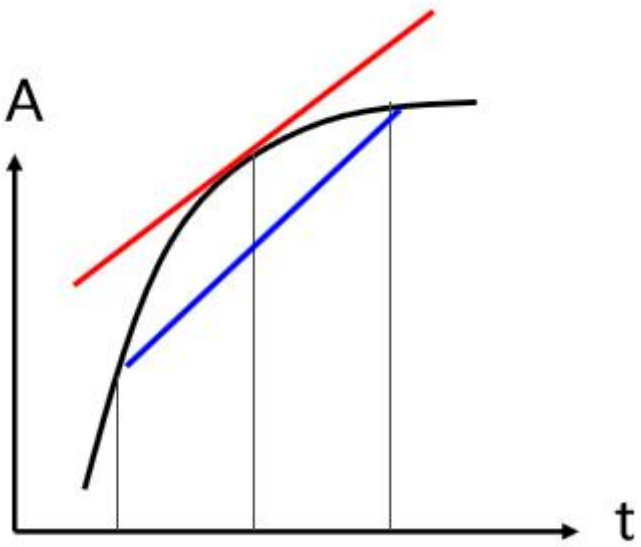
典型信号的微分与积分实验

在振动信号测量的位移、速度、加速度信号转换，以及 PID 控制等应用场合，常常需要对测量信号进行微分或积分处理。

对采样信号来说，信号的积分等于从 0 时刻开始到 t 时刻的连续累加面积输出。也就是当前点信号的积分值等于前一点的积分值+新增采样后的面积值。



而对采样信号来说，其微分则可以近似用信号 x(k)在(k-1)点和(k+1)点幅值连线的斜率来近似表示。



实验设计

用累加函数 cumsum 来近似数字积分(矩形法)，用差分函数 diff 来近似实现信号的微分。注意，积分会带来积分常数问题，通常取 0；另外，信号差分处理后数据长度会减少 1 个点。

典型信号的微分与积分实验样例代码

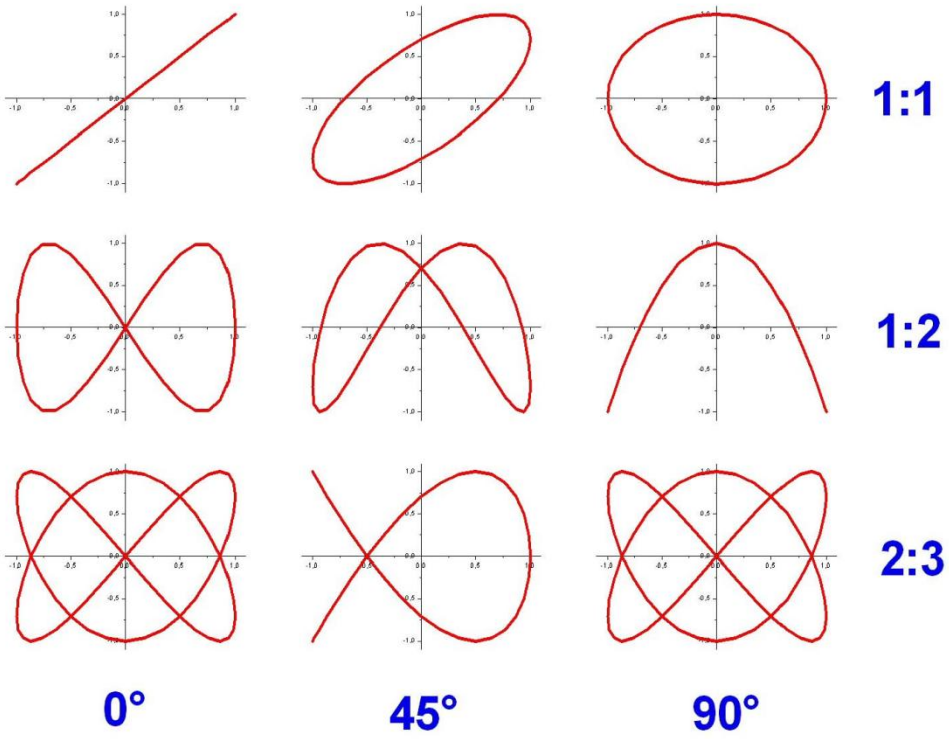
```
import tkinter as tk
import numpy as np
from scipy.fftpack import fft,ifft
import matplotlib.pyplot as plt
```

```
import drvi.drviDSP as dsp
import drvi.drviControlls as dr
#=======================================
def setValue2D(t,x):
  Fs=1/(t[1]-t[0]);
  dt=1/Fs
  N=len(x)
  y=dt*np.cumsum(x)
  y=y-np.mean(y)
  z=np.diff(y)/dt
  msubPlot.plot(1,t,x)
  msubPlot.plot(2,t,y)
  msubPlot.plot(3,t[0:N-1],z)
  msubPlot.draw()

#==Main Window=====================
win= tk.Tk()
winWidthCur=1100
winHeightCur=750
win.geometry("%dx%d" %(winWidthCur,winHeightCur))
win.config(bg="#ddeeee")
#==Layout of controls=====================
mLab1=dr.DRLabel(win,20,30,400,40,'#003355','#ffffff','典型信号的积分与微分',20)
msubPlot=dr.DRSubPlots(win,20,90,900,630,'#003355','#000000',"#ffffff","#00eeff","#ffff00",3,1)
mVButType=dr.DRRadioBut(win,940,90,120,140,'#003355','#005577',"#ffff00",'正弦波,方波,三角波,白噪声',12,0,0)
mKnobA=dr.DRKnob(win,930,260,130,130,'#004466','#222222','#aaaaaa','0,1',0,1,0.5,11)
mKnobFre=dr.DRKnob(win,930,380,130,130,'#004466','#222222','#aaaaaa','1,1000',1,1000,50,11)
mSwitch1=dr.DRSwitch(win,960,660,80,36,'#004466','#aaddee')
mType=0; Fs=44100; N=3000; A=0.6; Fre=100; Pha=0
mSignal=dsp.DRGenerator(mType,Fs,N,A,Fre,Pha)
#==DataStream Binding=====================
mSwitch1.addCallBackSingle(mSignal.setValueSingle)
mSignal.addCallBack2D(setValue2D)
mVButType.addCallBackSingle(mSignal.setVSType)
mKnobA.addCallBackSingle(mSignal.setVSAmp)
mKnobFre.addCallBackSingle(mSignal.setVSFre)
mSignal.setVSPha(0)
#==Main Loop=====================
win.mainloop()
```

李沙育图实验

将两个正弦信号分别加至示波器的 x 轴输入端和 y 轴输入端，则示波器将显示一个合成曲线，这个曲线就是李沙育图形。李沙育图形随两个输入信号的频率和相位不同，所呈现的波形也不同。当两个信号相位差为 90°时，合成图形为圆；当两个信号相位差为 0°时，合成图形为直线。



实验设计

分别生成 2 个频率和相位可用 GUI 控件调节的正弦波信号，作为 Plot 二维曲线的 X 和 Y 坐标数据进行显示。

李沙育图实验样例代码

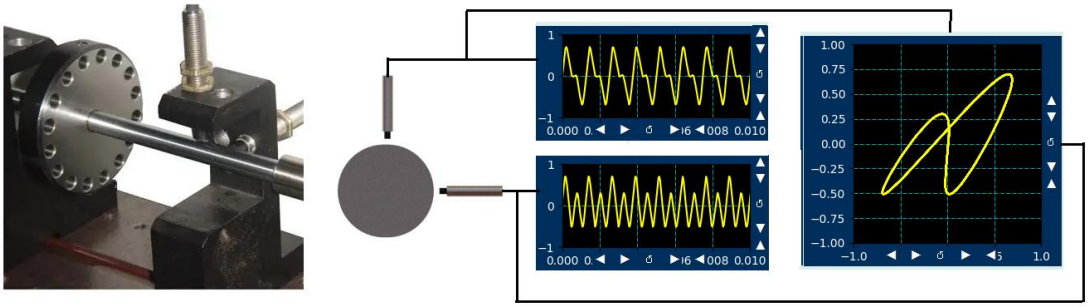
```
import tkinter as tk
import numpy as np
from scipy.fftpack import fft,ifft
import matplotlib.pyplot as plt
import drvi.drviDSP as dsp
import drvi.drviControlls as dr
#==CallBack Function==============
def myWorking(v):
  pha=mHSlider1.getValueSingle()
  rY=mHBut1.getValueSingle()+1
  rX=mHBut2.getValueSingle()+1
  Fs=44100;
  dt=1.0/Fs;
  N=500;
  tData=np.arange(N)*dt
  pa=10*mTimer1.getValueSingle()/250
  xData=0.8*np.sin(2*np.pi*800*rX*tData+(pa+pha)*np.pi/180)
  xData1=0.8*np.cos(2*np.pi*800*rY*tData+pa*np.pi/180)
  mPlotWave1.setValue2D(tData,xData)
  mPlotWave2.setValue2D(tData,xData1)
```

```
  mPlotOrbit.setValue2D(xData,xData1)
  return 1

#==Main Window=====================
win= tk.Tk()
winWidthCur=1100
winHeightCur=670
win.geometry("%dx%d" %(winWidthCur,winHeightCur))
win.config(bg="#ddeeee")
#==Layout of controls=====================
mTimer1=dr.DRTimer(win,300,30,80,32,'#000000','#ffffff',250,11)
mL1=dr.DRLabel(win,20,25,380,40,'#440000','#ffffff','信号的李沙育图形分析',20)
mPlotWave1=dr.DRPlot(win,20,90,480,270,'','#002f5e','#000000','#ffffff','#00eeff','#ffff00',0,0.01,-1,1,0,0)
mPlotWave2=dr.DRPlot(win,20,360,480,270,'','#002f5e','#000000','#ffffff','#00eeff','#ffff00',0,0.01,-1,1,0,0)
mPlotOrbit=dr.DRPlot(win,500,90,560,540,'','#002f5e','#000000','#ffffff','#00eeff','#ffff00',-1,1,-1,1,0,0)
mSwitch1=dr.DRSwitch(win,980,40,65,32,'#004466','#aaddee')
mHBut1=dr.DRHButtonGroup(win,450,52,200,25,'#004466','#ffffff','Y1,Y2,Y3,Y4,Y5,Y6',11,0)
mHBut2=dr.DRHButtonGroup(win,450,25,200,25,'#004466','#ffffff','X1,X2,X3,X4,X5,X6',11,0)
mHSlider1=dr.DRHSlider(win,670,40,280,30,'#444444','#004466','#008822',8,20,0,180,0)
#==Binding=====================================================
mSwitch1.addCallBackSingle(mTimer1.setValueSingle)
mTimer1.addCallBackSingle(myWorking)
mHBut1.addCallBackSingle(myWorking)
mHBut2.addCallBackSingle(myWorking)
mHSlider1.addCallBackSingle(myWorking)
#==============================
myWorking(0)
#==Main Loop=====================
win.mainloop()
```

旋转机械轴心轨迹模拟实验

对旋转机械来说，当存在转子质量不平衡时，振动位移信号中将存在 1 倍频信号，当存在轴不对中时，则存在 2 倍频信号。如果将 2 个振动位移传感器成 90° 角安装，则可以通过李沙育图来诊断旋转机械的运行状况。



实验设计

按如下公式生成 2 个幅值、频率、相位可变的仿真信号，用于仿真含 1 倍频和 2 倍频分量的旋转机械振动位移信号：

x(t)=a11*sin(2pi*F*t+p11)+a12*sin(2pi*2F*t+p12)

y(t)=a21*sin(2pi*F*t+p21)+a22*sin(2pi*2F*t+p22)

作为 X，Y 数据输入到二维曲线控件中进行显示，就可以模拟出各种不同状态下的轴心轨迹曲线。

旋转机械轴心轨迹模拟实验样例代码

```
import tkinter as tk
import numpy as np
from scipy.fftpack import fft,ifft
import matplotlib.pyplot as plt
import drvi.drviDSP as dsp
import drvi.drviControlls as dr

#==CallBack Function==============
def myWorking(v):
 A1=mK1.getValueSingle()
 Q1=mK2.getValueSingle()
 A2=mK3.getValueSingle()
 Q2=mK4.getValueSingle()
 A3=mK5.getValueSingle()
 Q3=mK6.getValueSingle()
 A4=mK7.getValueSingle()
 Q4=mK8.getValueSingle()
 Fs=44100;
 dt=1.0/Fs;
 N=500;
 tData=np.arange(N)*dt
 x11=A1*np.sin(2*np.pi*800*tData+Q1*np.pi/180)
 x12=A2*np.sin(2*np.pi*2*800*tData+Q2*np.pi/180)
 x=x11+x12
 mPlotWave1.setValue2D(tData,x)
 y11=A3*np.sin(2*np.pi*800*tData+Q3*np.pi/180)
 y12=A4*np.sin(2*np.pi*2*800*tData+Q4*np.pi/180)
 y=y11+y12
 mPlotWave2.setValue2D(tData,y)
 mPlotOrbit.setValue2D(x,y)
 return 1
```

```
#==Main Window=====================
win= tk.Tk()
winWidthCur=1100
winHeightCur=670
win.geometry("%dx%d" %(winWidthCur,winHeightCur))
win.config(bg="#ddeeee")
#==Layout of controls=====================
mL1=dr.DRLabel(win,20,20,420,40,'#440000','#ffffff','旋转机械的轴心轨迹曲线模拟',20)
mPlotWave1=dr.DRPlot(win,20,70,520,220,'','#002f5e','#000000','#ffffff','#00eeff','#ffff00',0,0.01,-1,1,0,0)
mPlotWave2=dr.DRPlot(win,20,290,520,220,'','#002f5e','#000000','#ffffff','#00eeff','#ffff00',0,0.01,-1,1,0,0)
mPlotOrbit=dr.DRPlot(win,540,70,520,440,'','#002f5e','#000000','#ffffff','#00eeff','#ffff00',-1,1,-1,1,0,0)
mK1=dr.DRKnob(win,40,525,120,120,'#004466','#222222','#aaaaaa','0,1',0,1,0.5,11)
mK2=dr.DRKnob(win,150,525,120,120,'#006644','#222222','#aaaaaa','0,180',0,180,0,11)
mK3=dr.DRKnob(win,260,525,120,120,'#004466','#222222','#aaaaaa','0,1',0,1,0.3,11)
mK4=dr.DRKnob(win,370,525,120,120,'#006644','#222222','#aaaaaa','0,180',0,180,0,11)
mK5=dr.DRKnob(win,590,525,120,120,'#004466','#222222','#aaaaaa','0,1',0,1,0.2,11)
mK6=dr.DRKnob(win,700,525,120,120,'#006644','#222222','#aaaaaa','0,180',0,180,45,11)
mK7=dr.DRKnob(win,810,525,120,120,'#004466','#222222','#aaaaaa','0,1',0,1,0.5,11)
mK8=dr.DRKnob(win,920,525,120,120,'#006644','#222222','#aaaaaa','0,180',0,180,0,11)
#==Binding=====================
mK1.addCallBackSingle(myWorking)
mK2.addCallBackSingle(myWorking)
mK3.addCallBackSingle(myWorking)
mK4.addCallBackSingle(myWorking)
mK5.addCallBackSingle(myWorking)
mK6.addCallBackSingle(myWorking)
mK7.addCallBackSingle(myWorking)
mK8.addCallBackSingle(myWorking)
myWorking(0)
#==Main Loop=====================
win.mainloop()
```

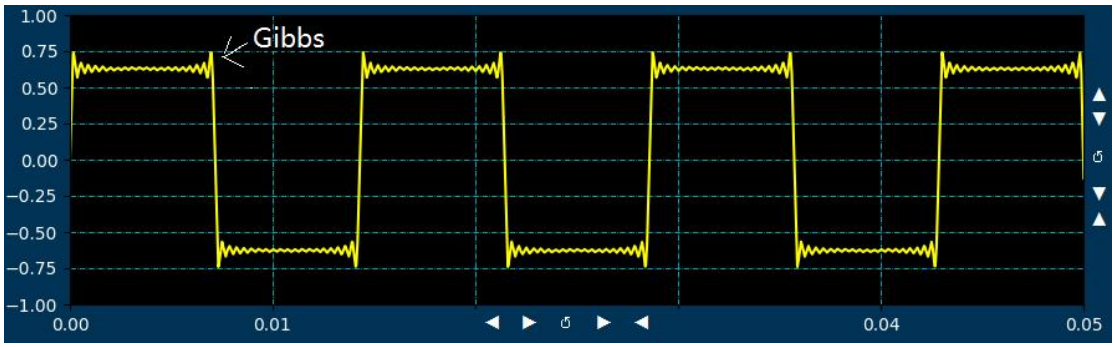
方波信号的合成与吉普斯现象实验

按傅里叶级数公式，方波可以展开为一组频率比为 1，3，5，7，...，幅值比为 1，1/3，1/5，1/7，...，的正弦波信号的合成；也就是说可以用一组正弦波信号来合成方波信号。

$$x(t)=\sum_{n=1}^{\infty}\frac{4}{\pi}A\sin(2\pi f_0 t)+\frac{4A}{3\pi}\sin(3*2\pi f_0 t)+\frac{4A}{5\pi}\sin(5*2\pi f_0 t)+.....$$

方波信号合成的吉普斯现象：

受采样定理限制，方波合成时只能取符合采样定理的前 N 项正弦波信号进行合成。因此，必须对方波信号的合成公式进行有限项截断，从而造成用有限项正弦波信号来合成方波。这时合成出的方波信号的幅值拐点处会出现振荡现象，这称之为吉普斯现象。



实验设计：

按方波合成公式，用前 1 项、前 3 项、前 8 项等不同的截断条件，对方波信号进行合成，观察合成信号中的吉普斯现象。

方波信号的合成与吉普斯现象实验样例代码

```
import tkinter as tk
import numpy as np
from scipy.fftpack import fft,ifft
import matplotlib.pyplot as plt
import drvi.drviDSP as dsp
import drvi.drviControlls as dr

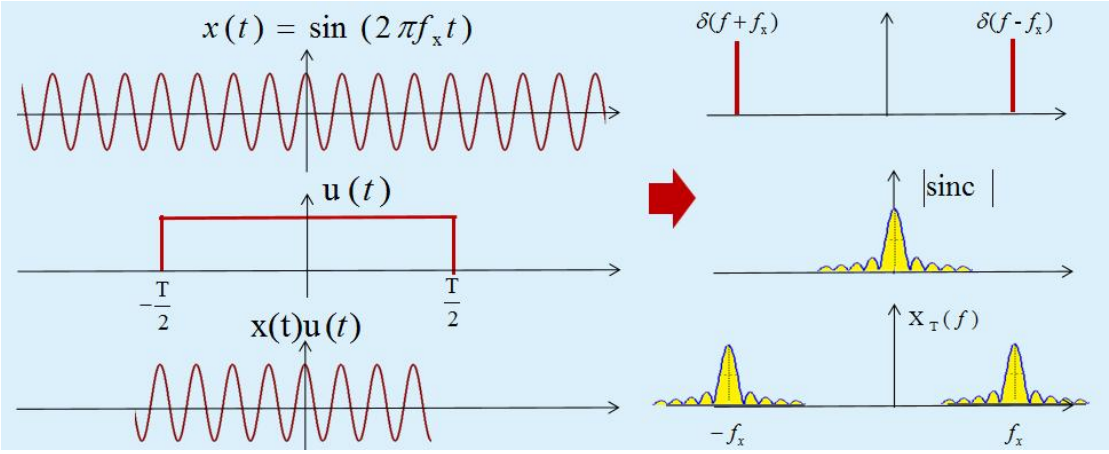
#=========================================
def setValue(v):
  Fs=32768;
  dt=1.0/Fs;
  N=32768;
  F=70
  t=np.arange(N)*dt
  x=0*t
  if v==0: M=1
  if v==1: M=2
```

```
  if v==2: M=3
  if v==3: M=4
  if v==4: M=5
  if v==5: M=6
  if v==6: M=7
  if v==7: M=8
  if v==8: M=9
  if v==9: M=10
  if v==10: M=20
  if v==11: M=40
  for i in range(M):
        k=1+2*i; a=1/k
        x=x+a*0.8*np.sin(2*np.pi*k*F*t)
  mPlotWave.setValue2D(t,x)
  [mFre,mAmp]=dsp.AmpSpetrum(N,Fs,x)
  mPlotAmp.setValue2D(mFre,mAmp)

#==Main Window=====================
win= tk.Tk()
winWidthCur=1100
winHeightCur=670
win.geometry("%dx%d" %(winWidthCur,winHeightCur))
win.config(bg="#ddeeee")
#==Layout of controls=====================
mLab1=dr.DRLabel(win,20,30,400,40,'#003355','#ffffff','方波的合成与吉普斯现象',20)
mPlotWave=dr.DRPlot(win,20,80,900,270,'','#003355','#000000','#ffffff','#00eeff','#ffff00',0,0.05,-1,1,0,0)
mPlotAmp=dr.DRPlot(win,20,350,900,270,'','#003355','#000000','#ffffff','#00eeff','#ffff00',0,4000,0,1,0,0)
mVButType=dr.DRRadioBut(win,930,80,120,540,'#003355','#005577',"#ffff00",'1 阶,前 2 阶,前 3 阶,前 4 阶,前 5 阶,前 6 阶,前 7 阶,前 8 阶,前 9 阶,前 10 阶,前 20 阶,前 40 阶',12,0,0)
#==================================
mVButType.addCallBackSingle(setValue)
setValue(0)
#==Main Loop=====================
win.mainloop()
```

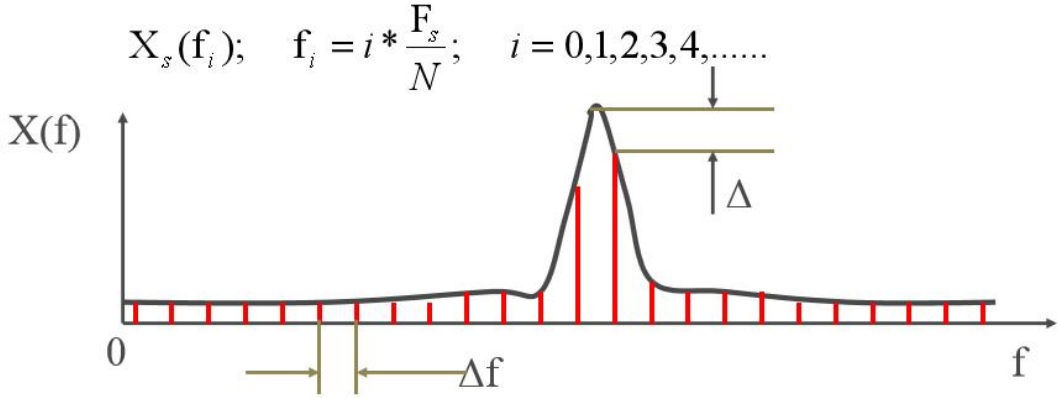
FFT 谱的能量泄漏与栅栏效应实验

为了实现傅里叶变换的数字计算，必须对原信号进行截断。从数学上看，相当于用一个矩形窗与原信号相乘。按傅里叶变换性质，时域相乘等价于频域卷积。假如待分析信号是一个频率 f 的正弦波信号，其频谱是 2 个分别位于 f 和-f 的 δ 函数；而矩形窗的谱是一个 sinc 函数。按 δ 函数卷积定理，sinc 函数被搬移到 2 个 δ 函数的位置。因此，正弦波信号截断后，原来集中在频率点 f 和-f 的集中能量，以 f 和-f 为中心，向两边泄漏，这称之为 FFT 谱的能量泄漏。



栅栏效应

信号截断后存在能量泄漏的频谱是一个连续谱。为提高效率,通常采用 FFT 对其进行离散采样计算。设信号的数据点数为 N，采样频率为 Fs，则计算的离散频率点为:k*(Fs/N)。如果信号谱峰与频率取样点不重合，则真实谱峰的值看不见，只能取有计算值的谱线上的数值来近似代替，这称之为栅栏效应误差。



实验设计：

设采样频率 Fs=1024Hz，采样长度 N=1024，有 df=Fs/N=1，故 FFT 的谱线，也就是频率取样点位置为 1Hz，2Hz，3Hz，……，512Hz，其位置是固定的。在在 40Hz—50Hz 区间生成一个幅值为 1，频率可微调的正弦波信号，显示信号波形和 FFT 幅值谱。可以看出当微调后的正弦波信号频率为非整数时，其幅值谱高度要明显小于正弦波信号的理论幅值 1，这就是栅栏效应带来的 FFT 谱幅值误差。然后，用连续傅里叶变换公式，在 30Hz—60Hz 区间范围内，以 0.1Hz 的间隔，密集计算线数，显示该范围内由于能量泄漏所产生的真实频谱。最后，对比显示出 FFT 谱线位置与带能量泄漏的频谱对应，从而了解 FFT 谱线所对应的能量泄漏谱的位置。

FFT 谱的能量泄漏与栅栏效应实验样例代码

```
import tkinter as tk
import numpy as np
from scipy.fftpack import fft,ifft
import drvi.drviDSP as dsp
import drvi.drviControlls as dr
import math

#==CFT============================
def CFT(Fs,N,x,f):
  dt=1.0/Fs; am=0; bm=0;
  kk=np.arange(N)*2*np.pi*f*dt;
  mC=x*np.cos(kk);
  mS=x*np.sin(kk);
  am=np.sum(mC);
  bm=np.sum(mS);
  A=np.sqrt(am*am+bm*bm)*(2/N);
  return A

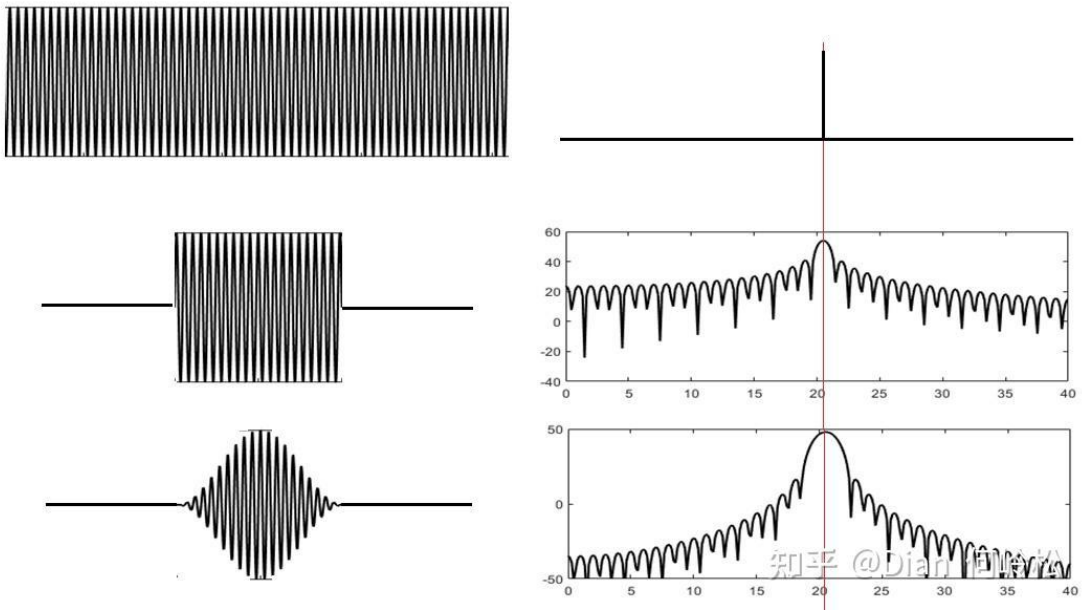
#==CallBack Function==============
def myWorking(v):
  v=mSpin1.getValueSingle()
  Fs1=1024;
  N1=1024
  dt1=1.0/Fs1;
  t1=np.arange(N1)*dt1
  x1=0.8*np.sin(2*np.pi*v*t1)
  mPlotWave1.setValue2D(t1,x1)
  kk=mOMenu1.getValueSingle()
  ww=dsp.winFunction(kk,N1)
  xx=ww*x1;
  [mFre,mAmp]=dsp.AmpSpetrum(N1,Fs1,xx)
  mPlotAmp.setValue2D(mFre,mAmp)
  ff=np.linspace(30,60,200)
  aa=np.zeros(200)
  for k in range(200):
        aa[k]=CFT(Fs1,N1,xx,ff[k])
  mPlotAmp2.setValue2D(ff,aa)
  ff1=30+np.arange(30)
  aa1=np.ones(30)
  mPlotAmp1.setValue2D(ff1,aa1)
  return 1

#==Main Window====================
win= tk.Tk()
winWidthCur=1100
winHeightCur=670
win.geometry("%dx%d" %(winWidthCur,winHeightCur))
```

```
#==Layout of controls======================
mL1=dr.DRLabel(win,20,30,500,40,'#440000','#ffffff','FFT 谱的能量泄漏与栅栏效应误差',20)
mPlotWave1=dr.DRPlot(win,20,90,1000,140,'','#113344','#000000','#ffffff','#00eeff','#ffff00',0,1,-1,1,0,0)
mPlotAmp=dr.DRPlot(win,20,225,1000,140,'','#113344','#000000','#ffffff','#00eeff','#ffff00',0,500,0,1,0,0)
mPlotAmp1=dr.DRStem(win,20,500,1000,140,'','#113344','#000000','#ffffff','#00eeff','#ffff00',30,60,0,1,0,0)
mPlotAmp2=dr.DRPlot(win,20,360,1000,140,'','#113344','#000000','#ffffff','#00eeff','#ffff00',30,60,0,1,0,0)
mL2=dr.DRLabel(win,560,40,140,40,'#ddeeee','#000000','信号频率：',18)
mDigital1=dr.DRDigital(win,700,40,120,40,'#000000','#00ffaa',45,2,22)
mSpin1=dr.DRSpinBox(win,880,40,100,36,'#ffffff','#000000',45,0.1,40,50,11)
mL3=dr.DRLabel(win,740,500,200,24,'#441100','#ffffff','df=1024Hz/1024',12)
mOMenu1=dr.DROptionMenu(win,880,380,100,26,txt='矩形,三角,汉宁,海明,BlackMan,平顶')
#==DataStream Binding======================
mSpin1.addCallBackSingle(mDigital1.setValueSingle)
mSpin1.addCallBackSingle(myWorking)
mOMenu1.addCallBackSingle(myWorking)
myWorking(45)
#==Main Loop======================
win.mainloop()
```

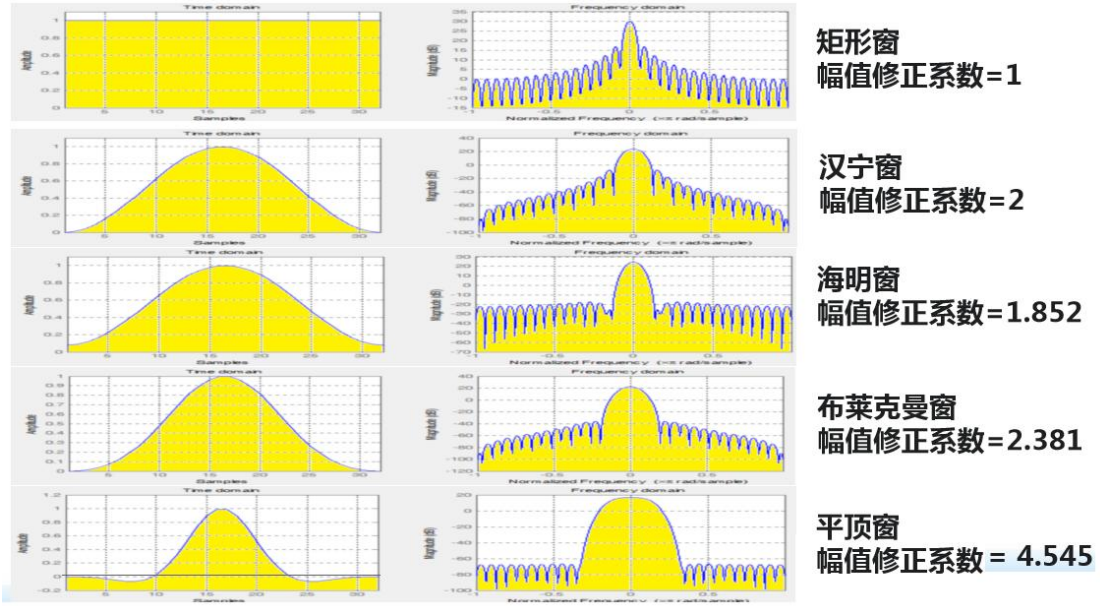
窗函数的作用实验

FFT 谱分析时的所谓加窗，就是用选定的窗函数对信号进行相乘、截断，这样频谱的能量泄漏形状就由不加窗时对应的矩形窗，变为选定窗函数的能量泄漏形状。例如，下面分别是加矩形窗截断和加汉宁窗截断时的情形。可以看出加汉宁窗截断时能量泄漏的主瓣明显的比加矩形窗截断时的宽和平坦，因此栅栏效应带来的误差就得到了一定的抑制。



实验设计：

设采样频率 Fs=1024Hz，采样长度 N=1024，有 df=Fs/N=1，故 FFT 的谱线，也就是频率取样点位置为 1Hz，2Hz，3Hz，……，512Hz，其位置是固定的。在在 40Hz—50Hz 区间生成一个幅值为 1，频率可微调的正弦波信号，显示信号波形和 FFT 幅值谱。可以看出当微调后的正弦波信号频率为非整数时，其幅值谱高度要明显小于正弦波信号的理论幅值 1，这就是栅栏效应带来的 FFT 谱幅值误差。然后，用不同的窗函数对信号进行截断，观察窗函数对栅栏效应误差的修正作用。注意，信号加窗后计算得到的 FFT 谱需进行幅值修正。



窗函数的作用实验样例代码

```
import tkinter as tk
import numpy as np
from scipy.fftpack import fft,ifft,fftshift
from scipy import signal
import matplotlib.pyplot as plt
import drvi.drviDSP as dsp
import drvi.drviControlls as dr

#==CallBack Function===============
def setWin(v):
  winT=v; N1=51
  w=np.ones(N1);
  if winT==0: w=np.ones(N1);
  if winT==1: w=signal.windows.bartlett(N1);
  if winT==2: w=signal.windows.hann(N1);
  if winT==3: w=signal.windows.hamming(N1);
  if winT==4: w=signal.windows.blackman(N1);
  if winT==5: w=signal.windows.flattop(N1);
  mPlotWinWave.setValue2DX(1,w)
  A=fft(w,2048)/(len(w)/2.0)
  freq=np.linspace(-0.5, 0.5,len(A))
  response = 20 * np.log10(np.abs(fftshift(A / abs(A).max())) + 0.000000000000001 )
  mPlotWinAmp.setValue2D(freq, response)
  return 1

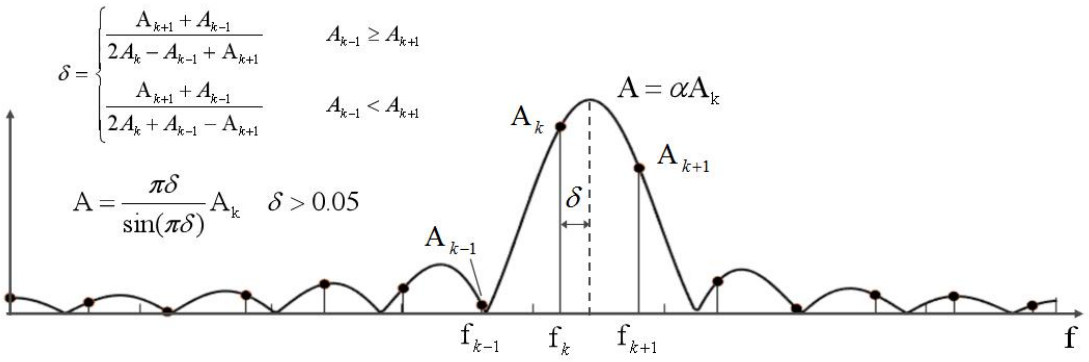
#==================================
def myWorking(v):
  global Fre,winT,winA
  if v<10: winT=v;
  if v>10: Fre=v;
  Fs1=1024;
  N1=1024
  dt1=1.0/Fs1;
  t1=np.arange(N1)*dt1
  w=np.ones(N1)
  x1=0.8*np.sin(2*np.pi*Fre*t1)
  mPlotWave1.setValue2D(t1,x1)
  [mFre,mAmp]=dsp.AmpSpetrum(N1,Fs1,x1)
  mPlotAmp.setValue2D(mFre,mAmp)
  if winT==0: w=signal.windows.bartlett(N1); winA=2
  if winT==1: w=signal.windows.hann(N1);          winA=2
  if winT==2: w=signal.windows.hamming(N1);    winA=1.852
  if winT==3: w=signal.windows.blackman(N1); winA=2.381
  if winT==4: w=signal.windows.flattop(N1);    winA=4.545
  y=x1*w
  mPlotWave2.setValue2D(t1,y)
  [mFre1,mAmp1]=dsp.AmpSpetrum(N1,Fs1,y)
  mAmp1=mAmp1*winA
```

```
 mPlotAmp1.setValue2D(mFre1,mAmp1)
 return 1

#==Main Window=====================
win= tk.Tk()
winWidthCur=1100
winHeightCur=670
win.geometry("%dx%d" %(winWidthCur,winHeightCur))
win.config(bg="#ddeeee")
#==Layout of controls=====================
mL1=dr.DRLabel(win,20,30,400,40,'#440000','#ffffff','谱窗函数的作用',20)
mTab=dr.DRNoteBook(win,20,90,1040,540)
#Pasge1
mPlotWave1=dr.DRPlot(mTab.win[0],0,0,520,250,'','#113344','#000000','#ffffff','#00eeff','#ffff00',0,1,-1,1,0,0)
mPlotWave2=dr.DRPlot(mTab.win[0],520,0,520,250,'','#113344','#000000','#ffffff','#00eeff','#ffff00',0,1,-1,1,0,0)
mPlotAmp=dr.DRPlot(mTab.win[0],0,250,520,250,'','#113344','#000000','#ffffff','#00eeff','#ffff00',0,500,0,1,0,0)
mPlotAmp1=dr.DRPlot(mTab.win[0],520,250,520,250,'','#113344','#000000','#ffffff','#00eeff','#ffff00',0,500,0,1,0,0)
#Page2
mPlotWinWave=dr.DRPlot(mTab.win[1],0,0,520,495,'','#113344','#000000','#ffffff','#00eeff','#ffff00',0,0,-1.2,1.2,0,0)
mPlotWinAmp=dr.DRPlot(mTab.win[1],520,0,520,495,'','#113344','#000000','#ffffff','#00eeff','#ffff00',0,0,0,0,0,0)
#==============
mL2=dr.DRLabel(win,660,40,140,40,'#ddeeee','#000000','信号频率：',18)
mDigital1=dr.DRDigital(win,800,40,120,40,'#000000','#00ffaa',45,2,22)
mSpin1=dr.DRSpinBox(win,960,40,100,36,'#ffffff','#000000',45,0.1,40,50,11)
mHBut1=dr.DRHButtonGroup(win,20,615,1040,36,'#113344','#ffffff','三角窗,汉宁窗,海明窗,BlackMan 窗,平顶窗',11,0)
#==DataStream Binding=====================
mSpin1.addCallBackSingle(mDigital1.setValueSingle)
mSpin1.addCallBackSingle(myWorking)
mHBut1.addCallBackSingle(myWorking)
mHBut1.addCallBackSingle(setWin)
#============================================
Fre=45
winT=0
winA=1
myWorking(45)
setWin(0)
#==Main Loop=====================
win.mainloop()
```

FFT 谱栅栏效应误差插值修正实验

对矩形窗截断(其它窗函数算法类似)的频谱分析来书，其能量泄漏形状是一个 sinc 函数。由于函数形状是确定的，当满足局部为单一频率成分的条件时，只要将峰值点位置周边的三条谱线的值代入 sinc 函数，求解方程，就可以插值求出峰值点的频率修正量 δ 和幅值修正系数 α，得到谱峰幅值和频率的估计值。然后将最大谱线对应的频率和幅值用修正值后的值替换，这样就可完成对 FFT 谱计算结果的频率和幅值插值修正。这称之为 FFT 谱的插值修正算法。



实验设计：

设采样频率 Fs=1024Hz，采样长度 N=1024，有 df=Fs/N=1，故 FFT 的谱线，也就是频率取样点位置为 1Hz，2Hz，3Hz，……，512Hz，其位置是固定的。在在 40Hz—50Hz 区间生成一个幅值为 1，频率可微调的正弦波信号，显示信号波形和 FFT 幅值谱。可以看出当微调后的正弦波信号频率为非整数时，其幅值谱高度要明显小于正弦波信号的理论幅值 1，这就是栅栏效应带来的 FFT 谱幅值误差。然后，查找谱峰，用谱峰处的 3 条谱线进行插值修正，得到修正谱。用 DRPlot 曲线控件显示插值修正谱，另外用 2 个 DRStem 二维曲线函数显示谱峰修正前和修正后的细节，以便于观察了解。



FFT 谱栅栏效应误差插值修正实验样例代码

```
import tkinter as tk
import numpy as np
from scipy.fftpack import fft,ifft
import matplotlib.pyplot as plt
```

```
import drvi.drviDSP as dsp
import drvi.drviControlls as dr

#==CallBack Function===============
def myWorking(v):
  Fs1=1024;
  N1=1024
  dt1=1.0/Fs1;
  t1=np.arange(N1)*dt1
  x1=0.8*np.sin(2*np.pi*v*t1)
  mPlotWave1.setValue2D(t1,x1)
  [mFre,mAmp]=dsp.AmpSpetrum(N1,Fs1,x1)
  mPlotAmp.setValue2D(mFre,mAmp)
  [mFre,A1]=dsp.AmpSpetrumX(N1,Fs1,x1)
  mPlotAmp1.setValue2D(mFre,A1)
  mPlotAmp0.setValue2D(mFre,A1)
  return 1

#==Main Window=======================
win= tk.Tk()
winWidthCur=1100
winHeightCur=670
win.geometry("%dx%d" %(winWidthCur,winHeightCur))
win.config(bg="#ddeeee")
#==Layout of controls=======================
mL1=dr.DRLabel(win,20,30,400,40,'#440000','#ffffff','FFT 谱的插值修正算法',20)
mPlotWave1=dr.DRPlot(win,20,90,1000,140,'','#113344','#000000','#ffffff','#00eeff','#ffff00',0,1,-1,1,0,0)
mPlotAmp0=dr.DRPlot(win,20,225,1000,140,'','#113344','#000000','#ffffff','#00eeff','#ffff00',0,500,0,1,0,0)
mPlotAmp=dr.DRStem(win,20,360,1000,140,'','#113344','#000000','#ffffff','#00eeff','#ffff00',30,60,0,1,0,0)
mPlotAmp1=dr.DRStem(win,20,500,1000,140,'','#113344','#000000','#ffffff','#00eeff','#ffff00',30,60,0,1,0,0)
mL2=dr.DRLabel(win,560,40,140,40,'#ddeeee','#000000','信号频率：',18)
mDigital1=dr.DRDigital(win,700,40,120,40,'#000000','#00ffaa',45,2,22)
mSpin1=dr.DRSpinBox(win,880,40,100,36,'#ffffff','#000000',45,0.1,40,50,11)
mL3=dr.DRLabel(win,820,255,165,24,'#441100','#ffffff','插值修正谱',12)
#==DataStream Binding=======================
mSpin1.addCallBackSingle(mDigital1.setValueSingle)
mSpin1.addCallBackSingle(myWorking)
myWorking(45)
#==Main Loop=======================
win.mainloop()
```

音频文件信号波形和频谱分析实验

机器工作时产生的振动噪声信号，人的语音信号，以及自然界的鸟鸣声等等，是常用的信号分析数据源；而 WAV/MP3 音频文件是最常用保存音频信号的文件格式。因此，掌握从音频文件中获取音频信号数据，并进行波形和频谱分析是非常必要的。



实验设计

利用 Librosa 音频信号分析库的读文件函数，打开 WAV/MP3 音频文件，获取采样频率和音频信号数组，为简化分析，建议采用强制单通道转换模式读取音频数据。

data,Fs=librosa.load(mFileName,mono=True);

读取完整个音频文件后，再从文件中获取一个信号片段进行波形和频谱分析。为简化起见，采用 DRVI 库中的 DRPlay 控件来完成音频文件的读取，播放和当前音频数据获取功能，用 DRSpectrum 控件来完成当前信号的谱分析。

音频文件信号波形和频谱分析实验样例代码

```
import tkinter as tk
import numpy as np
from scipy.fftpack import fft,ifft
import matplotlib.pyplot as plt
import drvi.drviAppendPath as mPath
import drvi.drviDSP as dsp
import drvi.drviControlls as dr
import drvi.drviSimpleAudio as ply

def setValue2DX(dx,x):
  mPlot.setValue2DXFast(dx,x)
  mFs=1/dx
  mLen=len(x)
  [mFre,mAmp]=dsp.AmpSpetrum(mLen,mFs,x)
  df=mFs/mLen
  mPlot1.setValue2DXFast(df,mAmp)
```

```
#==Main Window=====================
win= tk.Tk()
win.geometry("1100x670")
win.config(bg="#ddeeee")
#==Layout of controls=====================
mLab1=dr.DRLabel(win,20,30,400,40,'#445577','#ffffff','音频文件波形和频谱分析',20)
mHBut2=dr.DRHButtonGroup(win,450,30,200,36,'#006688','#ffffff','播放,停止,文件',12,0)
mEntry1=dr.DREntry(win,655,30,300,36,'#000000','#ffff00','',10)
mPlay=ply.DRPlay(win,960,30,80,36,'#002244','#ffffff',11)
mPlot=dr.DRPlot(win,20,260,1020,200,'','#003355','#000000','#ffffff','#00eeff','#ffff00',0,0.2,-1,1,0,0)
mPlot1=dr.DRPlot(win,20,460,1020,200,'','#003355','#000000','#ffffff','#00eeff','#ffff00',0,6000,0,0.1,0,0)
mPlot2=dr.DRPlot(win,20,80,1020,180,'','#003355','#000000','#ffffff','#00eeff','#ffff00',0,0,-1,1,0,0)
#====================================
mHBut2.addCallBackSingle(mPlay.setValueSingle)
mPlay.addCallBack2DX(setValue2DX)
mPlay.addCallBackStrFile(mEntry1.setValueString)
mPlay.addCallBack2DXAll(mPlot2.setValue2DX)
initFile=mPath.getDRVIPath()+'\\dataset\\voice1.mp3'
mEntry1.setValueString(initFile)
mPlay.setValueString(initFile)
#==Main Loop=====================
win.mainloop()
```

麦克风信号波形和频谱分析实验

在学习信号分析理论时，计算机麦克风是一个最方便的获取真实信号的来源。例如，你可以对着麦克风弹吉他，看吉他弦与弦之间的音高关系，了解吉他指板的作用。从弹奏的听觉感知，到信号波形和频谱的视觉感知，真实体验信号分析的作用



实验设计

利用 pyaudio 音频库来控制麦克风进行声音信号采集，请确已安装 pyaudio 资源包。核心代码:

```
p=pyaudio.PyAudio()
s=p.open(format= pyaudio.paInt16,channels=1,rate=Fs,input=True,frames_per_buffer=1024)
```

麦克风声音信号采集实时信号获取需要熟悉 Python 多线程编程知识。为简化起见，采用 DRVI 库中的 DRRecorder 控件来完成实时声音信号的采集和数据流推送，然后用 DRSpectrum 控件来完成当前信号的谱分析。

麦克风信号波形和频谱分析实验样例代码

```
import tkinter as tk
import matplotlib.pyplot as plt
import numpy as np
import drvi.drviDSP as dsp
import drvi.drviControlls as dr
import drvi.drviAudio as drAudio

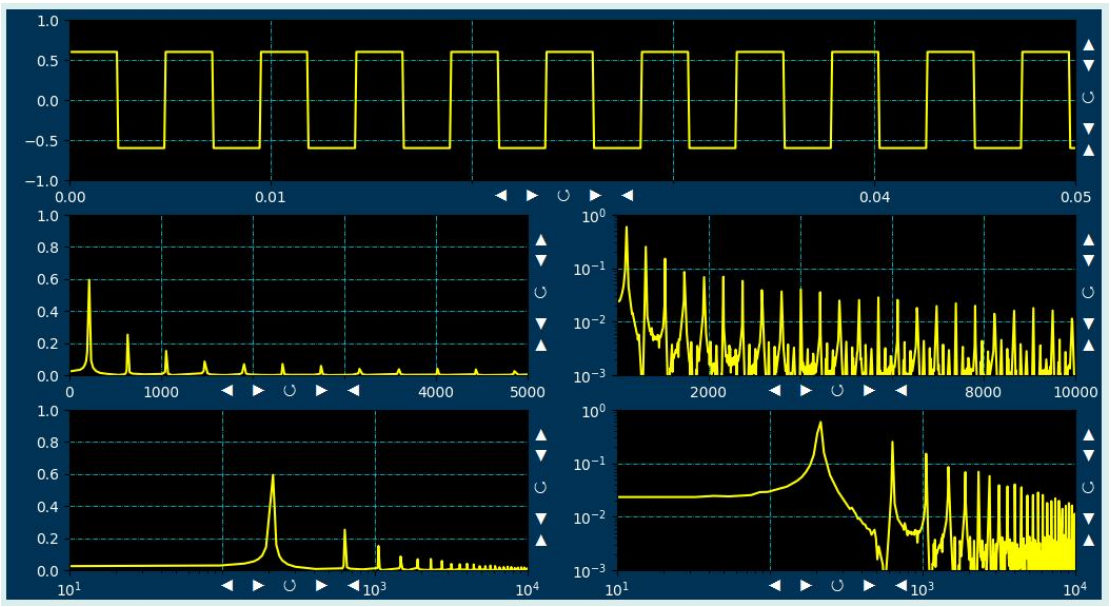
#==Event response function of controls=====
def setValue2DX(dx,x):
  mFs=1/dx
  mLen=len(x)
  [mFre,mAmp]=dsp.AmpSpetrum(mLen,mFs,x)
  t=np.arange(mLen)*dx
```

```
  msubPlot.plot(1,t,x)
  msubPlot.plot(2,mFre,mAmp)
  msubPlot.draw()

#==Main Window=======================
win= tk.Tk()
winWidthCur=1100
winHeightCur=670
win.geometry("%dx%d" %(winWidthCur,winHeightCur))
win.config(bg="#ddeeee")
#==Layout of controls=======================
mLab1=dr.DRLabel(win,20,30,380,40,'#003355','#ffffff','信号录音样例程序',20)
mHBut2=dr.DRHButtonGroup(win,690,30,200,36,'#006688','#ffffff','录音,停止',11,0)
mRec3=drAudio.DRRecorder(win,900,30,120,36,'#002244','#ffffff',44100,8192,60,11)
msubPlot=dr.DRSubPlot(win,30,80,1000,560,'#003355','#000000',"#ffffff","#00eeff","#ffff00",2,1)
msubPlot.setXlim(1,0,0.175)
msubPlot.setYlim(1,-1,1)
msubPlot.setXlim(2,0,5000)
msubPlot.setYlim(2,0,0.125)
#=========================================
mHBut2.addCallBackSingle(mRec3.setValueSingle)
mRec3.addCallBack2DX(setValue2DX)
mRec3.addCallBack2DX(setValue2DX)
#==Main Loop=======================
win.mainloop()
```

典型信号的线性谱和对数谱显示实验

信号频谱是指信号在频率域上的表示，以幅值谱为例，横坐标是频率，纵坐标是各频率点上信号的幅值。按 X、Y 坐标的线性、对数显示形式，频谱也可以分为线性谱、双对数谱等四种显示形式。注意，频谱图的数据并没有变，只是通过轴上的线性/对数刻度变化，突出的重点不同而已。



实验设计

产生正弦波、方波、三角波和白噪声等四种典型信号，然后利用 FFT 变换，将采样信号数组 x 由时间域变换到频率域，然后取取绝对值就是各频率点的幅值。核心代码:

```
import numpy as np
from scipy.fftpack import fft
fArray=np.arange(N)*(Fs/N)
yArray=np.abs(np.fft.fft(xArray))
```

然后对幅值谱进行：X 轴线性+Y 轴线性，X 轴线性+Y 轴对数，X 轴对数+Y 轴线性，X 轴对数+Y 轴对数，四种不同风格的显示。了解线性谱和对数谱的区别。

典型信号的线性谱和对数谱显示实验样例代码

```
import tkinter as tk
import numpy as np
from scipy.fftpack import fft,ifft
import matplotlib.pyplot as plt
import drvi.drviDSP as dsp
import drvi.drviControlls as dr

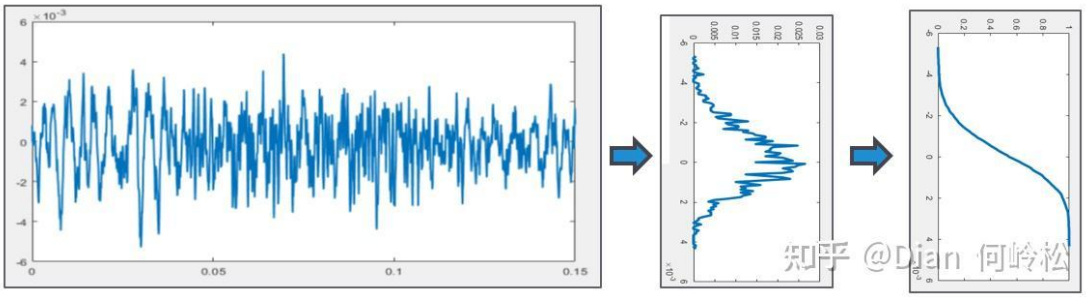
def setValue2D(t,x):
  Fs=1/(t[1]-t[0]);
  N=len(x)
  mPlotWave.setValue2D(t,x)
```

```
  [mFre,mAmp]=dsp.AmpSpetrum(N,Fs,x)
  mPlotAmp.setValue2D(mFre,mAmp)
  mPlotSemiX.setValue2D(mFre,mAmp)
  mPlotSemiY.setValue2D(mFre,mAmp)
  mPlotLoglog.setValue2D(mFre,mAmp)

#==Main Window=====================
win= tk.Tk()
winWidthCur=1100
winHeightCur=670
win.geometry("%dx%d" %(winWidthCur,winHeightCur))
win.config(bg="#ddeeee")
#==Layout of controls=====================
mType=0; Fs=44100; N=4096; A=0.6; Fre=100; Pha=0
mSignal=dsp.DRGenerator(mType,Fs,N,A,Fre,Pha)
mL1=dr.DRLabel(win,470,35,60,36,'#ddeeee','#000000','频率:',14)
mLab1=dr.DRLabel(win,20,30,430,40,'#003355','#ffffff','典型信号的线性谱和对数谱显示实验',18)
mPlotWave=dr.DRPlot(win,20,80,1040,190,'','#003355','#000000','#ffffff','#00eeff','#ffff00',0,0.05,-1,1,0,0)
mPlotAmp=dr.DRPlot(win,20,265,520,190,'','#003355','#000000','#ffffff','#00eeff','#ffff00',0,5000,0,1,0,0)
mPlotSemiX=dr.DRSemilogX(win,20,450,520,190,'','#003355','#000000','#ffffff','#00eeff','#ffff00',10,10000,0,1,0,0)
mPlotSemiY=dr.DRSemilogY(win,540,265,520,190,'','#003355','#000000','#ffffff','#00eeff','#ffff00',10,10000,0.001,1,0,0)
mPlotLoglog=dr.DRLoglog(win,540,450,520,190,'','#003355','#000000','#ffffff','#00eeff','#ffff00',10,10000,0.001,1,0,0)
mVButType=dr.DRComboBox(win,940,35,120,36,'#0099aa','#ffffff','1.正弦波,2.方波,3.三角波,4.白噪声',12,0)
mKnobFre=dr.DRHSlider(win,530,35,400,36,'#444444','#880000','#008822',8,20,1,1000,50)
#==DataStream Binding=====================
mSignal.addCallBack2D(setValue2D)
mVButType.addCallBackSingle(mSignal.setVSType)
mKnobFre.addCallBackSingle(mSignal.setVSFre)
mSignal.setVSPha(0)
#==Main Loop=====================
win.mainloop()
```

典型信号的概率密度和概率分布曲线实验

概率密度函数以幅值为横坐标，以信号中各幅值大小出现的概率密度为纵坐标，它反映了信号取不同值的可能性，对其积分则得到信号的概率分布曲线。为便于观察，通常将下图中的概率密度和概率分布曲线旋转 90 度来显示。



实验设计

产生正弦波、方波、三角波和白噪声等四种典型信号。在 Numpy 库中提供了数据集的直方图计算函数，除以数据集中的数据个数，就可得到概率密度函数。核心代码:

```
import numpy as np
M=128
minA=np.min(xArray)
maxA=np.max(xArray)
abin=np.linspace(minA,maxA,num=M)
[mHist,bins]=np.histogram(xArray,bins=abin)
mPDFarray=mHist/len(x)
mCDFarray=np.cumsum(mPDFarray)
```

为简化起见，实验中用 DRVI 库中 DRGenerator 控件来产生标准信号，用 DRPdfCdf 控件来计算概率密度和概率分布曲线；最后，用 DRPlot 控件来显示。

AI 知识库大多有 4KB 的数据块长度限制，因此，案例实验多数是直接用控件装配而成，信号分析部分不那么透明。你可以尝试用自定义函数，替换 DRGenerator 控件和 DRPdfCdf 等信号源和信号分析类控件，其实也没几行代码。

典型信号的概率密度和概率分布曲线实验样例代码

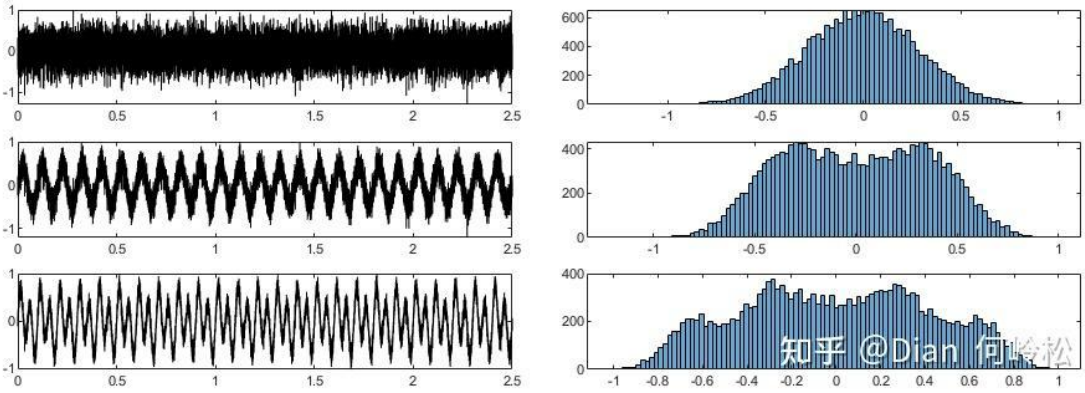
```
import tkinter as tk
import drvi.drviDSP as dsp
import drvi.drviControlls as dr

#==Main Window=====================
win= tk.Tk()
winWidthCur=1100
winHeightCur=670
win.geometry("%dx%d" %(winWidthCur,winHeightCur))
win.config(bg="#ddeeee")
```

```
#==Layout of controls=====================
mLab1=dr.DRLabel(win,20,30,400,40,'#003355','#ffffff','典型信号概率密度和分布曲线',20)
mPlotWave=dr.DRPlot(win,20,80,900,270,'','#003355','#000000','#ffffff','#00eeff','#ffff00',0,0.05,-1,1,0,0)
mPlotPDF=dr.DRPlot(win,20,350,450,280,'','#003355','#000000','#ffffff','#00eeff','#ffff00',-1,1,0,0,0,0)
mPlotCDF=dr.DRPlot(win,470,350,450,280,'','#003355','#000000','#ffffff','#00eeff','#ffff00',-1,1,0,0,0,0)
mVButType=dr.DRRadioBut(win,940,100,120,140,'#003355','#005577',"#ffff00",'正弦波,方波,三角波,白噪声',12,0,0)
mKnobA=dr.DRKnob(win,930,280,130,130,'#004466','#222222','#aaaaaa','0,1',0,1,0.5,11)
mKnobFre=dr.DRKnob(win,930,400,130,130,'#004466','#222222','#aaaaaa','1,1000',1,1000,50,11)
mSwitch1=dr.DRSwitch(win,960,580,80,36,'#004466','#aaddee')
mType=0;
Fs=44100;
N=8192;
A=0.6;
Fre=100;
Pha=0
mSignal=dsp.DRGenerator(mType,Fs,N,A,Fre,Pha)
abc=dsp.DRPdfCdf(ylow=-1,yhigh=1)
#==DataStream Binding=====================
mSwitch1.addCallBackSingle(mSignal.setValueSingle)
mSignal.addCallBack2DX(abc.setValue2DX)
mSignal.addCallBack2DX(mPlotWave.setValue2DX)
abc.addCallBack2Dpdf(mPlotPDF.setValue2D)
abc.addCallBack2Dcdf(mPlotCDF.setValue2D)
mVButType.addCallBackSingle(mSignal.setVSType)
mKnobA.addCallBackSingle(mSignal.setVSAmp)
mKnobFre.addCallBackSingle(mSignal.setVSFre)
#==Main Loop=====================
win.mainloop()
```

设备故障概率密度曲线诊断实验

在工业现场，常用概率密度函数来对设备运行故障进行早期诊断。一个典型的判据是概率密度曲线顶部出现塌陷。从理论上看，这等价于噪声信号的高斯分布曲线中叠加了一个正弦信号的浴盆分布曲线。因为旋转机械故障时、如基础松动，信号中常常会出现类似正弦波的周期性的成分。例如，下面分别是白噪声信号和白噪声+1 项正弦波信号，以及白噪声信号+2 项正弦波信号时的信号波形和概率密度函数曲线：



实验设计

在你工作的大楼转几圈，尝试用手机录几台噪声大，感觉有问题的空调噪声的 mp3 文件；然后，用 librosa 库的 open 功能将音频文件读取为信号数组；最后进行波形显示和概率密度曲线分析。为方便起见，分析时默认使用 drvi 数据集中的空调噪声样例文件：

AirConditioner.mp3
AirconditionerHumming.mp3
AirConditionerBroken.mp3

drvi 数据集位于 drvi 的安装路径中，位置：

Python 路径\Lib\site-packages\drvi\dataset

可参照下面格式定位和打开 drvi 数据集中的文件：

fileName=mPath.getDRVIPath()+'\\dataset\\数据集中的某一数据文件名'

设备故障概率密度曲线诊断实验样例代码

```
import tkinter as tk
import numpy as np
import drvi.drviDSP as dsp
import drvi.drviControlls as dr
import drvi.drviSimpleAudio as ply
import drvi.drviAppendPath as mPath

#=========================================
def demoFile(v):
    if v==0:
```

```
        initFile=mPath.getDRVIPath()+'\\dataset\\AirConditioner.mp3'
    if v==1:
        initFile=mPath.getDRVIPath()+'\\dataset\\AirconditionerHumming.mp3'
    if v==2:
        initFile=mPath.getDRVIPath()+'\\dataset\\AirConditionerBroken.mp3'
    mEntry1.setValueString(initFile)
    mPlay.setValueString(initFile)

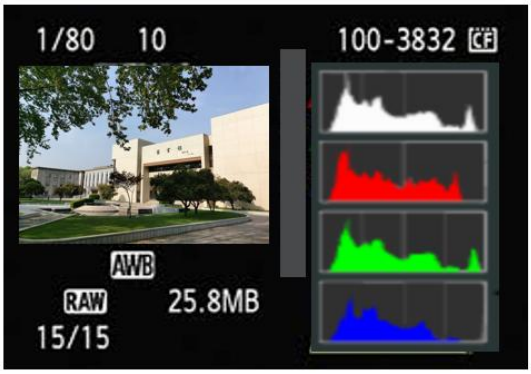
def setValue2DX(dx,x):
  mPlot.setValue2DXFast(dx,x)
  mFs=1/dx
  mLen=len(x)
  [mFre,mAmp]=dsp.AmpSpetrum(mLen,mFs,x)
  df=mFs/mLen
  mPlot1.setValue2DXFast(df,mAmp)

  M=48
  abin=np.linspace(-1,1,num=M)
  [mHist,bins]=np.histogram(x,bins=abin)
  mPdf=mHist/mLen
  mPlot3.setValue2D(abin[0:M-1],mPdf)

#==Main Window=====================
win= tk.Tk()
win.geometry("1100x670")
win.config(bg="#ddeeee")
#==Layout of controls=====================
mLab1=dr.DRLabel(win,20,30,360,40,'#445577','#ffffff','音频文件概率密度曲线分析',20)
mHBut2=dr.DRHButtonGroup(win,390,35,180,30,'#006688','#ffffff','播放,停止,文件',12,0)
mEntry1=dr.DREntry(win,655,35,300,30,'#000000','#ffff00','',10)
mPlay=ply.DRPlay(win,960,35,80,30,'#002244','#ffffff',11)
mPlot=dr.DRPlot(win,20,260,510,200,'','#003355','#000000','#ffffff','#00eeff','#ffff00',0,0.2,-1,1,0,0)
mPlot1=dr.DRPlot(win,20,460,510,200,'','#003355','#000000','#ffffff','#00eeff','#ffff00',0,6000,0,0.1,0,0)
mPlot2=dr.DRPlot(win,20,80,1020,180,'','#003355','#000000','#ffffff','#00eeff','#ffff00',0,0,-1,1,0,0)
mPlot3=dr.DRPlot(win,528,260,512,400)
mOMenu=dr.DROptionMenu(win,575,35,75,30,'#00aacc','#eeee00','空调 1,空调 2,空调 3',12,0)
#====================================
mHBut2.addCallBackSingle(mPlay.setValueSingle)
mPlay.addCallBack2DX(setValue2DX)
mPlay.addCallBackStrFile(mEntry1.setValueString)
mPlay.addCallBack2DXAll(mPlot2.setValue2DX)
mOMenu.addCallBackSingle(demoFile)
demoFile(0)
#==Main Loop=====================
win.mainloop()
```

图象的直方图分析实验

相片的直方图分析是一个非常有用的工具，可以帮助我们快速的确定相片的拍摄质量和改进依据。数码像片可以看作为一个离散的二维采样信号。对彩色相片而言，每个数据点 p(x,y)的色彩又由红、绿、蓝三个属性确定，而每个色彩属性的取值范围是 0 到 255 的整数。相片的直方图分析就是统计出相片上所有点各色彩属性取不同值的数量。然后绘制出各色彩值出现的统计次数。



$$\mathrm{Re}\,d(k)=\sum pixels_{\mathrm{Re}\,d}(i,j)=k;$$

$$Green(k)=\sum pixels_{Green}(i,j)=k;$$

$$Blue(k)=\sum pixels_{Blue}(i,j)=k;$$

$$k = 0,1,\ldots, 255$$

实验设计

选择一幅相片，统计得到红、绿、蓝三色的直方图，将相片转为灰度相片，得到灰度直方图。对相片做灰度直方图均衡处理和彩色直方图均衡处理，观察处理前和处理后的差别，体会直方图分析的作用。

图象的直方图分析实验样例代码

```
import tkinter as tk
from tkinter import filedialog
import numpy as np
import matplotlib.pyplot as plt
from PIL import Image, ImageTk
from PIL import Image,ImageOps
import drvi.drviControlls as dr
import drvi.drviAppendPath as mPath

def setDemo(v):
    if v==0:
        mfile=mPath.getDRVIPath()+'\\dataset\\PhotoGrey.jpg'
    if v==1:
        mfile=mPath.getDRVIPath()+'\\dataset\\PhotoColor.jpg'
    openFile(mfile)

def working(v):
    mfile=filedialog.askopenfilename(title="选择文件",filetypes=(("Jpg files", "*.jpg"),("all files", "*.*")))
    if not mfile: return 1
    openFile(mfile)

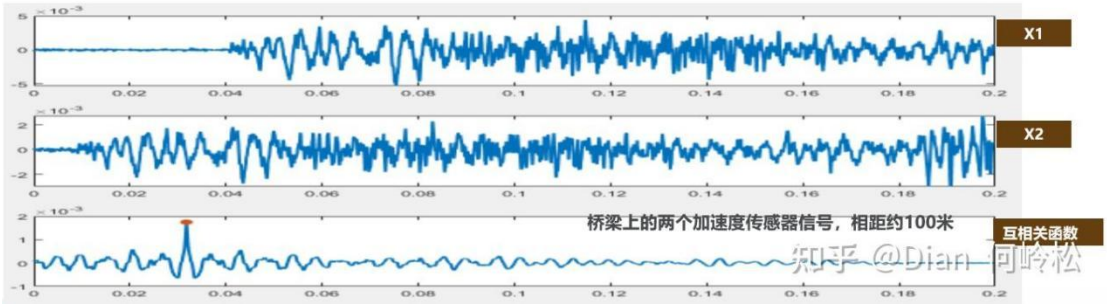
def openFile(mfile):
```

```
mEntry2.setValueString(mfile)
im1=Image.open(mfile)
data=np.array(im1)
dataR=np.zeros(data.shape,dtype=np.uint8)
dataR[:,:,0]=data[:,:,0]
dataG=np.zeros(data.shape,dtype=np.uint8)
dataG[:,:,1]=data[:,:,1]
dataB=np.zeros(data.shape,dtype=np.uint8)
dataB[:,:,2]=data[:,:,2]
dataGrey=np.zeros(data.shape,dtype=np.uint8)
mImg11.setValueImgArray(data)
mImg21.setValueImgArray(dataR)
mImg22.setValueImgArray(dataG)
mImg23.setValueImgArray(dataB)
mImg31.setValueImgArray(dataGrey)
#======
imHist=im1.convert("RGB")
r,g,b=imHist.split()
rHist=r.histogram()
gHist=g.histogram()
bHist=b.histogram()
mP11.setValue2DX(1,rHist)
mP12.setValue2DX(1,gHist)
mP13.setValue2DX(1,bHist)
mP21.setValue2DX(1,rHist)
mP22.setValue2DX(1,gHist)
mP23.setValue2DX(1,bHist)
#======================
imGrey=ImageOps.grayscale(im1)
greyHist=imGrey.histogram()
mImg31.setValueImage(imGrey)
mP31.setValue2DX(1,greyHist)
greyEQ=ImageOps.equalize(imGrey, mask = None)
greyEQHist=greyEQ.histogram()
mImg32.setValueImage(greyEQ)
mP32.setValue2DX(1,greyEQHist)
#======================
r=ImageOps.equalize(r,mask = None)
g=ImageOps.equalize(g,mask = None)
b=ImageOps.equalize(g,mask = None)
im2=Image.merge( 'RGB', (r,g,b))
mImg41.setValueImage(im2)
#======================
im1hsv=im1.convert("HSV")
h,s,v=im1hsv.split()
v=ImageOps.equalize(v,mask = None)
im1hsv=Image.merge( 'HSV', (h,s,v))
im1rgb = im1hsv.convert("RGB")
mImg42.setValueImage(im1rgb)
```

```
#==Main Window=====================
win= tk.Tk()
win.geometry("1200x690")
win.config(bg="#ddeeee")
#==Layout of controls=====================
mLab1=dr.DRLabel(win,20,30,400,40,'#445577','#ffffff','图象的直方图曲线分析',20)
mButton1=dr.DRButton(win,860,35,100,30,'#00aacc','#eeee00','选择文件',12,101)
mEntry2=dr.DREntry(win,440,35,400,30,'#ffffff','#000000','',12)
mHBut=dr.DRHButtonGroup(win,1000,35,160,30,'#00aacc','#eeee00','样例 1,样例 2',11,0)
mTab=dr.DRNoteBook(win,20,85,1150,580,'#003355','#000088','#ffffff','原图,RGB 分解,灰度变换和直方图均衡,彩图 RGB 均衡和 HSV 均衡')
mImg11=dr.DRImage(mTab.win[0],10,10,640,530,cb='#004466')
mP11=dr.DRPlot(mTab.win[0],640,10,480,180,'','#003355','#000000','#ffffff','#00eeff','#ffff00',0,0,0,0,0,0)
mP12=dr.DRPlot(mTab.win[0],640,10+180-4,480,180,'','#003355','#000000','#ffffff','#00eeff','#ffff00',0,0,0,0,0,0)
mP13=dr.DRPlot(mTab.win[0],640,10+2*180-8,480,180,'','#003355','#000000','#ffffff','#00eeff','#ffff00',0,0,0,0,0,0)
mImg21=dr.DRImage(mTab.win[1],10,10,365,310,cb='#004466')
mImg22=dr.DRImage(mTab.win[1],10+375,10,365,310,cb='#004466')
mImg23=dr.DRImage(mTab.win[1],10+750,10,365,310,cb='#004466')
mP21=dr.DRPlot(mTab.win[1],10,340,365,200,'','#003355','#000000','#ffffff','#00eeff','#ffff00',0,0,0,0,0,0)
mP22=dr.DRPlot(mTab.win[1],10+375,340,365,200,'','#003355','#000000','#ffffff','#00eeff','#ffff00',0,0,0,0,0,0)
mP23=dr.DRPlot(mTab.win[1],10+750,340,365,200,'','#003355','#000000','#ffffff','#00eeff','#ffff00',0,0,0,0,0,0)
mImg31=dr.DRImage(mTab.win[2],10,10,550,380,cb='#004466')
mImg32=dr.DRImage(mTab.win[2],10+560,10,550,380,cb='#004466')
mP31=dr.DRPlot(mTab.win[2],10,400,550,150,'','#003355','#000000','#ffffff','#00eeff','#ffff00',0,0,0,0,0,0)
mP32=dr.DRPlot(mTab.win[2],10+560,400,550,150,'','#003355','#000000','#ffffff','#00eeff','#ffff00',0,0,0,0,0,0)
mImg41=dr.DRImage(mTab.win[3],10,10,555,400,cb='#004466')
mImg42=dr.DRImage(mTab.win[3],10+565,10,555,400,cb='#004466')
#====================================
mButton1.addCallBackSingle(working)
mHBut.addCallBackSingle(setDemo)
#==Main Loop=====================
win.mainloop()
```

典型信号的相关分析实验

相关分析是一种常用的信号分析手段，常用于查找两个信号间在不同时刻差的相似关系，或同一个信号间在不同时刻时的相似关系，因此相关分析也可以看作为时差域分析。例如，下面是桥面上两个相距 100 米的加速度传感器测得的振动加速度信号。若将下图信号向右移动 0.03 秒，则上下两个信号最为相似。



实验设计

分别产生两个不同的典型信号，改变它们的信号类型、频率参数，然后对两个信号做互相关分析。Numpy 和 Scipy 库中均提供了相关分析计算函数：

correlate(in1, in2, mode='full', method='auto')

因此，实现信号的互相关分析还是很简单的，核心代码就一条语句：

```
cXY=np.correlate(x,y,'full')
```

互相关曲线的横坐标数据是时差，中心点是零：

```
N=len(cXY)
t=np.arange(N)*dt-N/2*dt
```

另外，为避免重迭失真，相关函数在计算时进行了补零，这造成得到的相关函数曲线受补零影响是向两边衰减的。必要时可以除以时移量进行校正。

为简化起见，实验用 DRVI 库中 DRGenerator 控件来产生标准信号，用 DRCorr 控件来相关函数；最后，用 2 个 DRPlot 控件来显示 X 通道和 Y 通道的信号波形；再用 2 个 DRPlot 控件来显示相关函数曲线和经过幅值校正的无偏估计的相关函数曲线。

典型信号的相关分析实验样例代码

```
import tkinter as tk
import numpy as np
import drvi.drviDSP as dsp
import drvi.drviControlls as dr

def getSelect(v):
    Fs=44100;
    N=5000;
```

```
    #产生仿真信号
    K=mVButType.getValueSingle()
    F=mHSlider.getValueSingle()
    mEntry1.setValueSingle(F)
    mS1=dsp.DRGenerator(K,Fs,N,0.6,F,0)
    dx,x1=mS1.getArrayTwoX()
    K=mVButType1.getValueSingle()
    F=mHSlider1.getValueSingle()
    mEntry2.setValueSingle(F)
    mS2=dsp.DRGenerator(K,Fs,N,0.6,F,0)
    dx,x2=mS2.getArrayTwoX()
    #相关分析
    T=mType.getValueSingle()
    if T==0: x2=x1
    mPlot1.setValue2DX(dx,x1)
    mPlot2.setValue2DX(dx,x2)
    abc=dsp.DRCorr(N=4096)
    abc.setValue3D_XY(dx,x1,x2)
    t1,c1=abc.getArrayTwo()
    t2,c2=abc.getArrayTwoCorrect()
    mPlot4.setValue2D(t1,c1)
    mPlot3.setValue2D(t2,c2)

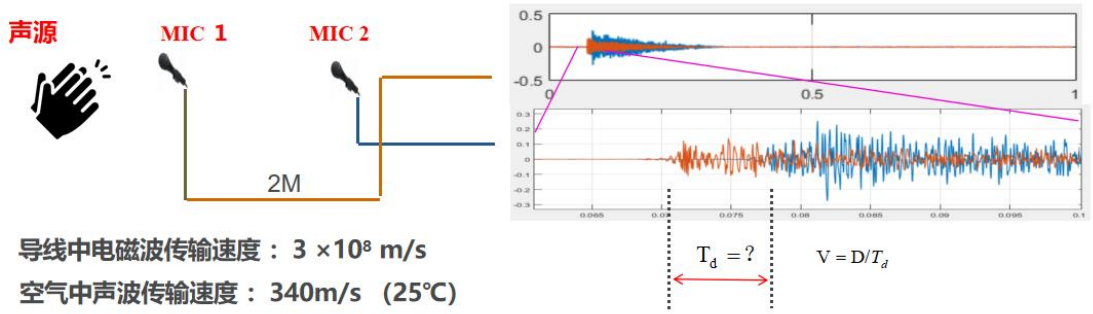
def getSelect1(v):
    F=mEntry1.getValueSingle()
    mHSlider.setValueSingle(F)
    F=mEntry2.getValueSingle()
    mHSlider1.setValueSingle(F)
    getSelect(0)

#==Main Window=====================
win= tk.Tk()
winWidthCur=1100
winHeightCur=750
win.geometry("%dx%d" %(winWidthCur,winHeightCur))
win.config(bg="#ddeeee")
#==Layout of controls=====================
mLab1=dr.DRLabel(win,20,30,240,40,'#003355','#ffffff','信号的相关分析',20)
mType=dr.DROptionMenu(win,280,35,80,30,'#003355','#eeee00','自相关,互相关',12,0)
mPlot1=dr.DRPlot(win,20,90,520,200,ylow=-1,yhigh=1)
mPlot2=dr.DRPlot(win,540,90,520,200,ylow=-1,yhigh=1)
mPlot3=dr.DRPlot(win,20,490,1040,200)
mPlot4=dr.DRPlot(win,20,290,1040,200)
mVButType=dr.DROptionMenu(win,370,35,80,30,'#003355','#eeee00','正弦波,方波,三角波,白噪声',12,0)
mEntry1=dr.DREntry(win,455,35,80,30,'#ffffff','#000000','100',11)
mHSlider=dr.DRHSlider(win,530,35,180,30,mlow=0,mhigh=1000,val=100)
mVButType1=dr.DROptionMenu(win,715,35,100,30,'#003355','#eeee00','正弦波,方波,三角波,白噪声',12,0)
mEntry2=dr.DREntry(win,820,35,90,30,'#ffffff','#000000','100',11)
mHSlider1=dr.DRHSlider(win,910,35,180,30,mlow=0,mhigh=1000,val=100)
#==DataStream Binding=====================
```

```
mVButType.addCallBackSingle(getSelect);
mVButType1.addCallBackSingle(getSelect);
mHSlider.addCallBackSingle(getSelect);
mHSlider1.addCallBackSingle(getSelect);
mType.addCallBackSingle(getSelect);
mEntry1.addCallBackSingle(getSelect1);
mEntry2.addCallBackSingle(getSelect1);
getSelect(0)
#==Main Loop=====================
win.mainloop()
```

互相关分析测距实验

利用信号自身或信号间的时间差测距是是一种常用手段，如利用回声定位的声纳和自然界的蝙蝠等。为便于应用，在 DRVI 的数据集中提供了一个互相关测距或测声速的样本文件 corrDistMeaClap.mp3。下面是该样本文件的测试实验环境：



实验过程就是将一个 USB 口的双独立通道的采访麦克风插在计算机上，然后拍一下掌；这样获取到的音频文件的两个通道信号间存在时间差。互相关分析是估计这个时间差的主要手段；得到时间差后，若已知声速，则可以计算得到 2 个麦克风间的距离；若已知 2 个麦克风间的距离，则可以得到实验时的声速。

实验设计

利用 Librosa 音频信号分析库的读文件函数，打开 WAV/MP3 音频文件，获取采样频率和音频信号数组。注意，Librosa 读文件时默认强制进行单通道转换，必须关掉。下面是双通道音频文件读法：

```
data,Fs=librosa.load(mFileName,mono=False,sr=None)
CH=np.ndim(data)
if CH!=2: print("单通道信号，不符合互相关测距要求")
x=data[0,:]
y=data[1,:]
```

然后，从音频文件中选取有效信号段进行互相关分析即可。样例文件已做剪辑，用全长即可。注意，返回相关系数长度建议选“same”。

```
cc=np.correlate(x,y,'same')
```

互相关分析测距实验样例代码

```
import tkinter as tk
import numpy as np
import librosa
import drvi.drviDSP as dsp
import drvi.drviControlls as dr
```

```
import drvi.drviSimpleAudio as ply
import drvi.drviAppendPath as mPath

#=========================================
def selSignal(txt):
    mFile=mEntry1.getValueString()
    data,Fs=librosa.load(mFile,mono=False,sr=None)
    CH=np.ndim(data)
    if CH!=2:
        print("单通道信号，不符合互相关测距要求")
        x=data
        mPlot.setValue2DX(1/Fs,x)
        mPlot1.setValue2DX(1/Fs,np.zeros(len(x)))
        mPlot2.setValue2DX(1/Fs,np.zeros(len(x)))
        return
    x=data[0,:]
    y=data[1,:]
    mPlot.setValue2DX(1/Fs,x)
    mPlot1.setValue2DX(1/Fs,y)
    cc=np.correlate(abs(x),abs(y),'same')
    NN=len(cc)
    tt=np.arange(NN)*(1/Fs)-NN/2*(1/Fs)
    mPlot2.setValue2D(tt,abs(cc))

#==Main Window=====================
win= tk.Tk()
win.geometry("1100x670")
win.config(bg="#ddeeee")
#==Layout of controls=====================
mLab1=dr.DRLabel(win,20,30,400,40,'#445577','#ffffff','互相关测距实验',20)
mHBut2=dr.DRHButtonGroup(win,450,30,200,36,'#006688','#ffffff','播放,停止,文件',12,0)
mEntry1=dr.DREntry(win,655,30,300,36,'#000000','#ffff00','',10)
mPlay=ply.DRPlay(win,960,30,80,36,'#002244','#ffffff',11)
mPlot=dr.DRPlot(win,20,80,1020,180)
mPlot1=dr.DRPlot(win,20,260,1020,200)
mPlot2=dr.DRPlot(win,20,460,1020,200)
#=====================================
mHBut2.addCallBackSingle(mPlay.setValueSingle)
mPlay.addCallBackStrFile(mEntry1.setValueString)
mPlay.addCallBackStrFile(selSignal)
initFile=mPath.getDRVIPath()+'\\dataset\\corrDistMeaClap.mp3'
mEntry1.setValueString(initFile)
mPlay.setValueString(initFile)
selSignal('')
#==Main Loop=====================
win.mainloop()
```

自相关分析回声时延测量实验

回声是自然界中常见的一种现象。原声和回声间存在时延，混在一起可以形成独特的音效。



实验设计

利用 Librosa 音频信号分析库的读文件函数，打开 WAV/MP3 音频文件，获取采样频率和音频信号数组。强制单通道读取：

```
data,Fs=librosa.load(mFileName,mono=True,sr=None)
```

然后，添加时延，制作原声+回声的混响音频信号。下面是 0.1 秒时延,幅值衰减 0.6：

```
NN=int(Fs*0.1)
xx=np.zeros(NN)
x1=np.append(x1,xx)
x2=np.append(xx,x2)
x=x1+0.6*x2
```

然后，对混有回声的信号做自相关分析，估计出回声的时延。

```
cc=np.correlate(x,x,'same')
```

自相关分析回声时延测量实验样例代码

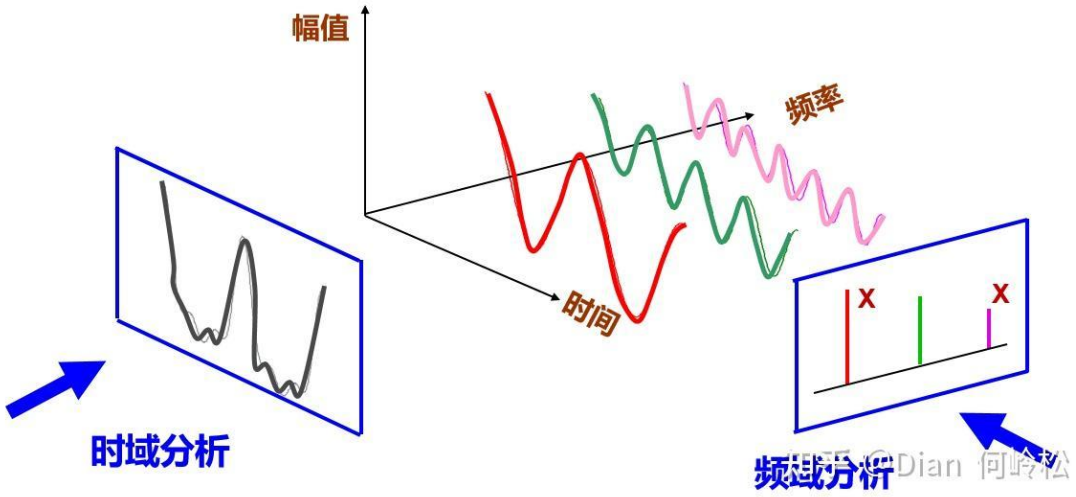
```
import tkinter as tk
import numpy as np
import librosa
import drvi.drviDSP as dsp
import drvi.drviControlls as dr
import drvi.drviSimpleAudio as ply
import drvi.drviAppendPath as mPath
#=========================================
def addEcho(v):
    if v==0: return
```

```
    print("添加时延作为回声混音")
    Fs=mPlay.FsPlay
    x1=mPlay.dataPlay
    x2=0.7*mPlay.dataPlay
    NN=int(Fs*v*0.1)
    xx=np.zeros(NN)
    x1=np.append(x1,xx)
    x2=np.append(xx,x2)
    mPlay.dataPlay=x1+x2
    x=mPlay.dataPlay[NN:4*NN]
    y=mPlay.dataPlay[NN:4*NN]
    mPlot1.setValue2DX(1/Fs,y)
    cc=np.correlate(x,y,'same')
    NN=len(cc)
    tt=np.arange(NN)*(1/Fs)-NN/2*(1/Fs)
    mPlot2.setValue2D(tt,abs(cc))

#==Main Window=====================
win= tk.Tk()
win.geometry("1100x670")
win.config(bg="#ddeeee")
#==Layout of controls=====================
mLab1=dr.DRLabel(win,20,30,300,40,'#445577','#ffffff','自相关回声时延测量',20)
mHBut2=dr.DRHButtonGroup(win,450,30,200,36,'#006688','#ffffff','播放,停止,文件',12,0)
mEntry1=dr.DREntry(win,655,30,300,36,'#000000','#ffff00','',10)
mPlay=ply.DRPlay(win,960,30,80,36,'#002244','#ffffff',11)
mPlot=dr.DRPlot(win,20,80,1020,180)
mPlot1=dr.DRPlot(win,20,260,1020,200)
mPlot2=dr.DRPlot(win,20,460,1020,200)
mOMenu=dr.DROptionMenu(win,335,30,105,36,'#00aacc','#eeee00','添加时延,0.1 秒,0.2 秒,0.3 秒,0.4 秒',12,0)
#====================================
mOMenu.addCallBackSingle(addEcho)
mHBut2.addCallBackSingle(mPlay.setValueSingle)
mPlay.addCallBackStrFile(mEntry1.setValueString)
mPlay.addCallBackStrFile(mEntry1.setValueString)
mPlay.addCallBack2DXAll(mPlot.setValue2DX)
initFile=mPath.getDRVIPath()+'\\dataset\\voice1.mp3'
mEntry1.setValueString(initFile)
mPlay.setValueString(initFile)
#==Main Loop=====================
win.mainloop()
```

信号滤波概念实验

测量信号中除传感器测量的有用信号外，各种形式的干扰信号也会混叠在其中，它们与有用信号叠加在一起，扭曲了测量结果。滤波器是一种选频装置，可以使信号中特定频率成分通过，而衰减其他频率成分。当干扰信号与测量信号位于不同的频带，我们就可以通过滤波器来将干扰信号对应的频率成分去掉，从测量信号中提取出有用信号成分。



实验设计

生成由 500Hz、638Hz、815Hz、1040Hz、1329Hz 等 5 个正弦波信号组成的复合信号，然后应用一个滤波频带可调的带通滤波器对信号进行滤波，从复合信号中将所需的特定信号提取出来。例如，提取出 638Hz 的信号。为简化起见，信号滤波用频域滤波的方法实现，H(f)频率通带则简单的用 2 个水平滚动条来实现。

xnew=ifft(fft(x)*H(f))

信号滤波概念实验样例代码

```
import tkinter as tk
import numpy as np
from scipy import signal
from scipy.fftpack import fft,ifft
from scipy.fft import rfft,irfft
import drvi.drviDSP as dsp
import drvi.drviControlls as dr
import drvi.drviSimpleAudio as sa

#=========================================
def playing(v):
    global N,Fs,tData,xData1
    sa.play(1,Fs,xData1,0)

def setData():
    global N,Fs,tData,xData,xData1
```

```
    Fs=44100;
    N=40000;
    tData=np.arange(N)*(1/Fs)
    x1=0.25*np.sin(2*np.pi*500*tData)
    x2=0.25*np.cos(2*np.pi*638*tData)
    x3=0.25*np.sin(2*np.pi*815*tData)
    x4=0.25*np.cos(2*np.pi*1040*tData)
    x5=0.25*np.sin(2*np.pi*1329*tData)
    xData=x1+x2+x3+x4+x5
    mPlotWave.setValue2D(tData,xData)
    [mFre,mAmp]=dsp.AmpSpetrumX(N,Fs,xData)
    mPlotAmp.setValue2DFast(mFre,mAmp)
    filtering(0)

def setFilter(v):
    v1=mHScale1.getValueSingle()
    v2=mHScale2.getValueSingle()
    f=np.arange(2500)
    y=np.zeros(2500)
    for i in range(v1,v2): y[i]=0.25
    mPlotAmp1.setValue2DFast(f,y)

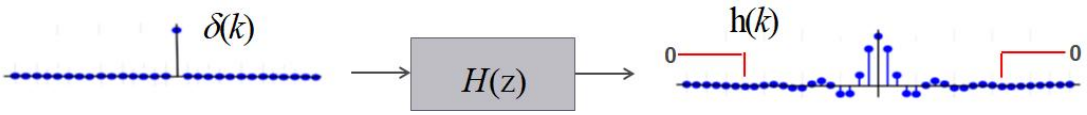
def filtering(v):
    global N,Fs,tData,xData,xData1
    v1=mHScale1.getValueSingle()
    v2=mHScale2.getValueSingle()
    abc.setBand(v1,v2)
    abc.setValue2DX(1/Fs,xData)
    dt1,xData1=abc.getValue2DX()
    mPlotWave1.setValue2DFast(tData,xData1)
    [mFre,mAmp]=dsp.AmpSpetrumX(N,Fs,xData1)
    mPlotAmp1.setValue2DFast(mFre,mAmp)

#==Main Window=====================
win= tk.Tk()
winWidthCur=1100
winHeightCur=640
win.geometry("%dx%d" %(winWidthCur,winHeightCur))
win.config(bg="#ddeeee")
#==Layout of controls=====================
mLab1=dr.DRLabel(win,20,30,380,40,'#003355','#ffffff','信号数字滤波的概念',20)
mPlotWave=dr.DRPlot(win,20,110,520,240,'','#003355','#000000','#ffffff','#00eeff','#ffff00',0.01,0.05,-1.25,1.25,0,0)
mPlotAmp=dr.DRPlot(win,540,110,520,240,'','#003355','#000000','#ffffff','#00eeff','#ffff00',0,2500,0,0.4,0,0)
mPlotWave1=dr.DRPlot(win,20,350,520,240,'','#003355','#000000','#ffffff','#00eeff','#ffff00',0.01,0.05,-1.25,1.25,0,0)
mPlotAmp1=dr.DRPlot(win,540,350,520,240,'','#003355','#000000','#ffffff','#00eeff','#ffff00',0,2500,0,0.4,0,0)
mHScale1=dr.DRHScale(win,600,10,440,50,'#002266','#000000',0,2500,0,11)
mHScale2=dr.DRHScale(win,600,55,440,50,'#002266','#000000',0,2500,2500,11)
mBut2=dr.DRButton(win,500,30,80,40,'#003355','#eeee00','播放',12,101)
mBut3=dr.DRButton(win,420,30,80,40,'#003355','#eeee00','滤波',12,101)
mBut2.addCallBackSingle(playing)
```

```
mBut3.addCallBackSingle(filtering)
mHScale1.addCallBackSingle(setFilter)
mHScale2.addCallBackSingle(setFilter)
abc=dsp.DRFilter(Fs=44100,f1=00,f2=2500,mt=0)
setData()
#==Main Loop=====================
win.mainloop()
```

FIR 滤波器脉冲响应设计法和吉普斯效应实验

FIR 滤波器又称为有限脉冲响应滤波器，FIR 就来源于英文单词“Finite Impulse Response”的首字符缩写。FIR 滤波器的单位脉冲响应 h(k)是一个有限长度的序列，长度 M 以外恒为零，故称为有限脉冲响应滤波器。



实验设计

为简单起见，以低通 FIR 滤波器设计为对象。首先在频域定义一个理想的低通滤波器，傅里叶变换得到时域单位脉冲响应函数；然后单位脉冲响应函数在时域进行截断，取前 M 项作为 FIR 滤波器系数，这就是所谓的 FIR 滤波器脉冲响应设计法。求该滤波器的频域特性，就可以看到由于脉冲响应函数时域截断所造成的滤波器频率特性的振荡，也就是吉普斯效应。这一现象与方波信号合成时取前 N 项(频域截断)，所带来的时域振荡是一样的。

FIR 滤波器脉冲响应设计法和吉普斯效应实验样例代码

```
import tkinter as tk
import numpy as np
from scipy import signal
import drvi.drviDSP as dsp
import drvi.drviControlls as dr

#==========================================
def setData(v):
    global N,Fs,tData,xData,xData1
    v1=mHScale1.getValueSingle()/2
    v2=mHScale2.getValueSingle()/2
    N=256;
    f=np.arange(2*N+1)-N
    H=np.zeros(2*N+1);
    H1=np.zeros(2*N+1);
    H[N]=v2-v1
    for i in range(1,N):
        w=i*np.pi;
        h1=np.sin(v1*w)/w;
        h2=np.sin(v2*w)/w;
        H[N+i]=h2-h1
        H[N-i]=h2-h1
    mPlotWave.setValue2D(f,H)
    M=int(mHScale3.getValueSingle()/2)
    H1[N]=H[N]
    print('M=',M);
    print('C(0)=',H1[N]);
    for i in range(1,M):
```

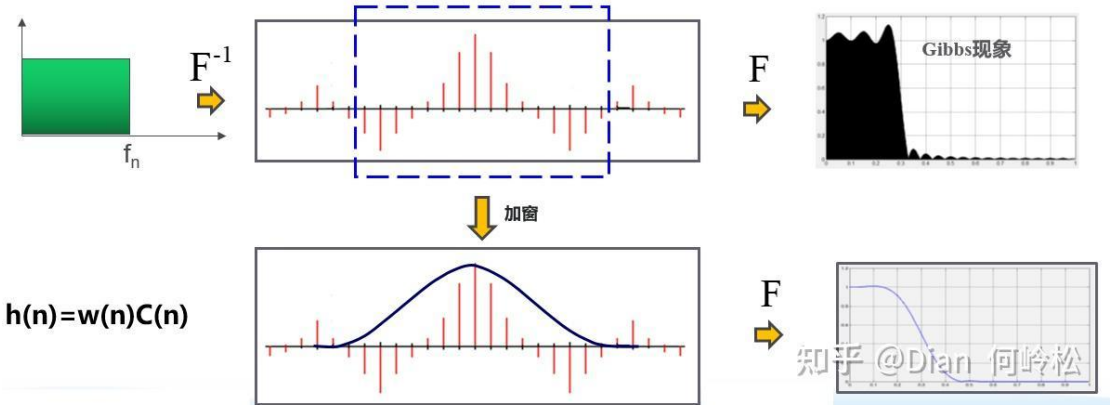
```
            H1[N+i]=H[N+i]
            H1[N-i]=H[N-i]
            print('C(-',i,')=C(',i,')=',H[N+i])
        mPlotWave1.setValue2D(f,H1)
        w, h = signal.freqz(H1);
        f=np.arange(512)/256.0
        A=20 * np.log10(abs(h))
        mPlotAmp1.setValue2D(f,abs(h))

def myfilter(v):
        v1=mHScale1.getValueSingle()
        v2=mHScale2.getValueSingle()
        f=np.arange(1000)*0.001
        y=np.ones(1000)
        for i in range(1000):
            if i<(1000*v1): y[i]=0
            if i>(1000*v2): y[i]=0
        mPlotAmp.setValue2D(f,y)
        setData(0);

#==Main Window=====================
win= tk.Tk()
winWidthCur=1100
winHeightCur=640
win.geometry("%dx%d" %(winWidthCur,winHeightCur))
win.config(bg="#ddeeee")
#==Layout of controls=====================
mLab1=dr.DRLabel(win,20,30,500,40,'#003355','#ffffff','FIR 滤波器脉冲响应设计法和吉普斯效应',20)
mPlotWave=dr.DRPlot(win,20,130,520,240,'','#003355','#000000','#ffffff','#00eeff','#ffff00',-250,250,-0.3,0.3,0,0)
mPlotAmp=dr.DRPlot(win,540,130,520,240,'','#003355','#000000','#ffffff','#00eeff','#ffff00',0,1,0,1.2,0,0)
mPlotWave1=dr.DRPlot(win,20,370,520,240,'','#003355','#000000','#ffffff','#00eeff','#ffff00',-250,250,-0.3,0.3,0,0)
mPlotAmp1=dr.DRPlot(win,540,370,520,240,'','#003355','#000000','#ffffff','#00eeff','#ffff00',0,1,0,1.2,0,0)
mHScale1=dr.DRHScale(win,600,30,440,50,'#002266','#000000',0,1,0.0,11)
mHScale2=dr.DRHScale(win,600,75,440,50,'#002266','#000000',0,1,0.6,11)
mHScale3=dr.DRHScale(win,20,75,500,50,'#002266','#000000',0,250,50,11)
mHScale1.addCallBackSingle(myfilter)
mHScale2.addCallBackSingle(myfilter)
mHScale3.addCallBackSingle(setData)
myfilter(0)
#==Main Loop=====================
win.mainloop()
```

FIR 滤波器窗函数设计法实验

理想滤波器的单位脉冲响应，也就是对应的 FIR 滤波器的系数 Ck 是无限长序列，用脉冲响应函数设计法进行设计时只能截取有限长度，但截断造成时域跳变，从而导致滤波器频域特性的震荡，这称之为吉普斯现象。为此，可以引入一个有限长度的向两边衰减的窗函数序列 Wn 与 Cn 相乘，将截断窗口外的系数强行衰减到零。这样就不存在非零系数的截断，从而可以规避因时域截断而带来的滤波器频域特性的震荡；这称之为



实验设计

为简单起见，以低通 FIR 滤波器设计为对象。首先在频域定义一个理想的低通滤波器，傅里叶变换得到时域单位脉冲响应函数；然后按截断窗口宽度 M 对单位脉冲响应函数在时域进行加窗截断，截断后的前 M 项就是 FIR 滤波器系数，这就是 FIR 滤波器的窗函数设计法。求该滤波器的频域特性，就可以看到由于对脉冲响应函数进行加窗截断，吉普斯效应被大大减弱。

FIR 滤波器窗函数设计法实验样例代码

```
import tkinter as tk
import numpy as np
from scipy.fftpack import fft,ifft
from scipy import signal
import drvi.drviDSP as dsp
import drvi.drviControlls as dr

#=========================================
def setData(v):
    global N,Fs,tData,xData,xData1
    v1=mHScale1.getValueSingle()/2
    v2=mHScale2.getValueSingle()/2
    N=256;
    f=np.arange(2*N+1)-N
    H=np.zeros(2*N+1);
    H1=np.zeros(2*N+1);
    Win=np.zeros(2*N+1);
    H[N]=v2-v1
```

```
    for i in range(1,N):
        w=i*np.pi;
        h1=np.sin(v1*w)/w;
        h2=np.sin(v2*w)/w;
        H[N+i]=h2-h1
        H[N-i]=h2-h1
    mPlotWave.setValue2D(f,H)
    M=int(mHScale3.getValueSingle()/2)
    Whan=signal.windows.hann(2*M+1);
    Win[N]=Whan[M]
    for i in range(1,M):
        Win[N+i]=Whan[M+i]
        Win[N-i]=Whan[M-i]
    mPlotWave2.setValue2D(f,Win)
    H1[N]=H[N]
    for i in range(1,M):
        H1[N+i]=H[N+i]
        H1[N-i]=H[N-i]
    H1=H1*Win
    print('M=',M);
    print('C(0)=',H1[N]);
    for i in range(1,M):
        print('C(-',i,')=C(',i,')=',H1[N+i])
    mPlotWave1.setValue2D(f,H1)
    w, h = signal.freqz(H1);
    f=np.arange(512)/256.0
    A=20 * np.log10(abs(h))
    mPlotAmp1.setValue2D(f,abs(h))

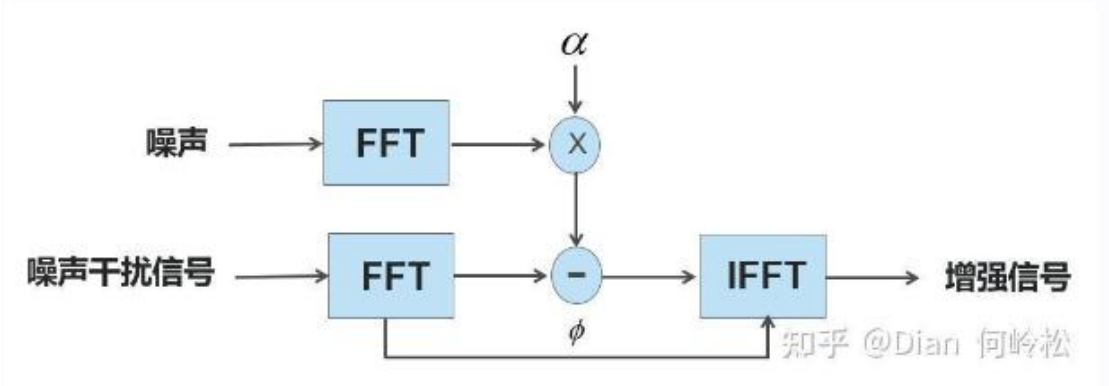
def myfilter(v):
    v1=mHScale1.getValueSingle()
    v2=mHScale2.getValueSingle()
    f=np.arange(1000)*0.001
    y=np.ones(1000)
    for i in range(1000):
        if i<(1000*v1): y[i]=0
        if i>(1000*v2): y[i]=0
    mPlotAmp.setValue2D(f,y)
    setData(0);

#==Main Window=====================
win= tk.Tk()
winWidthCur=1100
winHeightCur=640
win.geometry("%dx%d" %(winWidthCur,winHeightCur))
win.config(bg="#ddeeee")
#==Layout of controls=====================
mLab1=dr.DRLabel(win,20,30,500,40,'#003355','#ffffff','FIR 滤波器窗函数设计法',20)
mPlotWave=dr.DRPlot(win,20,130,520,160,'','#003355','#000000','#ffffff','#00eeff','#ffff00',-250,250,-0.3,0.3,0,0)
mPlotWave1=dr.DRPlot(win,20,450,520,160,'','#003355','#000000','#ffffff','#00eeff','#ffff00',-250,250,-0.3,0.3,0,0)
```

```
mPlotWave2=dr.DRPlot(win,20,290,520,160,'','#003355','#000000','#ffffff','#00eeff','#ffff00',-250,250,0,1.2,0,0)
mPlotAmp=dr.DRPlot(win,540,130,520,240,'','#003355','#000000','#ffffff','#00eeff','#ffff00',0,1,0,1.2,0,0)
mPlotAmp1=dr.DRPlot(win,540,370,520,240,'','#003355','#000000','#ffffff','#00eeff','#ffff00',0,1,0,1.2,0,0)
mHScale1=dr.DRHScale(win,600,30,440,50,'#002266','#000000',0,1,0.0,11)
mHScale2=dr.DRHScale(win,600,75,440,50,'#002266','#000000',0,1,0.6,11)
mHScale3=dr.DRHScale(win,20,75,500,50,'#002266','#000000',0,250,50,11)
mHScale1.addCallBackSingle(myfilter)
mHScale2.addCallBackSingle(myfilter)
mHScale3.addCallBackSingle(setData)
myfilter(0)
#==Main Loop=======================
win.mainloop()
```

音频信号谱减法降噪实验

所谓谱减法就是在频域将噪声信号的频谱从受干扰信号的频谱中减去，然后 再通过傅里叶逆变换，获得消除噪声的增强信号，因此可以看作为一种特殊的频 域滤波方法。谱减法的优点是不再要求噪声信号与有用信号必须位于不同的频 带。但有新的假设，就是信号中的干扰噪声只是简单的加性噪声，另外噪声信号 是稳态信号。因此，只要将带噪声干扰的信号谱减去噪声谱，就可以得到滤除噪 声后的信号谱，再变换回时域，就可以实现频域滤波。该方法由 Boll, S.F 于 1979 年提出，其原理框图如下：



谱减法的关键是噪声谱的估计，为简化起见，应用时一般认为信号的头部只 是由噪声信号组成，因此可以从信号头部取一段短暂的信号来计算噪声谱；当然 也可以借助时频谱从信号中可视化的选取，则效果更好。由于实际中的噪声干扰 信号很难严格满足稳态信号的要求，信号头部的噪声谱与后续段的噪声谱会有出 入，导致谱减法处理后的滤波信号也许会存在所谓的“Music Noise”。有兴趣的 读者可参阅“Theoretical analysis of musical noise in generalized spectral subtraction: Why should not use power/amplitude subtraction?“ 等文献和改进算法。

实验设计

在 Python 中，PyTorch、Noisereduce 等库中均有谱减法的 API，为减少对这些库的依赖，笔者将相关代码看懂后提取出来到 DRVI 中使用。对读者而言，建议安装完整的 Noisereduce 库来使用，本实验仅仅是一个技术示范。另外，在实验中还添加了一个频域带通滤波环节，可以消除低频、高频等听觉范围外的干扰声。

音频信号谱减法降噪实验样例代码

```
import tkinter as tk
from tkinter import filedialog
import numpy as np
import librosa.display
import matplotlib.pyplot as plt
import drvi.drviControlls as dr
import drvi.drviSimpleAudio as sa
import drvi.drKillNoise as kNoise
import drvi.drviDSP as dsp
import drvi.drviAppendPath as mPath
#================================================
```

```
def setFile(v):
    global Fs,N,CH,data
    mfile=filedialog.askopenfilename(title="选择文件",filetypes=(("MP3 files", "*.mp3"),("WAV files", "*.wav"),("all
files", "*.*")))
    if not mfile: return 1
    openFile(mfile)

def openFile(mfile):
    global Fs,N,data,data1
    data, Fs = librosa.load(mfile,mono=True,sr=None)
    N=len(data);
    data1=data
    drawTF(0,Fs,N,data)

def setReset(v):
    global Fs,N,data,data1
    data1=data
    drawTF(0,Fs,N,data)

def setFilter(v):
    global Fs,N,CH,data,data1
    data1= kNoise.removeStationaryNoise(data1,Fs)
    drawTF(1,Fs,N,data1)

def setFilter1(v):
    global Fs,N,CH,data,data1
    f1=mEntry1.getValueSingle()
    f2=mEntry2.getValueSingle()
    abc=dsp.DRFilter()
    abc.setBand(f1,f2)
    abc.setValue2DX(1/Fs,data1)
    dt,data1=abc.getValue2DX()
    drawTF(1,Fs,N,data1)

def drawTF(k,Fs,N,data):
    T=N/Fs;
    y = librosa.amplitude_to_db(np.abs(librosa.stft(data)), ref=np.max)
    plt.close('all')
    fig, ax = plt.subplots(1,1)
    img=librosa.display.specshow(y, sr=Fs, x_axis='time', y_axis='log')
    plt.ylim(128, 16000)
    fig.colorbar(img, ax=ax, format="%+2.f dB")
    if k==0:
        mPlotWave.setValue2DX(1/Fs,data)
        mPlotWave.setXlim(0,T)
        mSpec.setValueFig(fig)
        mPlotWave1.setValue2DX(1/Fs,data)
        mPlotWave1.setXlim(0,T)
        mSpec1.setValueFig(fig)
    if k==1:
        mPlotWave1.setValue2DX(1/Fs,data1)
        mPlotWave1.setXlim(0,T)
        mSpec1.setValueFig(fig)

def setPlay(v):
    global Fs,N,CH,data,pObj;
    if N==0: return;
    if v==0: pObj=sa.play(1,Fs,data,0)
    if v==1: pObj.stop()
```

```
def setPlay1(v):
    global Fs,N,CH,data1,pObj;
    if N==0: return;
    if v==0: pObj=sa.play(1,Fs,data1,0)
    if v==1: pObj.stop()

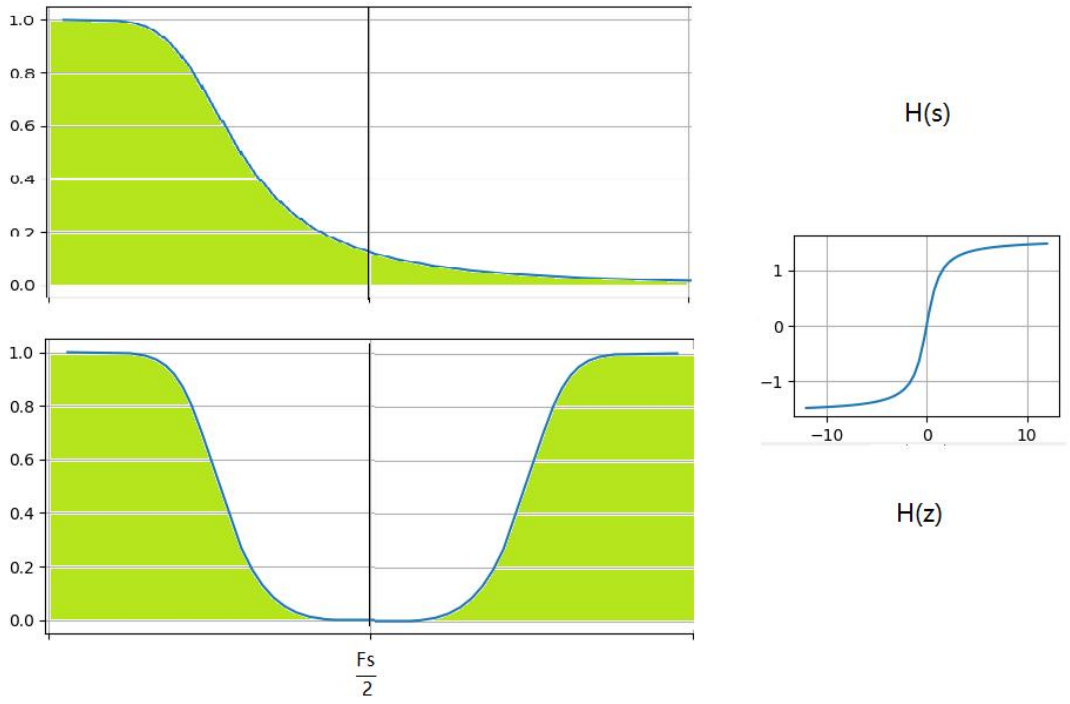
#==Main Window=====================
win= tk.Tk()
win.geometry("1260x700")
win.config(bg="#ddeeee")
#==Layout of controls=====================
mL1=dr.DRLabel(win,20,35,300,40,'#440000','#ffffff','音频信号的降噪实验',20)
mBut1=dr.DRButton(win,340,40,100,36,'#003355','#eeee00','打开文件',12,101)
mBut1.addCallBackSingle(setFile)
mHBut5=dr.DRHButtonGroup(win,440,40,140,36,'#003355','#ffffff','播放,停止',12,0)
mHBut5.addCallBackSingle(setPlay)
mBReset=dr.DRButton(win,580,40,80,36,'#003355','#eeee00','复位',12,101)
mBReset.addCallBackSingle(setReset)
mBand=dr.DRButton(win,690,40,100,36,'#003355','#eeee00','带通滤波',12,101)
mEntry1=dr.DREntry(win,790,40,80,36,'#ffffff','#000000','200',12)
mEntry2=dr.DREntry(win,870,40,90,36,'#ffffff','#000000','6000',12)
mBand.addCallBackSingle(setFilter1)
mBut12=dr.DRButton(win,960,40,100,36,'#003355','#eeee00','谱减法滤波',12,101)
mBut12.addCallBackSingle(setFilter)
mHBut51=dr.DRHButtonGroup(win,1060,40,160,36,'#003355','#ffffff','播放,停止',12,0)
mHBut51.addCallBackSingle(setPlay1)
mPlotWave=dr.DRPlot(win,20,90,600,150,'','#113344','#000000','#ffffff','#00eeff','#00ff00',0,0,-1,1,0,0)
mPlotWave1=dr.DRPlot(win,620,90,600,150,'','#113344','#000000','#ffffff','#00eeff','#00ff00',0,0,-1,1,0,0)
mSpec=dr.DRFigure(win,20,250,600,430,cb='#113344',cb1='#000000',cf='#ffffff',cf1='#009999',cf2='#ffff00')
mSpec1=dr.DRFigure(win,620,250,600,430,cb='#113344',cb1='#000000',cf='#ffffff',cf1='#009999',cf2='#ffff00')
#============================
openFile(mPath.getDRVIPath()+'\\dataset\\NoiseBird.mp3')
#==Main Loop=====================
win.mainloop()
```

双线性变换的频带压缩实验

双线性变换是 IIR 滤波器设计的一个主要放法，其核心思想是通过一个数学变换，将模拟滤波器的 s 平面映射到数字滤波器的 z 平面。

```
s = 2 * Fs * (z - 1) / (z + 1)
```

在这个过程中，模拟滤波器的频率空间{0-∞},被压缩到{0-Fs/2}内，从而可避免从模拟系统到数字系统转化过程中由于时域离散采样可能带来的频率混叠现象。



实验设计

用传递函数 H(s)定义一个模拟低通滤波器和一个模拟高通滤波器，求模拟滤波器的频率响应：

```
w, h = freqs(b, a, worN=np.logspace(-2, 2, 512))
```

用 w,h 绘制模拟滤波器频率特性，然后通过双线性变换进行频率映射：

wz=arctan(w)

用 wz,h 绘制映射后滤波器的频率特性曲线。对比观察区别。

双线性变换的频带压缩实验样例代码

```
import tkinter as tk
import numpy as np
from scipy import signal
import drvi.drviControlls as dr
#=======================================
def setData(v):
    ss='Fs/2:'+"{:.2f}".format(v/2)+' <--∞'
    mEntry1.setValueString(ss)
    fs=mHScale1.getValueSingle()/np.pi
    #Analog filter, H(s) = 1 / (s^3 + 2s^2 + 2s + 1)
```

```
    w1, h1 = signal.freqs([0,0,0,1],[1,2,2,1],worN=np.logspace(-2,2,256))
    mPlotFL0.setValue2D(w1,np.abs(h1))
    w12=fs*np.arctan(w1)
    w12=np.append(w12,[10])
    h1=np.append(h1,[0])
    mPlotFL1.setValue2D(w12,np.abs(h1))
    w2, h2 = signal.freqs([1,0,0,0],[1,4,8,8],worN=np.logspace(-2,2,256))
    mPlotFH0.setValue2D(w2,np.abs(h2))
    w22=fs*np.arctan(w2)
    M=len(w2)
    w22=np.append(w22,w22[M-1])
    h2=np.append(h2,[0])
    w22=np.append(w22,[10])
    h2=np.append(h2,[0])
    mPlotFH1.setValue2D(w22,np.abs(h2))

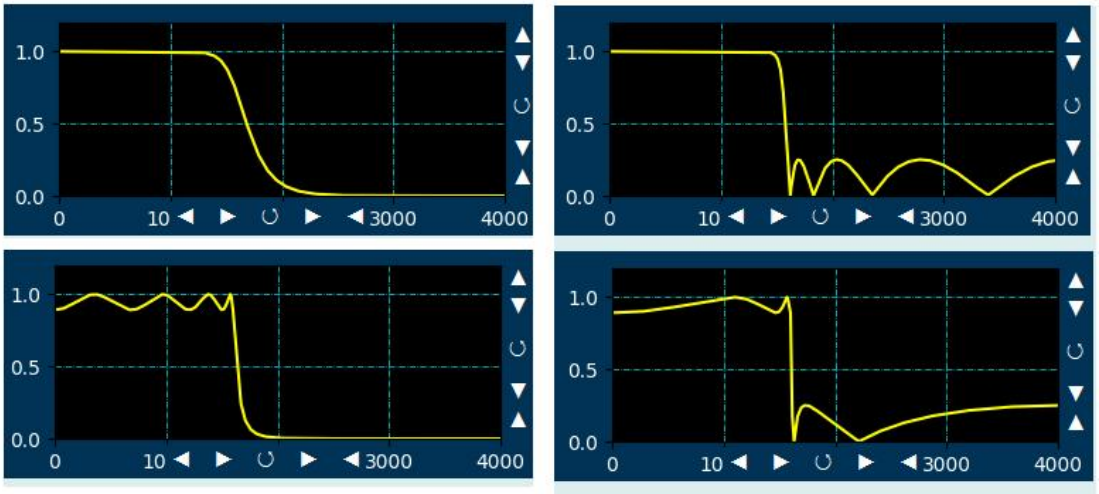
#==Main Window=====================
win= tk.Tk()
winWidthCur=1100
winHeightCur=640
win.geometry("%dx%d" %(winWidthCur,winHeightCur))
win.config(bg="#ddeeee")
#==Layout of controls=====================
mLab1=dr.DRLabel(win,20,30,450,40,'#003355','#ffffff','双线性变换的频率映射实验',20)
mPlotFL0=dr.DRPlot(win,20,100,520,240,'','#003355','#000000','#ffffff','#00eeff','#ffff00',0,10,0,1.1,0,0)
mPlotFH0=dr.DRPlot(win,530,100,520,240,'','#003355','#000000','#ffffff','#00eeff','#ffff00',0,10,0,1.1,0,0)
mPlotFL1=dr.DRPlot(win,20,340,520,240,'','#003355','#000000','#ffffff','#00eeff','#ffff00',0,10,0,1.1,0,0)
mPlotFH1=dr.DRPlot(win,530,340,520,240,'','#003355','#000000','#ffffff','#00eeff','#ffff00',0,10,0,1.1,0,0)
mEntry1=dr.DREntry(win,500,34,140,30,'#ffffff','#000000','∞  <--> 10',12)
mHScale1=dr.DRHScale(win,660,25,380,50,'#002266','#000000',0,20,10,11)
mHScale1.addCallBackSingle(setData)
setData(10);
#==Main Loop=====================
win.mainloop()
```

常用 IIR 滤波器的特点实验

IIR 滤波器采用又称为无限脉冲响应滤波器，IIR 就来源于名词“无限脉冲响应-Infinite Impulse Response”的英文单词首字符缩写。IIR 滤波器的传递函数为递归型结构：

$$H(z) = \sum_{k=0}^{N} a_k Z^{-k} / [1 + \sum_{k=1}^{M} b_k Z^{-k}]$$

IIR 滤波器的系数通常比 FIR 滤波器少很多，这意味着其滤波效率更高。常用的 IIR 滤波器种类包括：巴特沃斯滤波器、切比雪夫 I 型滤波器、切比雪夫 II 型滤波器和椭圆滤波器。



实验设计

生成一组由 500Hz、900Hz、1900Hz、2600Hz、3300HZ，然后分别设计巴特沃斯滤波器、切比雪夫 I 型滤波器、切比雪夫 II 型滤波器和椭圆滤波器。为夸大各类滤波器的特征，初始通带波纹度和阻带波纹度限制的很低。

常用 IIR 滤波器的特点实验样例代码

```
import tkinter as tk
import numpy as np
from scipy import signal
import drvi.drviDSP as dsp
import drvi.drviControlls as dr

def playing(v):
    global N,Fs,tData,xData1
    w=signal.windows.tukey(N,0.1);
    y=xData1*w
    sa.play(1,8000,y,0)

def setData():
    global N,Fs,tData,xData,xData1
    Fs=8192;
    dt=1.0/Fs;
    N=8192;
    tData=np.arange(N)*dt
```

```
    x1=0.25*np.sin(2*np.pi*500*tData)
    x2=0.25*np.sin(2*np.pi*1200*tData)
    x3=0.25*np.sin(2*np.pi*1900*tData)
    x4=0.25*np.sin(2*np.pi*2600*tData)
    x5=0.25*np.sin(2*np.pi*3300*tData)
    xData=x1+x2+x3+x4+x5
    xData1=xData
    mPlotWave.setValue2DFast(tData,xData)
    mPlotWave1.setValue2DFast(tData,xData)
    [mFre,mAmp]=dsp.AmpSpetrumX(N,Fs,xData)
    mPlotAmp.setValue2DFast(mFre,mAmp)
    mPlotAmp1.setValue2DFast(mFre,mAmp)

def myfilter(v):
    global N,Fs,bz,az
    F2=Fs/2;
    tT=mRBut.getValueSingle()
    M=mOMenu1.getValueSingle()+1
    RP=mEntry4.getValueSingle()
    RS=mEntry5.getValueSingle()
    f1=mHScale1.getValueSingle()/F2
    f2=mHScale2.getValueSingle()/F2
    if f1<0.01: f1=0;
    if f2>0.95: f2=1;
    if ((f1==0)&(f2==1)): f2=0.95
    if f1==0:
        mF=f2; mT='lowpass';
    if f2==1:
        mF=f1; mT='highpass';
    if ((f1>0)&(f2<1)):
        mF=[f1,f2]; mT='bandpass';
    if tT==0: bz,az=signal.butter(M,mF,mT)
    if tT==1: bz,az=signal.cheby1(M,RP,mF,mT)
    if tT==2: bz,az=signal.cheby2(M,RS,mF,mT)
    if tT==3: bz,az=signal.ellip(M,RP,RS,mF,mT)
    w, h = signal.freqz(bz, az);
    w=Fs/2*(w/np.pi)
    mPlotAmp2.setValue2DFast(w,abs(h))

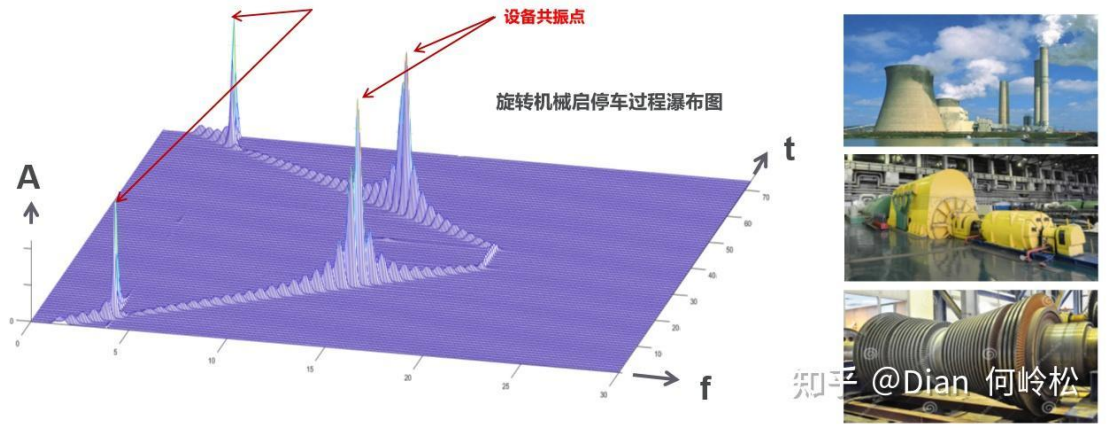
def filtering(v):
    global N,Fs,tData,xData,xData1,bz,az
    xData1 = signal.lfilter(bz,az,xData)
    mPlotWave1.setValue2DFast(tData,xData1)
    [mFre,mAmp]=dsp.AmpSpetrumX(N,Fs,xData1)
    mPlotAmp1.setValue2DFast(mFre,mAmp)

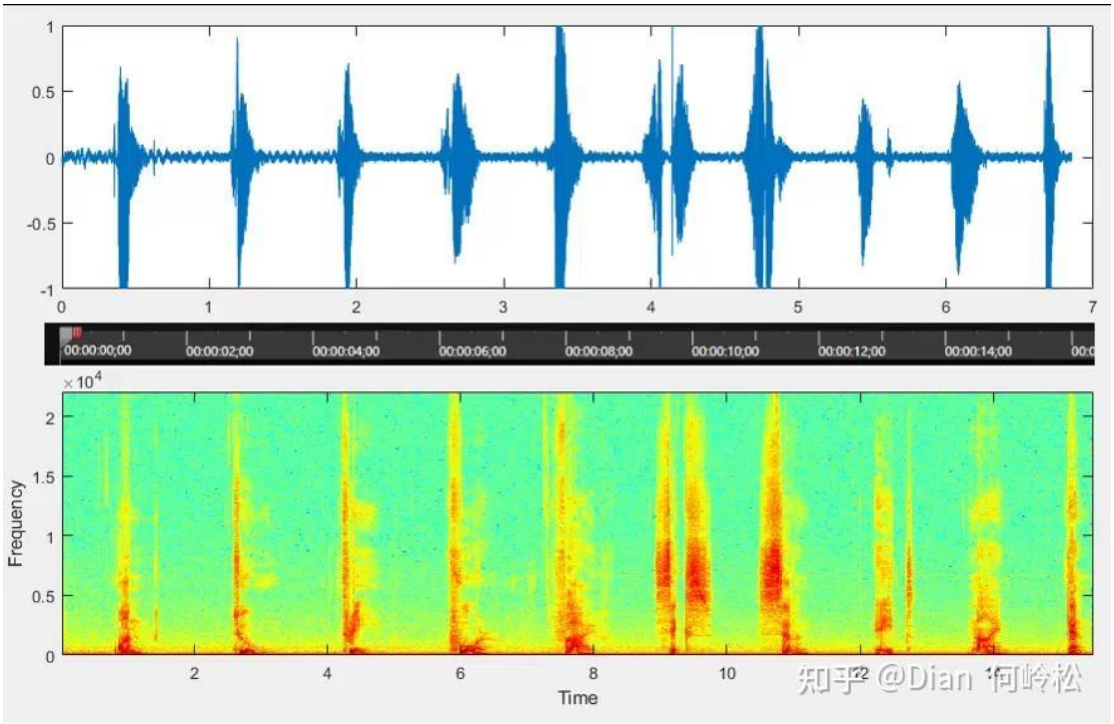
#==Main Window=======================
win= tk.Tk()
winWidthCur=1100
winHeightCur=640
```

```
win.geometry("%dx%d" %(winWidthCur,winHeightCur))
win.config(bg="#ddeeee")
#==Layout of controls=====================
mLab1=dr.DRLabel(win,20,30,420,40,'#003355','#ffffff','几种常用 IIR 滤波器性能特点实验',20)
mPlotWave=dr.DRPlot(win,20,110,520,240,xlow=0.01,xhigh=0.04,ylow=-1.25,yhigh=1.25)
mPlotAmp=dr.DRPlot(win,540,110,400,160,xlow=0,xhigh=4000,ylow=0,yhigh=0.4)
mPlotWave1=dr.DRPlot(win,20,350,520,240,xlow=0.01,xhigh=0.04,ylow=-1.25,yhigh=1.25)
mPlotAmp1=dr.DRPlot(win,540,270,400,160,xlow=0,xhigh=4000,ylow=0,yhigh=0.4)
mPlotAmp2=dr.DRPlot(win,540,430,400,160,xlow=0,xhigh=4000,ylow=0,yhigh=1.2)
mHScale1=dr.DRHScale(win,500,10,570,50,'#002266','#000000',0,4000,200,11)
mHScale2=dr.DRHScale(win,500,55,570,50,'#002266','#000000',0,4000,3500,11)
mOMenu1=dr.DROptionMenu(win,960,110,110,40,'#003355','#eeee00','N=1,N=2,N=3,N=4,N=5,N=6,N=7,N=8,N=9,N=10',12,7)
mRBut=dr.DRRadioBut(win,960,160,110,160,'#003355','#005577',"#ffff00",'巴特沃斯,切比雪夫 I,切比雪夫 II,椭圆滤波器',12,3,0)
mLab3=dr.DRLabel(win,960,340,110,30,'#bbeeff','#000000','通带波纹度 db',12)
mEntry4=dr.DRSpinBox(win,960,380,110,30,'#ccffcc','#000000',1,1,1,10,12)
mLab4=dr.DRLabel(win,960,420,110,30,'#bbeeff','#000000','阻带波纹度 db',12)
mEntry5=dr.DRSpinBox(win,960,460,110,30,'#ccffcc','#000000',14,2.5,5,50,12)
mBut3=dr.DRButton(win,960,510,110,40,'#003355','#eeee00','滤波',12,101)
mBut2=dr.DRButton(win,960,551,110,40,'#003355','#eeee00','播放',12,101)
mLab5=dr.DRLabel(win,600,135,300,20,'#000000','#ffffff','500 1200 1900 2600 3300Hz',12)
#==========================
mRBut.addCallBackSingle(myfilter)
mOMenu1.addCallBackSingle(myfilter)
mBut2.addCallBackSingle(playing)
mBut3.addCallBackSingle(filtering)
mHScale1.addCallBackSingle(myfilter)
mHScale2.addCallBackSingle(myfilter)
mEntry4.addCallBackSingle(myfilter)
mEntry5.addCallBackSingle(myfilter)
setData()
myfilter(0)
#==Main Loop=====================
win.mainloop()
```

典型时变信号的谱阵分析和语谱图分析实验

如果一个信号的幅值或者频率随着时间变化，则称其为时变信号。 对时变信号而言，其周期/频率、均值、有效值等特征参数也随着时间变化。这时，不应采用没有时间维度的傅里叶变换和频谱分析，而应采用添加了时间标尺的短时傅里叶变换和时频分析。时频分析有 2 种表示形式，工业上的谱阵分析，语音处理领域的语谱图。





实验设计

生成分段不同频率的多频信号、调频信号，以及调频+调幅信号，进行短时傅里叶变换，得到时频谱，然后并以分别以语谱图和谱阵图的方式显示。

典型时变信号的谱阵分析和语谱图分析实验样例代码

```
import numpy as np
import tkinter as tk
from scipy import signal
import matplotlib.pyplot as plt
import drvi.drviDSP as dsp
```

```
import drvi.drviControlls as dr

def setPos(v):
    global Fs,N,t,x
    K=int(v*N)
    if K<512: K=512
    xdata=x[K-512:K+512]
    mPlotWave2.setValue2DX(1/Fs,xdata);
    [mFre,mAmp]=dsp.AmpSpetrumX(1024,Fs,xdata)
    mPlotAmp2.setValue2DFast(mFre,mAmp)

def setValueSingle1(v):
    global Fs,N,t,x
    Fs=5000; N=18000; dt=1.0/Fs;
    t=np.arange(6000)*dt
    x1=0.8*np.sin(2*np.pi*200*t)
    x2=0.6*np.sin(2*np.pi*800*t)
    x3=0.4*np.sin(2*np.pi*1600*t)
    x=np.concatenate((x1,x2))
    x=np.concatenate((x,x3))
    mPlotWave.setValue2DX(1/Fs,x)
    setPos(0)

def setValueSingle2(v):
    global Fs,N,t,x
    Fs=5000; N=18000; dt=1.0/Fs;
    t=np.arange(N)*dt
    x=0.8*np.sin(2*np.pi*(10+300*t)*t)
    mPlotWave.setValue2DX(1/Fs,x)
    setPos(0)

def setValueSingle3(v):
    global Fs,N,t,x
    Fs=5000; N=18000; dt=1.0/Fs;
    t=np.arange(N)*dt
    x1=0.8*np.sin(2*np.pi*(10+300*t)*t)
    x2=0.5+0.4*np.sin(2*np.pi*1*t)
    x=x2.transpose()*x1
    mPlotWave.setValue2DX(1/Fs,x)
    setPos(0)

def setMesh(v):
    global Fs,N,t,x,xx,yy,Sxx
    ff,tt,Sxx=signal.spectrogram(x,Fs,nperseg=1024,noverlap=512)
    plt.pcolormesh(tt,ff,Sxx,shading='gouraud')
    plt.show()

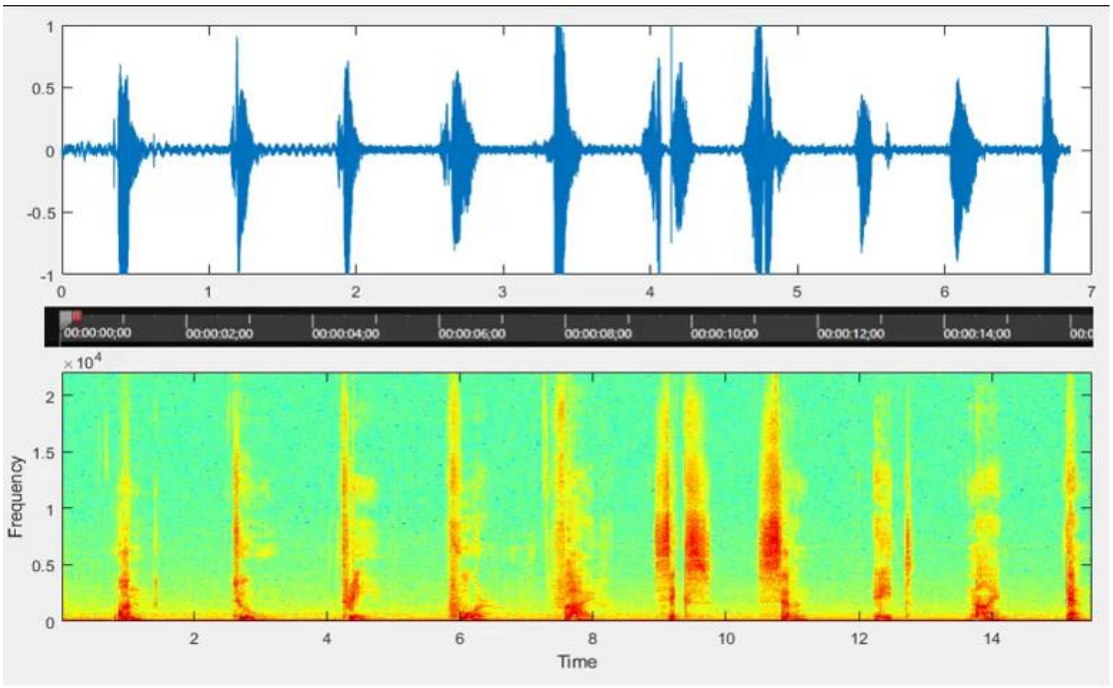
def setArray(v):
    global Fs,N,t,x,xx,yy,Sxx
    ff,tt,Sxx=signal.spectrogram(x,Fs,nperseg=1024,noverlap=512)
```

```
    fig = plt.figure()
    dsp.plotWaterfall(ff,tt,np.sqrt(Sxx),ax=plt,zDist=0.2)
    plt.show()

#==Main Window=======================
win= tk.Tk()
winWidthCur=1100
winHeightCur=700
win.geometry("%dx%d" %(winWidthCur,winHeightCur))
win.config(bg="#ddeeee")
#==Layout of controls=======================
mL1=dr.DRLabel(win,20,35,500,40,'#440000','#ffffff','典型信号的语谱图/谱阵图',20)
mBut1=dr.DRButton(win,550,40,100,36,'#003355','#ffffff','多频信号',12,101)
mBut2=dr.DRButton(win,650,40,100,36,'#003355','#ffffff','调频信号',12,101)
mBut3=dr.DRButton(win,750,40,120,36,'#003355','#ffffff','调频调幅信号',12,101)
mPlotWave=dr.DRPlot(win,20,90,1060,180,'','#113344','#000000','#ffffff','#00eeff','#00ff00',0,3,-1,1,0,0)
mBut4=dr.DRButton(win,900,40,80,36,'#003355','#ffffff','谱阵图',12,101)
mBut5=dr.DRButton(win,980,40,80,36,'#003355','#ffffff','语谱图',12,101)
mHSlider1=dr.DRHSlider(win,70,280,1000,30,'#444444','#004466','#008822',8,20,0,1,0)
mPlotWave2=dr.DRPlot(win,20,320,1060,180,'','#113344','#000000','#ffffff','#00eeff','#00ff00',0,0.2,-1,1,0,0)
mPlotAmp2=dr.DRPlot(win,20,500,1060,180,'','#113344','#000000','#ffffff','#00eeff','#00ff00',0,2500,0,1,0,0)
mHSlider1.addCallBackSingle(setPos)
mBut1.addCallBackSingle(setValueSingle1)
mBut2.addCallBackSingle(setValueSingle2)
mBut3.addCallBackSingle(setValueSingle3)
mBut4.addCallBackSingle(setArray)
mBut5.addCallBackSingle(setMesh)
#====================================
setValueSingle1(0)
#==Main Loop=======================
win.mainloop()
```

音频信号的语谱图分析实验

语谱图是表示语音信号频谱随时间变化的图形，它采用伪彩色图的方式来表达三维信息，其横轴为时间，纵轴为频率，任一给定时刻各频率的强弱则用色彩来表示。对语音信号分析来说，一般滑移窗长设定为 30 或 40 毫秒左右（44KHz, 1024 点前后）。



实验设计

用 Librosa 库的 open 函数打开不同的音频文件，鸟鸣、人声、吉他弹奏等，然后用语谱图分析功能显示它们的语谱图。

音频信号的语谱图分析实验样例代码

```
import tkinter as tk
from tkinter import filedialog
import numpy as np
import matplotlib.pyplot as plt
import librosa.display
import librosa
import drvi.drviControlls as dr
import drvi.drviSimpleAudio as sa
import drvi.drviAppendPath as mPath

def setValueSingle(v):
    mfile=filedialog.askopenfilename(title="选择文件",filetypes=(("MP3 files", "*.mp3"),("WAV files", "*.wav"),("all files", "*.*")))
    if not mfile: return 1
    openFile(mfile)

def setDemo(v):
    if v==0:
        mfile=mPath.getDRVIPath()+'\\dataset\\BGBird.mp3'
    if v==1:
        mfile=mPath.getDRVIPath()+'\\dataset\\oneTwoThree.mp3'
    if v==2:
```

```
        mfile=mPath.getDRVIPath()+'\\dataset\\TF_Giuta_DoRuiMe.mp3'
    openFile(mfile)

def openFile(mfile):
    global Fs,N,data
    data, Fs = librosa.load(mfile,mono=True,sr=None)
    N=len(data); T=N/Fs
    mPlotWave.setValue2DX(1/Fs,data)
    mPlotWave.setXlim(0,T)
    doSpec(0)

def doSpec(v):
    v=mOpt.getValueSingle()
    if v==0: setSpecGram()
    if v==1: setLogSpecGram()
    if v==2: setMelSpecGram()

def setSpecGram():
    global Fs,N,data
    y = librosa.amplitude_to_db(np.abs(librosa.stft(data)), ref=np.max)
    plt.close('all')
    fig, ax = plt.subplots(1,1)
    img=librosa.display.specshow(y, sr=Fs, x_axis='time', y_axis='linear')
    plt.ylim(10, 8500)
    fig.colorbar(img, ax=ax, format="%+2.f dB")
    mPlotArray.setValueFig(fig)

def setLogSpecGram():
    global Fs,N,data
    y = librosa.amplitude_to_db(np.abs(librosa.stft(data)), ref=np.max)
    plt.close('all')
    fig, ax = plt.subplots(1,1)
    img=librosa.display.specshow(y, sr=Fs, x_axis='time', y_axis='log')
    plt.ylim(10, 8500)
    fig.colorbar(img, ax=ax, format="%+2.f dB")
    mPlotArray.setValueFig(fig)

def setMelSpecGram():
    global Fs,N,data
    melspec = librosa.feature.melspectrogram(y=data,sr=Fs)
    logmelspec = librosa.power_to_db(melspec,ref=np.max)
    plt.close('all')
    fig, ax = plt.subplots(1,1)
    img=librosa.display.specshow(logmelspec, sr=Fs, x_axis='time', y_axis='mel')
    plt.ylim(10, 8500)
    fig.colorbar(img, ax=ax, format="%+2.f dB")
    mPlotArray.setValueFig(fig)

def setPlay(v):
    global Fs,N,CH,data,mPosition,mLength,pObj;
```

```
    if N==0: return;
    if v==0: pObj=sa.play(1,Fs,data,0)
    if v==1: pObj.stop()

def setTime(v):
    global Fs,N,CH,data,xdata,mPosition,mLength,pObj;
    if Fs==0:
        mTimer2.setValueSingle(1)
        return;
    T=N/Fs
    v=v/1000
    if v>T:
        pObj.stop()
        mTimer2.setValueSingle(1)
    else:
        n1=int(v*Fs)
        n2=int(0.15*Fs)
        xx=data[n1:n1+n2]
        mPlotWave1.setValue2DX(1/Fs,xx)

#==Main Window=======================
win= tk.Tk()
win.geometry("1100x700")
win.config(bg="#ddeeee")
#==Layout of controls=======================
mL1=dr.DRLabel(win,20,20,400,40,'#440000','#ffffff','音乐文件语谱图分析',20)
mBut1=dr.DRButton(win,630,25,100,36,'#003355','#eeee00','打开文件',12,101)
mOMenu=dr.DROptionMenu(win,740,25,100,36,'#003355','#eeee00','样例 1,样例 2,样例 3',12,0)
mPlotWave=dr.DRPlot(win,20,75,660,150,ylow=-1,yhigh=1)
mPlotWave1=dr.DRPlot(win,680,75,400,150,ylow=-1,yhigh=1)
mHBut5=dr.DRHButtonGroup(win,850,25,160,36,'#003355','#ffffff','播放,停止',12,0)
mHBut5.addCallBackSingle(setPlay)
mTimer2=dr.DRTimer(win,1020,25,60,36,'#000000','#ffff00',150,11)
mTimer2.addCallBackSingle(setTime)
mHBut5.addCallBackSingle(mTimer2.setValueSingle)
mBut1.addCallBackSingle(setValueSingle)
mPlotArray=dr.DRFigure(win,20,235,1060,450,cb='#003355',cb1='#000000',cf='#ffffff',cf1='#009999',cf2='#ffff00')
mOpt=dr.DROptionMenu(win,470,25,120,36,txt='线性,对数,梅尔',val=2)
mOpt.addCallBackSingle(doSpec)
mOMenu.addCallBackSingle(setDemo)
#============================================
mPosition=0
mLength=8192
setDemo(0)
#==Main Loop=======================
win.mainloop()
```

工业信号的谱阵分析实验

在旋转机械临界转速测量的启停车实验等工业测量信号分析的场合，谱阵分析是一种常用的手段。其做法就是沿时间轴将整个实验信号分为若干个段，然后计算各段信号的频谱，最后再以谱阵图的方式显示。



实验设计

用 Librosa 库的 open 函数打开不同的以音频文件保存机器变速实验的噪声信号，汽车发动机加减速噪声、电磁加速弹射噪声，消防车、教堂钟声等，然后用谱阵功能进行分析。

工业信号的谱阵分析实验样例代码

```
import tkinter as tk
from tkinter import filedialog
import numpy as np
from scipy import signal
import drvi.drviDSP as dsp
import drvi.drviControlls as dr
import drvi.drviSimpleAudio as sa
import librosa
import drvi.drviAppendPath as mPath

def setValueSingle(v):
    mfile=filedialog.askopenfilename(title="选择文件",filetypes=(("MP3 files", "*.mp3"),("WAV files", "*.wav"),("all files", "*.*")))
    if not mfile: return 1
    openFile(mfile)

def setDemo(v):
    if v==0:
        mfile=mPath.getDRVIPath()+'\\dataset\\TF_CARSpeedup.mp3'
    if v==1:
```

```
        mfile=mPath.getDRVIPath()+'\\dataset\\TF_SpaceShipSpeedup.mp3'
    if v==2:
        mfile=mPath.getDRVIPath()+'\\dataset\\TF_alarm.mp3'
    openFile(mfile)

def openFile(mfile):
    global Fs,N,data,mSpec
    data, Fs = librosa.load(mfile,mono=True,sr=None)
    N=len(data)
    mPlotWave.setValue2DX(1/Fs,data)
    T=N/Fs; mPlotWave.setXlim(0,T)
    MM=len(data)
    kk=int(MM/150)
    ff,tt,Sxx=signal.spectrogram(data,Fs,nperseg=kk,noverlap=0)
    mPlotArray.setValue3D(tt,ff,Sxx)

def setPlay(v):
    global Fs,N,CH,data,mPosition,mLength,pObj;
    if N==0: return;
    if v==0: pObj=sa.play(1,Fs,data,0)
    if v==1: pObj.stop()

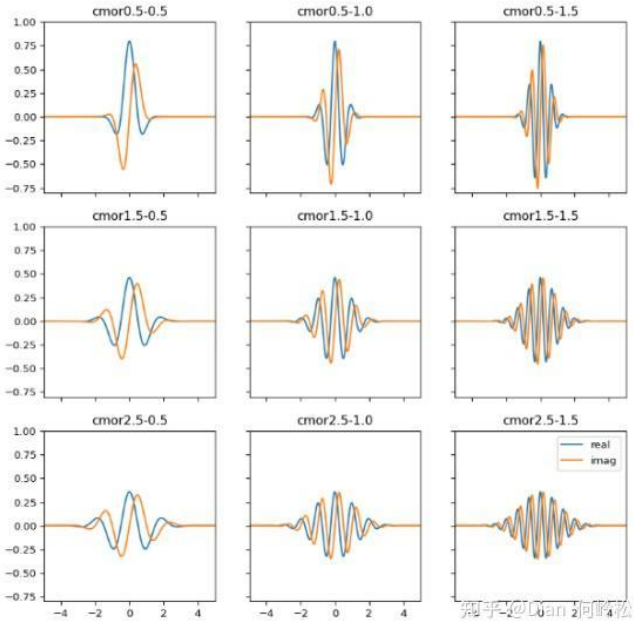
def setTime(v):
    global Fs,N,CH,data,xdata,mPosition,mLength,pObj;
    if Fs==0:
        mTimer2.setValueSingle(1)
        return;
    T=N/Fs
    v=v/1000
    if v>T:
        pObj.stop()
        mTimer2.setValueSingle(1)

#==Main Window=====================
win= tk.Tk()
winWidthCur=1100
winHeightCur=720
win.geometry("%dx%d" %(winWidthCur,winHeightCur))
win.config(bg="#ddeeee")
#==Layout of controls=====================
mL1=dr.DRLabel(win,20,35,500,40,'#440000','#ffffff','音频信号的谱阵分析',20)
mBut1=dr.DRButton(win,575,40,100,36,'#003355','#eeee00','打开文件',12,101)
mOMenu=dr.DROptionMenu(win,700,40,100,36,'#003355','#eeee00','样例 1,样例 2,样例 3',12,0)
mPlotWave=dr.DRPlot(win,20,90,1060,150,'','#113344','#000000','#ffffff','#00eeff','#00ff00',0,0,-1,1,0,0)
mHBut5=dr.DRHButtonGroup(win,840,40,160,36,'#003355','#ffffff','播放,停止',12,0)
mHBut5.addCallBackSingle(setPlay)
mTimer2=dr.DRTimer(win,1020,40,50,36,'#000000','#ffff00',500,11)
mTimer2.addCallBackSingle(setTime)
mHBut5.addCallBackSingle(mTimer2.setValueSingle)
mBut1.addCallBackSingle(setValueSingle)
```

```
mPlotArray=dr.DRwaterFall(win,20,250,1060,450,xDist=0.02,fhigh=4000)
mOMenu.addCallBackSingle(setDemo)
#==========================================
mPosition=0; mLength=8192;
setDemo(0)
#==Main Loop=======================
win.mainloop()
```

小波函数的时移和频移实验

小波变换就是将信号投影到由展缩和平移后形成的小波函数集上，得到信号在各小波函数上的投影系数，也就是小波系数。小波函数不同，则小波变换的结果也不相同。因此了解常用小波函数时域/频域特性，对应用好小波变换来说至关重要。



实验设计

安装 pywt 小波分析扩展库，它提供了常用的小波分析函数，其安装命令为：

pip install PyWavelets

其中提供了一个 wavelist 函数，可以列出该库支持的所有小波函数，下面是列出其中的连续小波函数：

wtName=pywt.wavelist(family=None, kind='continuous')

从中选择一个小波函数，就可以显示其波形：

```
wav = pywt.ContinuousWavelet('mexh')
wav.lower_bound=-10
wav.upper_bound=10
[psi, xval] =wav.wavefun(length=2048)
```

做 FFT，则可显示出其对应的频率特性。

小波函数的时移和频移实验样例代码

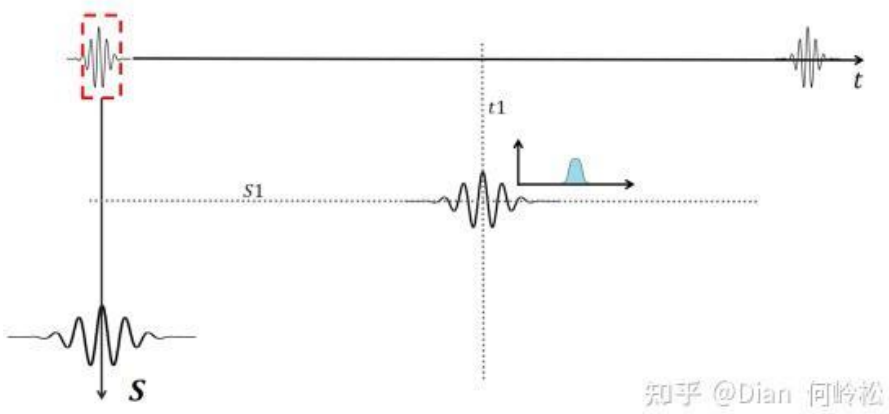
```
import tkinter as tk
import numpy as np
import pywt
import drvi.drviDSP as dsp
import drvi.drviControlls as dr
```

```
def setData(tt):
    wName='morl'
    t=mOMenu1.getValueSingle()
    if t==0: wName='morl'
    if t==1: wName='mexh'
    if t==2: wName='gaus4'
    if t==4: wName='shan1.5-1.0'
    if t==3: wName='fbsp2-1-1'
    wav = pywt.ContinuousWavelet(wName)
    wav.lower_bound=-10
    wav.upper_bound=10
    [y,x] =wav.wavefun(length=16384)
    s=1+mHSlider1.getValueSingle()/10
    x=x*s
    mPlotWave.setValue2DFast(x,y)
    mFs=160/s
    mLen=len(y)
    [mFre,mAmp]=dsp.AmpSpetrum(mLen,mFs,y)
    mAmp=mAmp/max(mAmp)
    mPlotAmp.setValue2DFast(mFre,mAmp)
    x0=10*mHSlider2.getValueSingle()/100
    N1=int(x0*16384)
    if N1>(16384*9) : N1=16384*9
    N2=16384*9-N1
    y11=np.zeros(N1);
    y12=np.zeros(N2);
    y2=np.append(y11,y)
    y2=np.append(y2,y12)
    mPlotWave0.setValue2DX(0.001,y2)

#==Main Window=====================
win= tk.Tk()
winWidthCur=1100
winHeightCur=670
win.geometry("%dx%d" %(winWidthCur,winHeightCur))
win.config(bg="#ddeeee")
#==Layout of controls=====================
mLab1=dr.DRLabel(win,20,30,400,40,'#003355','#ffffff','小波函数的时移和频移特性',20)
mOMenu1=dr.DROptionMenu(win,460,30,160,36,'#003355','#eeee00','Morlet,MexicanHat,Gaussian-4,B-Spline-2-1-1,Shanno-1.5-1',12,0)
mOMenu1.addCallBackSingle(setData)
mHSlider1=dr.DRHSlider(win,850,30,200,30,'#444444','#880000','#008822',8,20,0,100,0)
mHSlider1.addCallBackSingle(setData)
mHSlider2=dr.DRHSlider(win,650,30,200,30,'#444444','#880000','#008822',8,20,0,100,0)
mHSlider2.addCallBackSingle(setData)
mPlotWave0=dr.DRPlot(win,20,80,1040,220,'','#003355','#000000','#ffffff','#00eeff','#ffff00',0,165,-1.5,1.5,0,0)
mPlotWave=dr.DRPlot(win,20,240+80,520,320,'','#003355','#000000','#ffffff','#00eeff','#ffff00',-10,10,-1.5,1.5,0,0)
mPlotAmp=dr.DRPlot(win,540,240+80,520,320,'','#003355','#000000','#ffffff','#00eeff','#ffff00',0,0.5,0,1.25,0,0)
setData(0)
#==Main Loop=====================
win.mainloop()
```

时变信号的连续小波变换实验

选定小波函数后，用尺度参数 S 和平移参数 τ 对小波函数进行连续展缩和平移，然后计算信号投影到时频面上每一个展缩/平移小波上的小波系数，再将其以伪彩色图形式显示，这个计算过程就是连续小波变换。



实验设计

生成分段频率的多频信号和扫频波信号，选择一个连续小波函数；然后用 pywt 库中的连续小波函数 cwt 进行连续小波变换，如下所示：

coef, freqs=pywt.cwt(y,S,'morl')

最后用 colormesh 或 contourf 显示即可

plt.contourf(tt,freqs*Fs,abs(coef))

时变信号的连续小波变换实验样例代码

```
import numpy as np
import pywt
import tkinter as tk
import drvi.drviDSP as dsp
import drvi.drviControlls as dr

def getScale(M,wName):
    fc=pywt.central_frequency(wName)
    S1=2*fc
    S=np.geomspace(S1,1024,num=M)
    return S

def setValueSingle1(v):
    global Fs,N,t,x,mName,dType
    dType=0
    Fs=4000;
    N=256;
    dt=1.0/Fs;
    t=np.arange(N)*dt;
```

```
    x1=np.sin(2*np.pi*200*t)
    x2=np.sin(2*np.pi*400*t)
    x3=np.sin(2*np.pi*800*t)
    x=np.concatenate((x1,x2))
    x=np.concatenate((x,x3))
    t=np.arange(3*N)*dt;
    mPlotWave.setValue2DX(1/Fs,x)

def setValueSingle2(v):
    global Fs,N,t,x,mName,dType
    dType=1
    Fs=4000;
    N=256;
    dt=1.0/Fs;
    t=np.arange(3*N)*dt;
    x=np.sin(2*np.pi*(100+3000*t)*t)
    mPlotWave.setValue2DX(1/Fs,x)

def setValueSingle3(v):
    global mName
    wav = pywt.ContinuousWavelet(mName)
    wav.lower_bound=-10
    wav.upper_bound=10
    [psi, xval] =wav.wavefun(length=2048)
    mPlotWave1.setValue2D(xval,psi.real)
    mPlotWave2.setValue2D(xval,psi.imag)

def setValueSingle4(v):
    global Fs,N,t,x,mName,dType
    S=getScale(128,mName)
    coef, freqs=pywt.cwt(x,S,mName)
    mPCon2.setValue3D(t,freqs*Fs,abs(coef))
    mTab.setPage(0)

def setValueType(v):
    global mName
    if v==0: mName='gaus1'
    if v==1: mName='gaus4'
    if v==2: mName='gaus8'
    if v==3: mName='cgau1'
    if v==4: mName='cgau4'
    if v==5: mName='cgau8'
    if v==6: mName='morl'
    if v==7: mName='cmor1.0-0.5'
    if v==8: mName='cmor1.0-1.0'
    if v==9: mName='cmor1.0-1.5'
    if v==10: mName='fbsp1-1.0-0.5'
    if v==11: mName='fbsp1-1.0-1.0'
    if v==12: mName='fbsp1-1.0-1.5'
    if v==13: mName='shan1.0-0.5'
```

```
    if v==14: mName='shan1.0-1.0'
    if v==15: mName='shan1.0-1.5'
    mTab.setPage(1)
    setValueSingle3(0)

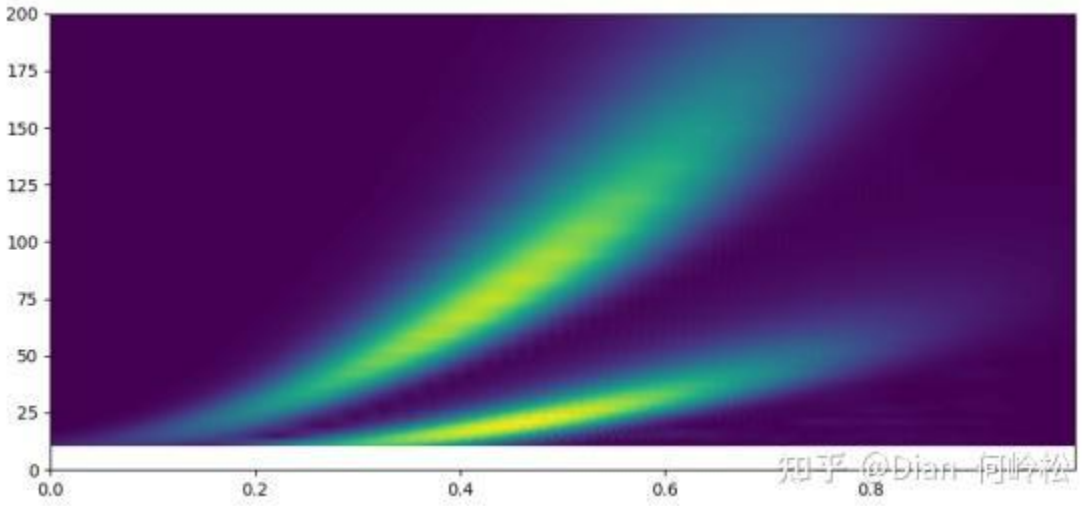
#==Main Window=======================
win= tk.Tk()
winWidthCur=1100
winHeightCur=700
win.geometry("%dx%d" %(winWidthCur,winHeightCur))
win.config(bg="#ddeeee")
#==Layout of controls=================
mL1=dr.DRLabel(win,20,35,500,40,'#440000','#ffffff','时变信号的连续小波变换',20)
mBut1=dr.DRButton(win,680,40,100,36,'#003355','#ffffff','多频信号',12,101)
mBut2=dr.DRButton(win,780,40,100,36,'#003355','#ffffff','调频信号',12,101)
mBut3=dr.DRButton(win,900,40,140,36,'#003355','#ffffff','小波函数',12,101)
mBut4=dr.DRButton(win,560,40,100,36,'#003355','#ffffff','CWT',12,101)
mPlotWave=dr.DRPlot(win,20,90,860,160)
mTab=dr.DRNoteBook(win,20,260,860,420,txt='A,B')
mPCon2=dr.DRColormesh(mTab.win[0],10,10,840,370)
mPlotWave1=dr.DRPlot(mTab.win[1],10,10,420,370,'','#113344','#000000','#ffffff','#00eeff','#00ff00',-10,10,-1.25,1.25,0,0)
mPlotWave2=dr.DRPlot(mTab.win[1],430,10,420,370,'','#113344','#000000','#ffffff','#00eeff','#00ff00',-10,10,-1.25,1.25,0,0)
sName='gaus1,gaus4,gaus8,cgau1,cgau4,cgau8,'
sName=sName+'morl,cmor1.0-0.5,cmor1.0-1.0,cmor1.0-1.5,'
sName=sName+'fbsp1-1.0-0.5,fbsp1-1.0-1.0,fbsp1-1.0-1.5,'
sName=sName+'shan1.0-0.5,shan1.0-1.0,shan1.0-1.5'
mLBut1=dr.DRListBox(win,900,90,140,590,'#ddffdd','#000000',sName,12,0)
mLBut1.addCallBackSingle(setValueType)
mBut1.addCallBackSingle(setValueSingle1)
mBut2.addCallBackSingle(setValueSingle2)
mBut4.addCallBackSingle(setValueSingle4)
mBut3.addCallBackSingle(setValueSingle3)
#=====================================
mName='gaus1'
setValueSingle1(0)
setValueSingle3(0)
setValueSingle4(0)
#==Main Loop=========================
win.mainloop()
```

从连续小波变换到离散小波变换实验

连续小波变换功能非常强，但计算量也非常大。其情形就象连续傅里叶变换。从应用角度，为了降低计算量，必须对尺度因子进行离散化。

s=[s1,s2,s3,s4,.....,]

尺度 s 选的计算点越多，连续小波变换的结果就越精细。

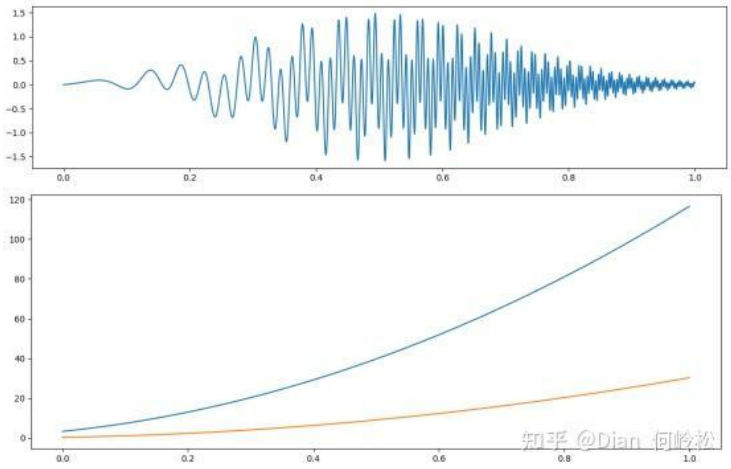


实验设计

按不同的 s 计算点数和线性、对数取值方式，生成尺度 s 的取样点，然后进行连续小波变换，对比不同 s 取样，也就是离散化后的小波变换结果，如：

s=np.arange(2,256)

测试信号采用 pywt 库给出的双扫频信号，以及分段多频率信号和扫频波信号。



从连续小波变换到离散小波变换实验样例代码

```
import numpy as np
import pywt
import tkinter as tk
import drvi.drviDSP as dsp
```

```
import drvi.drviControlls as dr

def gaussian(x, x0, sigma):
    return np.exp(-np.power((x - x0) / sigma, 2.0) / 2.0)

def make_chirp(t, t0, a):
    frequency = (a * (t + t0)) ** 2
    chirp = np.sin(2 * np.pi * frequency * t)
    return chirp, frequency

#==Event response function of controls==
def setValueSingle0(v):
    global Fs,N,t,x
    Fs=2000;N=2000; dt=1/2000
    t=np.arange(N)*dt;
    chirp1, frequency1 = make_chirp(t, 0.05, 22)
    chirp2, frequency2 = make_chirp(t, 0.05, 12)
    x = chirp1 + 0.6 * chirp2
    x *= gaussian(t, 0.5, 0.2)*0.5
    mPlotWave.setValue2DX(1/Fs,x)

def setValueSingle1(v):
    global Fs,N,t,x
    Fs=4000;
    N=256;
    dt=1.0/Fs;
    t=np.arange(N)*dt;
    x1=np.sin(2*np.pi*200*t)
    x2=np.sin(2*np.pi*400*t)
    x3=np.sin(2*np.pi*800*t)
    x=np.concatenate((x1,x2))
    x=np.concatenate((x,x3))
    mPlotWave.setValue2DX(1/Fs,x)
    t=np.arange(3*N)*dt;

def setValueSingle2(v):
    global Fs,N,t,x
    dType=1
    Fs=4000;
    N=256;
    dt=1.0/Fs;
    t=np.arange(3*N)*dt;
    x=np.sin(2*np.pi*(100+3000*t)*t)
    mPlotWave.setValue2DX(1/Fs,x)

def setCWT(v):
    global Fs,N,t,x,mName,dType
    sType=mOMenu1.getValueSingle()
    if sType==1:
        S=np.arange(2,128)
```

```
    if sType==2:
        k=np.arange(8,65)*(1/8);
        S=pow(2,k)
    if sType==3:
        k=np.arange(6,49)*(1/6)
        S=pow(2,k)
    if sType==4:
        k=np.arange(3,25)*(1/3)
        S=pow(2,k)
    if sType==5:
        k=np.arange(1,9)*(1/1)
        S=pow(2,k)
    if sType==0:
        S=np.geomspace(1,1024,num=256)
    coef, freqs=pywt.cwt(x,S,'cmorl1.5-1')
    mPCon2.setValue3D(t,freqs*Fs,abs(coef))

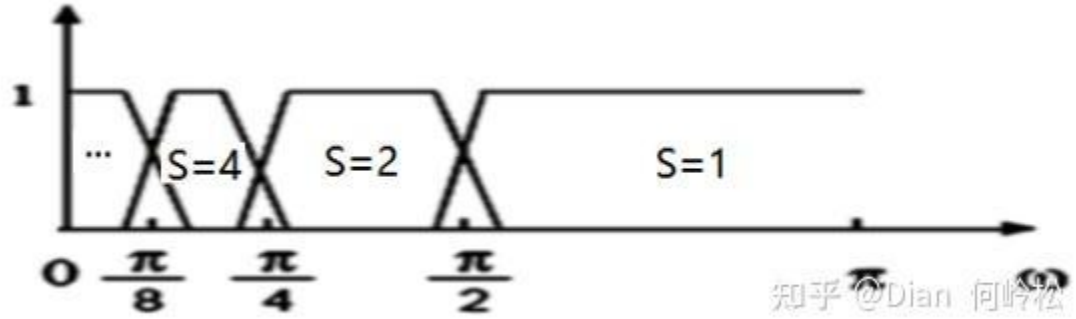
#==Main Window=====================
win= tk.Tk()
winWidthCur=1100
winHeightCur=700
win.geometry("%dx%d" %(winWidthCur,winHeightCur))
win.config(bg="#ddeeee")
#==Layout of controls=====================
mL1=dr.DRLabel(win,20,35,500,40,'#440000','#ffffff','从连续小波变换到离散小波变换',20)
mBut1=dr.DRButton(win,680,40,100,36,'#003355','#ffffff','多频信号',12,101)
mBut2=dr.DRButton(win,780,40,100,36,'#003355','#ffffff','调频信号',12,101)
mOMenu1=dr.DROptionMenu(win,900,40,160,36,'#003355','#eeee00','CWT(256),线形展缩(128),1/8 倍频程(58),1/6 倍频程(44),1/3 倍频程
(22),1/1 倍频程(8)',12,5)
mBut4=dr.DRButton(win,536,40,140,36,'#003355','#ffffff','双频调频调幅信号',12,101)
mPlotWave=dr.DRPlot(win,20,90,1040,160,'','#113344','#000000','#ffffff','#00eeff','#00ff00',0,0,-1,1,0,0)
mL2=dr.DRLabel(win,20,260,1040,420,'#003355','#ffffff','',20)
mPCon2=dr.DRPlotContour(win,30,270,1020,400,cmap='bone')
mBut1.addCallBackSingle(setValueSingle1)
mBut2.addCallBackSingle(setValueSingle2)
mBut4.addCallBackSingle(setValueSingle0)
mOMenu1.addCallBackSingle(setCWT)
#==================================
setValueSingle0(0)
setCWT(5)
#==Main Loop=====================
win.mainloop()
```

离散小波变换的数字滤波算法实验

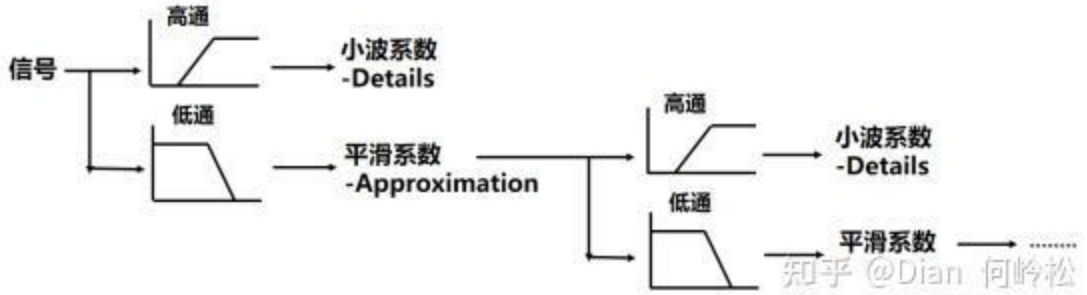
所谓离散(二进)小波变换就是展缩小波的尺度因子按 2 的幂次方进行离散化：

S=2^j； j=0,1,2,3,4,....

这时，其各阶展缩小波的频带按 1/1 倍频程方式划分和递进。



在离散二进小波变换的特殊条件下，小波变换可以用滤波器组实现：



之所以在尺度二进时才有滤波器组算法，是因为这时只需设计出第一级的滤波器对，其它各级的小波变换滤波器对可以在第一级滤波器的基础上，通过在滤波器系数中简单补零实现。

$$\begin{cases} h_{jk}=h_{1l};\ g_{jk}=g_{1l} & k=2^{j-1}l \\ h_{jk}=0;\ g_{jk}=0 & k=\text{其它} \end{cases} \qquad j=1,2,\ldots\ldots P$$

知乎 @Dian 何岭松

实验设计

定义一对小波滤波器：

h1=[-0.014,0.050,0.018,-0.097,-0.020,0.315,0.521,0.315,-0.020,-0.097,0.018,0.050,-0.014];

g1=[-0.014,-0.050,0.018,0.097,-0.020,-0.315,0.521,-0.315,-0.020,0.097,0.018,-0.050,-0.014];

然后通过补零的方式实现其它级小波滤波器系数，然后显示滤波器特性。

离散小波变换的数字滤波算法实验样例代码

```
import tkinter as tk
```

```
import numpy as np
from scipy import signal
from scipy.fftpack import fft,ifft
import matplotlib.pyplot as plt
import drvi.drviDSP as dsp
import drvi.drviControlls as dr

#==================================
def getWTFilter(v):
    h1=[-0.014,0.050,0.018,-0.097,-0.020,0.315,0.521,0.315,-0.020,-0.097,0.018,0.050,-0.014];
    g1=[-0.014,-0.050,0.018,0.097,-0.020,-0.315,0.521,-0.315,-0.020,0.097,0.018,-0.050,-0.014];
    N=len(h1)
    if v==0: return N,h1,g1
    M=2**(v-1)
    N1=(M+1)*N
    h=np.zeros(N1-M)
    g=np.zeros(N1-M)
    for i in range(N):
        h[(M+1)*i]=h1[i]
        g[(M+1)*i]=g1[i]
    N1=len(h)
    return N1,h,g

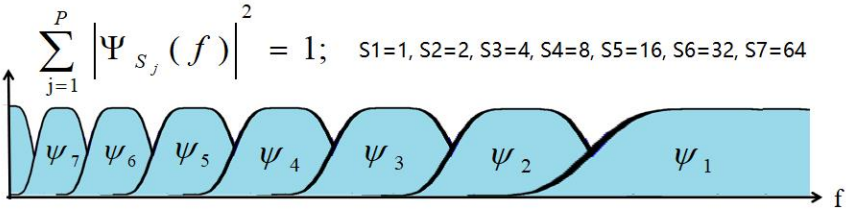
def working():
    fig1,ax1=plt.subplots(nrows=5,ncols=2)
    fig2,ax2=plt.subplots(nrows=5,ncols=2)
    fig3,ax3=plt.subplots()
    fig1.tight_layout()
    fig2.tight_layout()
    fig3.tight_layout()
    for i in range(5):
        N,h1,g1=getWTFilter(i)
        w,h=signal.freqz(h1)
        Ah=np.abs(h)
        k=np.arange(N)-int(N/2)
        ax1[i][0].stem(k,h1)
        ax1[i][1].plot(w,Ah)
        ax1[i][0].set_xlim(-31,31)
        ax1[i][1].set_xlim(0,np.pi)
        #============
        w,h=signal.freqz(g1)
        Ag=np.abs(h)
        ax2[i][0].stem(k,g1)
        ax2[i][1].plot(w,Ag)
        ax2[i][0].set_xlim(-31,31)
        ax2[i][1].set_xlim(0,np.pi)
        #==============
        if i==0:
            ax3.plot(w,Ag)
            A=Ah
```

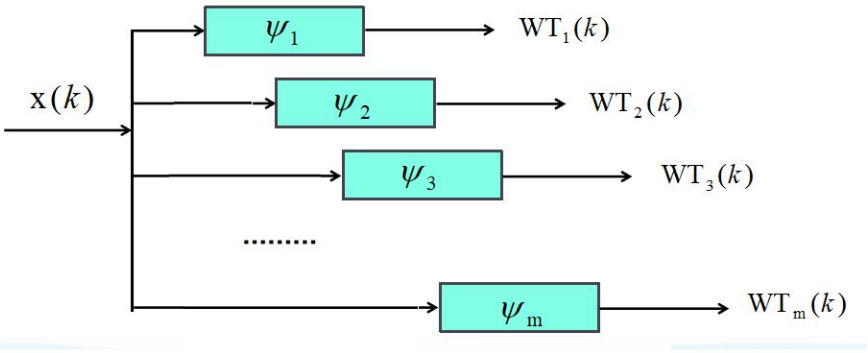
```
            else:
                Ak=A*Ag
                ax3.plot(w,Ak)
                A=A*Ah
            if i==4:
                ax3.plot(w,A)
            ax3.set_xlim(0,np.pi)
        mFig1.setValueFig(fig1)
        mFig2.setValueFig(fig2)
        mFig3.setValueFig(fig3)

#==Main Window=======================
win= tk.Tk()
win.wm_title('离散小波变换的数字滤波算法实验')
winWidthCur=1120
winHeightCur=680
win.geometry('1120x680')
win.config(bg="#ddeeee")
#==Layout of controls================
mTab=dr.DRNoteBook(win,10,10,1100,660,'#dddddd','#888888','#000000','低通滤波器,高通滤波器,多尺度分解')
mFig1=dr.DRFigure(mTab.win[0],5,5,1090,650)
mFig2=dr.DRFigure(mTab.win[1],5,5,1090,650)
mFig3=dr.DRFigure(mTab.win[2],5,5,1090,650)
#====================
working()
#==Main Loop=====================
win.mainloop()
```

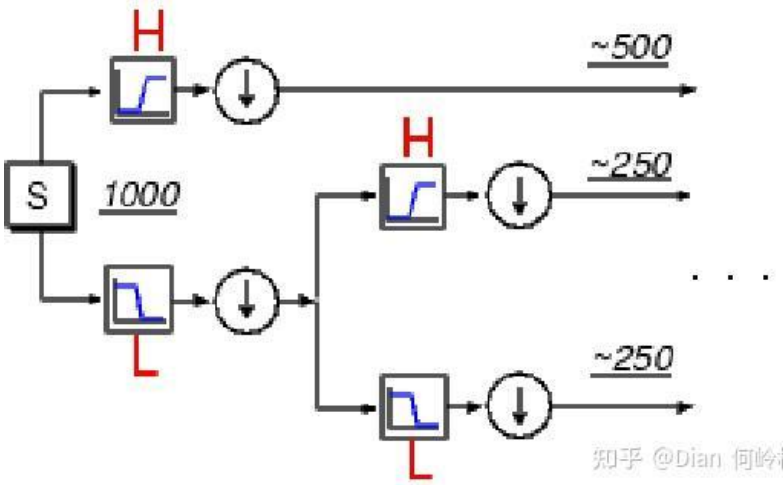
离散小波变换的多尺度分解与重构实验

当采用滤波器组快速算法时，离散小波变换将信号滤波分解为一组位于不同频带的小波系数，也就是对按频带对信号进行多尺度分解，这也称之为 SWT (Stationary Wavelet Transform)变换：





SWT 将信号分解为位于不同二进频段的小波系数，每增加一级小波变换，都将增加一个数据块的量。从频域角度，小波系数中有大量的冗余。为此，可采用数据选抽的方式去除分解后小波系数中的数据冗余。改进后的算法称之为 DWT：



由于选抽操作，信号采样频率由 Fs 降低为 Fs/2；选抽后的高通滤波信号部分存在严重的频率混叠，这时得到的小波系数波形无物理意义。

实验设计

生成分段不同频率的多频信号和扫频信号，然后分别用 SWT 和 DWT 对信号进行分解与重构，体会二者分解出的小波系数的异同。下面是 4 层 SWT 变换的样例代码：

```
wtCoeff=pywt.swt(x,'db8',level=4)
z=pywt.iswt(wtCoeff,'db8')
```

这是是 4 层 DWT 变换的样例代码：

```
wtCoeff=pywt.wavedec(x,'db8',level=4)
z1=pywt.waverec(wtCoeff,'db8')
```

反而是小波系数的显示要费点劲，为此还专门写了一个用于多通道信号显示的控件 DRMonitor。

离散小波变换的多尺度分解与重构实验样例代码

```
import numpy as np
import pywt
import tkinter as tk
from scipy import signal
import drvi.drviDSP as dsp
import drvi.drviControlls as dr

def setValueSingle1(v):
    global Fs,x,mName
    Fs=4000; N=768; dt=1.0/Fs;
    tt=np.arange(256)*dt*2*np.pi;
    x1=np.sin(200*tt)
    x2=np.sin(400*tt)
    x3=np.sin(800*tt)
    x=np.concatenate((x1,x2))
    x=np.concatenate((x,x3))
    setSWT(0)
    setDWT(0)

def setValueSingle2(v):
    global Fs,x,mName
    Fs=4000; N=768; dt=1.0/Fs;
    t=np.arange(N)*dt;
    x=np.sin(2*np.pi*(100+3000*t)*t)
    setSWT(0)
    setDWT(0)

def drawSWT():
    global Fs,t,x,y,z,mName
    mPlot.setValue2DXM(0,1/Fs,x,'#ffff00')
    mPlot.setValue2DXM(1,1/Fs,y[3][0],'#00ffff')
    mPlot.setValue2DXM(2,1/Fs,y[3][1],'#00ff00')
    mPlot.setValue2DXM(3,1/Fs,y[2][0],'#00ffff')
    mPlot.setValue2DXM(4,1/Fs,y[2][1],'#00ff00')
    mPlot.setValue2DXM(5,1/Fs,y[1][0],'#00ffff')
    mPlot.setValue2DXM(6,1/Fs,y[1][1],'#00ff00')
    mPlot.setValue2DXM(7,1/Fs,y[0][0],'#00ffff')
    mPlot.setValue2DXM(8,1/Fs,y[0][1],'#00ff00')
    mPlot.setValue2DXM(9,1/Fs,z,'#ff88ff')

def setSWT(v):
    global Fs,t,x,y,z,mName
```

```
    y=pywt.swt(x,mName,level=4)
    z=pywt.iswt(y,mName)
    drawSWT()
    mTab.setPage(0)


def setISWT(v):
    global Fs,t,x,y,z,mName
    N=len(x)
    if v==0: y[0][0][0:N]=0
    if v==1: y[0][1][0:N]=0
    if v==2: y[1][1][0:N]=0
    if v==3: y[2][1][0:N]=0
    if v==4: y[3][1][0:N]=0
    z=pywt.iswt(y,mName)
    drawSWT()

def drawDWT():
    global Fs,t,x,y1,z1,mName
    mPlot1.setValue2DXM(0,1/Fs,x,'#ffff00')
    mPlot1.setValue2DXM(1,1/Fs,y1[4],'#00ff00')
    mPlot1.setValue2DXM(2,1/Fs,y1[3],'#00ff00')
    mPlot1.setValue2DXM(3,1/Fs,y1[2],'#00ff00')
    mPlot1.setValue2DXM(4,1/Fs,y1[1],'#00ff00')
    mPlot1.setValue2DXM(5,1/Fs,y1[0],'#00ffff')
    mPlot1.setValue2DXM(6,1/Fs,z1,'#ff88ff')

def setDWT(v):
    global Fs,t,x,y1,z1,mName
    y1=pywt.wavedec(x,mName,level=4)
    z1=pywt.waverec(y1,mName)
    drawDWT()

def setIDWT(v):
    global Fs,t,x,y1,z1,mName
    if v==0:
        M=len(y1[0])
        y1[0][0:M]=0
    if v==1:
        M=len(y1[1])
        y1[1][0:M]=0
    if v==2:
        M=len(y1[2])
        y1[2][0:M]=0
    if v==3:
        M=len(y1[3])
        y1[3][0:M]=0
    if v==4:
        M=len(y1[4])
        y1[4][0:M]=0
```

```
    z1=pywt.waverec(y1,mName)
    drawDWT()

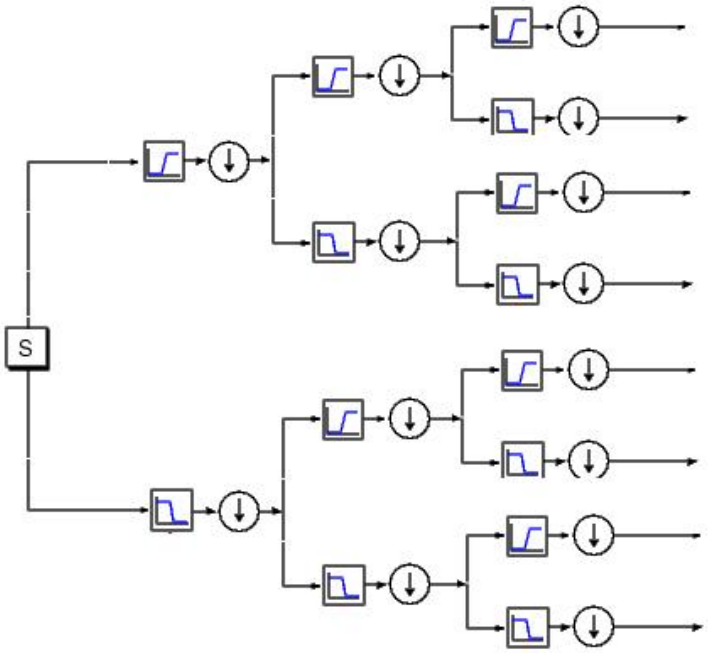

def setWTType(v):
    global mName
    if v==0: mName='db1'
    if v==1: mName='db2'
    if v==2: mName='db4'
    if v==3: mName='db8'
    if v==4: mName='db16'
    if v==5: mName='bior1.1'
    if v==6: mName='bior1.3'
    if v==7: mName='bior1.5'
    if v==8: mName='bior2.2'
    if v==9: mName='bior2.4'
    if v==10: mName='bior2.8'
    if v==11: mName='coif2'
    if v==12: mName='coif4'
    if v==13: mName='coif8'
    if v==14: mName='coif16'
    if v==15: mName='rbio2.2'
    if v==16: mName='rbio2.4'
    if v==17: mName='rbio2.8'
    if v==18: mName='sym2'
    if v==19: mName='sym4'
    if v==20: mName='sym8'
    if v==21: mName='sym16'
    if v==22: mName='sym20'
    w = pywt.Wavelet(mName)
    w1, h1 = signal.freqz(w.dec_lo)
    w2, h2 = signal.freqz(w.dec_hi)
    h1=abs(h1)
    h2=abs(h2)
    h3=h1*h2
    f=w1/(2*np.pi)
    mPlotW2.setValue2D(f,h1)
    mPlotW2.addValue2D(f,h2)
    mPlotW4.setValue2D(f,h3)
    x=np.zeros(512)
    x[256]=1
    coeffs = pywt.swt(x, mName, level=4)
    mPlotW1.setValue2DX(1,coeffs[0][0])
    mPlotW3.setValue2DX(1,coeffs[0][1])
    mTab.setPage(2)

#==Main Window=======================
win= tk.Tk()
win.geometry("1060x680")
win.config(bg="#ddeeee")
```

```
#==Layout of controls=====================
mL1=dr.DRLabel(win,20,20,500,40,'#440000','#ffffff','从 SWT 到 DWT 的变迁',20)
mBut1=dr.DRButton(win,680,25,100,36,'#003355','#ffffff','多频信号',12,101)
mBut2=dr.DRButton(win,780,25,100,36,'#003355','#ffffff','调频信号',12,101)
mBut3=dr.DRButton(win,940,25,100,36,'#003355','#ffffff','小波函数',12,101)
mTab=dr.DRNoteBook(win,20,75,910,590,txt='SWT 小波变换,DWT 小波变换,小波函数')
mPlot=dr.DRMonitor(mTab.win[0],10,10,888,500,wn=10,T=0.192)
mRBut1=dr.DRHButtonGroup(mTab.win[0],10,520,888,30,'#000000','#ffffff','删 A4,删 D4,删 D3,删 D2,删 D1',12,0)
mPlot1=dr.DRMonitor(mTab.win[1],10,10,888,500,wn=7,T=0.192)
mRBut11=dr.DRHButtonGroup(mTab.win[1],10,520,888,30,'#000000','#ffffff','删 A4,删 D4,删 D3,删 D2,删 D1',12,0)
mPlotW1=dr.DRPlot(mTab.win[2],10,15,445,265,'','#113344','#000000','#ffffff','#00eeff','#00ff00',-0,0,0,0,0,0)
mPlotW2=dr.DRPlot(mTab.win[2],455,15,445,265,'','#113344','#000000','#ffffff','#00eeff','#00ff00',0,0,0,0,0,0)
mPlotW3=dr.DRPlot(mTab.win[2],10,280,445,265,'','#113344','#000000','#ffffff','#00eeff','#00ff00',-0,0,0,0,0,0)
mPlotW4=dr.DRPlot(mTab.win[2],455,280,445,265,'','#113344','#000000','#ffffff','#00eeff','#00ff00',0,0,0,0,0,0)
sName='db1,db2,db4,db8,db16,'
sName=sName+'bior1.1,bior1.3,bior1.5,bior2.2,bior2.4,bior2.8,'
sName=sName+'coif1,coif4,coif8,coif16,'
sName=sName+'rbio2.2,rbio2.4,rbio2.8,sym2,sym4,sym8,sym16,sym20'
mLBut1=dr.DRListBox(win,940,75,100,590,'#ddffdd','#000000',sName,12,22)
mLBut1.addCallBackSingle(setWTType)
mBut1.addCallBackSingle(setValueSingle1)
mBut2.addCallBackSingle(setValueSingle2)
mRBut1.addCallBackSingle(setISWT)
mRBut11.addCallBackSingle(setIDWT)
#==================================
setWTType(22)
setValueSingle2(0)
#==Main Loop=====================
win.mainloop()
```

从离散小波变换到小波包变换的变迁实验

从连续小波变换 CWT 到尺度参数二进离散的非选抽型离散小波变换 SWT；再从 SWT 到对小波系数进行选抽的离散小波变换 DWT；最后，从 DWT 到对小波系数和平滑系数均进行选抽的小波包变换 Packet；可以说是一条充满了数学之美的演化之路。



实验设计

生成分段不同频率的多频信号和扫频信号，然后分别用小波包算法对信号进行分解与重构。下面是 3 层小波包变换的样例代码：

wtCoeff=pywt.WaveletPacket(data=x, wavelet='db8', maxlevel=3)

z=wtCoeff.reconstruct()

正反小波包都很简单，但分为若干个小波包的系数显示很费劲。

从离散小波变换到小波包变换的变迁实验样例代码

```
import numpy as np
import pywt
import tkinter as tk
from scipy import signal
import drvi.drviDSP as dsp
import drvi.drviControlls as dr

def setValueSingle1(v):
    global Fs,x,mName
    Fs=4000; N=768; dt=1.0/Fs;
    tt=np.arange(256)*dt*2*np.pi;
    x1=np.sin(200*tt)
    x2=np.sin(400*tt)
    x3=np.sin(800*tt)
    x=np.concatenate((x1,x2))
    x=np.concatenate((x,x3))
    setPacket(0)

def setValueSingle2(v):
```

```
    global Fs,t,x,mName
    Fs=4000; N=768; dt=1.0/Fs;
    t=np.arange(N)*dt;
    x=np.sin(2*np.pi*(100+3000*t)*t)
    setPacket(0)

def drawPacket():
    global Fs,t,x,y2,z2,mName
    mPlot2.clearFig()
    mPlot2.setValue2DXM(0,1/Fs,x,'#ffff00')
    x11=y2['a'].data; d11=len(x11)*(1/Fs)
    x12=y2['d'].data
    mPlot2.setValue2DXM(1,1/Fs,x11,'#1fffff',cl=1)
    mPlot2.setValue2DXM(1,1/Fs,x12,'#ff7f0e',d11,cl=1)
    x21=y2['aa'].data;d21=len(x21)*(1/Fs)
    x22=y2['ad'].data;d22=len(x22)*(1/Fs)
    x23=y2['da'].data;d23=len(x23)*(1/Fs)
    x24=y2['dd'].data
    mPlot2.setValue2DXM(2,1/Fs,x21,'#1fffff',cl=1)
    mPlot2.setValue2DXM(2,1/Fs,x22,'#ff7f0e',d21,cl=1)
    mPlot2.setValue2DXM(2,1/Fs,x23,'#eea0ee',d21+d22,cl=1)
    mPlot2.setValue2DXM(2,1/Fs,x24,'#ee2728',d21+d22+d23,cl=1)
    x31=y2['aaa'].data;d31=len(x31)*(1/Fs)
    x32=y2['aad'].data;d32=len(x32)*(1/Fs)
    x33=y2['ada'].data;d33=len(x33)*(1/Fs)
    x34=y2['add'].data;d34=len(x34)*(1/Fs)
    x35=y2['daa'].data;d35=len(x35)*(1/Fs)
    x36=y2['dad'].data;d36=len(x36)*(1/Fs)
    x37=y2['dda'].data;d37=len(x37)*(1/Fs)
    x38=y2['ddd'].data
    mPlot2.setValue2DXM(3,1/Fs,x31,'#1fffff',cl=1)
    mPlot2.setValue2DXM(3,1/Fs,x32,'#ff7f0e',d31,cl=1)
    mPlot2.setValue2DXM(3,1/Fs,x33,'#eea0ee',d31+d32,cl=1)
    mPlot2.setValue2DXM(3,1/Fs,x34,'#ee2728',d31+d32+d33,cl=1)
    mPlot2.setValue2DXM(3,1/Fs,x35,'#94eebd',d31+d32+d33+d34,cl=1)
    mPlot2.setValue2DXM(3,1/Fs,x36,'#eeaa99',d31+d32+d33+d34+d35,cl=1)
    mPlot2.setValue2DXM(3,1/Fs,x37,'#9e00f7',d31+d32+d33+d34+d35+d36,cl=1)
    mPlot2.setValue2DXM(3,1/Fs,x38,'#6d6dff',d31+d32+d33+d34+d35+d36+d37,cl=1)
    mPlot2.setValue2DXM(4,1/Fs,z2,'#ffff99')

def setPacket(v):
    global Fs,t,x,y2,z2,mName
    y2=pywt.WaveletPacket(data=x, wavelet=mName, maxlevel=3)
    z2=y2.reconstruct()
    drawPacket()

def setIPacket(v):
    global Fs,t,x,y2,z2,mName
    if v==0:
        M=len(y2['aaa'].data)
```

```
            y2['aaa'].data[0:M]=0
        if v==1:
            M=len(y2['aad'].data)
            y2['aad'].data[0:M]=0
        if v==2:
            M=len(y2['ada'].data)
            y2['ada'].data[0:M]=0
        if v==3:
            M=len(y2['add'].data)
            y2['add'].data[0:M]=0
        if v==4:
            M=len(y2['daa'].data)
            y2['daa'].data[0:M]=0
        if v==5:
            M=len(y2['dad'].data)
            y2['dad'].data[0:M]=0
        if v==6:
            M=len(y2['dda'].data)
            y2['dda'].data[0:M]=0
        if v==7:
            M=len(y2['ddd'].data)
            y2['ddd'].data[0:M]=0
        z2=y2.reconstruct()
        drawPacket()

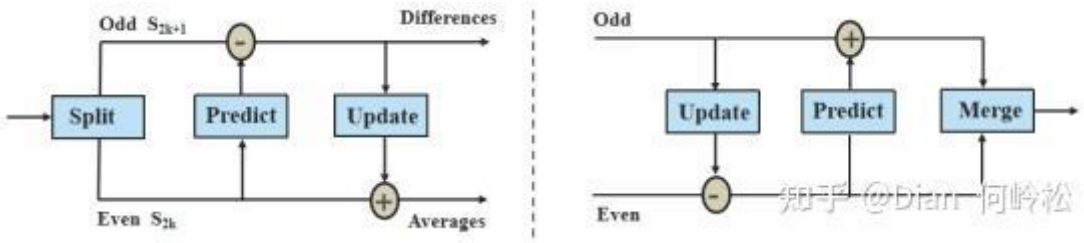
def setWTType1(v):
        global mName
        if v==0:
            mName='db2'
            mPlot2.setXWidth(0.197)
        if v==1:
            mName='db4'
            mPlot2.setXWidth(0.205)
        if v==2:
            mName='db8'
            mPlot2.setXWidth(0.22)
        if v==3:
            mName='db16'
            mPlot2.setXWidth(0.25)
        if v==4:
            mName='sym2'
            mPlot2.setXWidth(0.197)
        if v==5:
            mName='sym4'
            mPlot2.setXWidth(0.205)
        if v==6:
            mName='sym8'
            mPlot2.setXWidth(0.22)
        if v==7:
            mName='sym16'
```

```
            mPlot2.setXWidth(0.25)
        if v>7:
            mName='sym16'
            mPlot2.setXWidth(0.25)
        w = pywt.Wavelet(mName)
        w1, h1 = signal.freqz(w.dec_lo)
        w2, h2 = signal.freqz(w.dec_hi)
        h1=abs(h1)
        h2=abs(h2)
        h3=h1*h2
        f=w1/(2*np.pi)
        mPlotW2.setValue2D(f,h1)
        mPlotW2.addValue2D(f,h2)
        mPlotW4.setValue2D(f,h3)
        x=np.zeros(512)
        x[256]=1
        coeffs = pywt.swt(x, mName, level=4)
        mPlotW1.setValue2DX(1,coeffs[0][0])
        mPlotW3.setValue2DX(1,coeffs[0][1])
        if v<10: setPacket(0)

#==Main Window=======================
win= tk.Tk()
win.geometry("1060x680")
win.config(bg="#ddeeee")
#==Layout of controls=======================
mL1=dr.DRLabel(win,20,20,500,40,'#440000','#ffffff','从 SWT->DWT->WTPacket 的变迁',20)
mBut1=dr.DRButton(win,680,25,100,36,'#003355','#ffffff','多频信号',12,101)
mBut2=dr.DRButton(win,780,25,100,36,'#003355','#ffffff','调频信号',12,101)
mTab=dr.DRNoteBook(win,20,75,1020,590,txt='Packet 变换,小波函数')
mPlot2=dr.DRMonitor(mTab.win[0],10,10,998,500,wn=5,T=0.25)
mRBut21=dr.DRHButtonGroup(mTab.win[0],10,520,998,30,'#000000','#ffffff','aaa,aad,ada,add,daa,dad,dda,ddd',12,0)
mPlotW1=dr.DRPlot(mTab.win[1],10,15,500,265,'','#113344','#000000','#ffffff','#00eeff','#00ff00',-0,0,0,0,0,0)
mPlotW2=dr.DRPlot(mTab.win[1],510,15,500,265,'','#113344','#000000','#ffffff','#00eeff','#00ff00',0,0,0,0,0,0)
mPlotW3=dr.DRPlot(mTab.win[1],10,280,500,265,'','#113344','#000000','#ffffff','#00eeff','#00ff00',-0,0,0,0,0,0)
mPlotW4=dr.DRPlot(mTab.win[1],510,280,500,265,'','#113344','#000000','#ffffff','#00eeff','#00ff00',0,0,0,0,0,0)
sName='db2,db4,db8,db16,sym2,sym4,sym8,sym16'
mMenu=dr.DROptionMenu(win,920,25,120,36,'#003355','#ffff00',sName,12,7)
mMenu.addCallBackSingle(setWTType1)
mBut1.addCallBackSingle(setValueSingle1)
mBut2.addCallBackSingle(setValueSingle2)
mRBut21.addCallBackSingle(setIPacket)
#====================================
setWTType1(10)
setValueSingle2(0)
#==Main Loop=======================
win.mainloop()
```

从小波滤波器算法到提升小波算法实验

如果将 Malla 教授提出的离散小波变换滤波器算法称为第一代小波变换快速算法的话，那么由贝尔实验室 Sweldens W.博士于 1995 年提出的提升小波算法则可以被看作为第二代小波变换技术。提升小波变换主要由：分解、预测、更新，3 个计算步骤组成；通过一次提升小波正变换，可以将信号分解为低频和高频两个部分，从而构成对信号的多尺度分解或小波包分解。



与小波变换的滤波器组算法相比，发明者总结的主要优点如下：

1、继承了第一代小波的多分辨率的特性

2、不依赖傅立叶变换

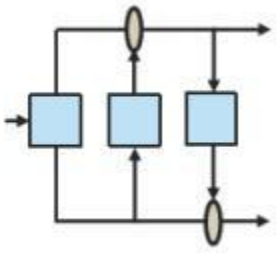
3、不占用系统内存，有利于 FPGA 等硬件实现

4、反变换很容易从正变换得到,只是改变了数据流的方向和正负号

5、可实现整数小波变换

实验设计

生成分段不同频率的多频信号和扫频信号，然后用提升小波算法对信号进行分解与重构。当前版本的 pywt 包中没有提升小波变换，只好自己写了 LTHaar、LT53、LT97 三种提升小波的正反变换算法。网上有公式，实现并不难。例如，下面是 haar 提升小波正变换的框图和公式：



$$d(2n) = x(2n) + \frac{1}{4}[c(2n-1) + c(2n+1) + 2]$$

$$c(2n+1) = x(2n+1) - \frac{1}{2}[x(2n) + x(2n+2)]$$

从小波滤波器算法到提升小波算法实验样例代码

```
import numpy as np
import tkinter as tk
from scipy import signal
import drvi.drviDSP as dsp
import drvi.drviControlls as dr
```

```
def setValueSingle1(v):
    global Fs,x,mName
    Fs=4000; N=768; dt=1.0/Fs;
    tt=np.arange(256)*dt*2*np.pi;
    x1=np.sin(200*tt)
    x2=np.sin(400*tt)
    x3=np.sin(800*tt)
    x=np.concatenate((x1,x2))
    x=np.concatenate((x,x3))
    setLPacket(0)
    setLWT(0)

def setValueSingle2(v):
    global Fs,t,x
    Fs=4000; N=768; dt=1.0/Fs;
    t=np.arange(N)*dt;
    x=np.sin(2*np.pi*(100+3000*t)*t)
    setLPacket(0)
    setLWT(0)

def setWTType(v):
    global wt;
    wt=v
    setLPacket(0)
    setLWT(0)

def setLPacket(v):
    global Fs,N,t,x,y,wt,ww
    ww=dsp.LWTPacket(wt,x,3)
    y=dsp.ILWTPacket(wt,ww,3)
    drawPacket()

def setILPacket(v):
    global Fs,N,t,x,y,wt,ww
    if v==0: ww[0]=ww[0]*0
    if v==1: ww[1]=ww[1]*0
    if v==2: ww[2]=ww[2]*0
    if v==3: ww[3]=ww[3]*0
    if v==4: ww[4]=ww[4]*0
    if v==5: ww[5]=ww[5]*0
    if v==6: ww[6]=ww[6]*0
    if v==7: ww[7]=ww[7]*0
    y=dsp.ILWTPacket(wt,ww,3)
    drawPacket()

def drawPacket():
    global Fs,N,t,x,y,wt,ww
    aaa=ww[0]; aad=ww[1]
    ada=ww[2]; add=ww[3]
    daa=ww[4]; dad=ww[5]
```

```
    dda=ww[6]; ddd=ww[7]
    mPlot2.clearFig()
    mPlot2.setValue2DXM(0,1/Fs,x,'#ffff00')
    d31=len(aaa)*(1/Fs)
    d32=len(aad)*(1/Fs)
    d33=len(ada)*(1/Fs)
    d34=len(add)*(1/Fs)
    d35=len(daa)*(1/Fs)
    d36=len(dad)*(1/Fs)
    d37=len(dda)*(1/Fs)
    mPlot2.setValue2DXM(1,1/Fs,aaa,'#1fffff',cl=1)
    mPlot2.setValue2DXM(1,1/Fs,aad,'#ff7f0e',d31,cl=1)
    mPlot2.setValue2DXM(1,1/Fs,ada,'#eea0ee',d31+d32,cl=1)
    mPlot2.setValue2DXM(1,1/Fs,add,'#ee2728',d31+d32+d33,cl=1)
    mPlot2.setValue2DXM(1,1/Fs,daa,'#94eebd',d31+d32+d33+d34,cl=1)
    mPlot2.setValue2DXM(1,1/Fs,dad,'#eeaa99',d31+d32+d33+d34+d35,cl=1)
    mPlot2.setValue2DXM(1,1/Fs,dda,'#9e00f7',d31+d32+d33+d34+d35+d36,cl=1)
    mPlot2.setValue2DXM(1,1/Fs,ddd,'#6d6dff',d31+d32+d33+d34+d35+d36+d37,cl=1)
    mPlot2.setValue2DXM(2,1/Fs,y,'#ffff99')

def setLWT(v):
    global Fs,N,t,x,ym,wt,wm
    wm=dsp.decLWT(wt,x,4)
    ym=dsp.recLWT(wt,wm,4)
    drawLWT()

def setILWT(v):
    global Fs,N,t,x,y,ym,wt,wm
    if v==0: wm[0]=wm[0]*0
    if v==1: wm[1]=wm[1]*0
    if v==2: wm[2]=wm[2]*0
    if v==3: wm[3]=wm[3]*0
    if v==4: wm[4]=wm[4]*0
    ym=dsp.recLWT(wt,wm,4)
    drawLWT()

def drawLWT():
    global Fs,N,t,x,ym,wt,wm
    mPlot.setValue2DXM(0,1/Fs,x,'#ffff00')
    mPlot.setValue2DXM(5,1/Fs,wm[0],'#99ff00')
    mPlot.setValue2DXM(4,1/Fs,wm[1],'#99ff00')
    mPlot.setValue2DXM(3,1/Fs,wm[2],'#99ff00')
    mPlot.setValue2DXM(2,1/Fs,wm[3],'#99ff00')
    mPlot.setValue2DXM(1,1/Fs,wm[4],'#99ff00')
    mPlot.setValue2DXM(6,1/Fs,ym,'#ffff99')

#==Main Window=====================
win= tk.Tk()
win.geometry("1060x680")
win.config(bg="#ddeeee")
```

```
#==Layout of controls=====================
mL1=dr.DRLabel(win,20,20,500,40,'#440000','#ffffff','了解提升小波变换',20)
mBut1=dr.DRButton(win,680,25,100,36,'#003355','#ffffff','多频信号',12,101)
mBut2=dr.DRButton(win,780,25,100,36,'#003355','#ffffff','调频信号',12,101)
mTab=dr.DRNoteBook(win,20,75,1020,590,txt='多尺度分解,小波包变换')
mPlot=dr.DRMonitor(mTab.win[0],10,10,998,500,wn=7,T=0.192)
mRBut1=dr.DRHButtonGroup(mTab.win[0],10,520,998,30,'#000000','#ffffff','删 A4,删 D4,删 D3,删 D2,删 D1',12,0)
mPlot2=dr.DRMonitor(mTab.win[1],10,10,998,500,wn=3,T=0.192)
mRBut21=dr.DRHButtonGroup(mTab.win[1],10,520,998,30,'#000000','#ffffff','aaa,aad,ada,add,daa,dad,dda,ddd',12,0)
sName='db2,db4,db8,db16,sym2,sym4,sym8,sym16'
mMenu=dr.DROptionMenu(win,900,25,140,36,'#003355','#ffff00','Haar 提升小波,53 提升小波,97 提升小波',12,2)
mMenu.addCallBackSingle(setWTType)
mBut1.addCallBackSingle(setValueSingle1)
mBut2.addCallBackSingle(setValueSingle2)
mRBut1.addCallBackSingle(setILWT)
mRBut21.addCallBackSingle(setILPacket)
#==================================
wt=0
setValueSingle2(0)
#==================================
win.mainloop()
```

希尔伯特变换和解析信号实验

在数学与信号处理领域，一个实值函数的希尔伯特变换(Hilbert transform)是将信号 x(t)与 1/(πt)做卷积分，得到信号 y(t)。信号经希尔伯特变换后，各频率分量的幅度保持不变，但相位将出现 90°相移，因此希尔伯特变换器又称为 90°相移滤波器。例如，sin(t)经希尔伯特变换后变为 cos(t)

$$Y(t)=x(t)*(1/\pi t)$$

而所谓的解析信号(Analytic signal)是频谱没有负频率分量的复信号；它由原信号和它的希尔伯特变换组成。

$$z(t)=x(t)+jHT[x(t)]=x(t)+jy(t)$$

解析信号有一个特点，就是可以求出其瞬时幅值 A(t)、瞬时频率 f(t)和瞬时相位 P(t)，也就是得到它们随时间 t 的变化曲线。信号的瞬时幅值计算公式为：

$$A(t)=abs(z(t))$$

瞬时相位计算公式为：

$$P(t)=arctan(y(t)/x(t))$$

瞬时角频率计算公式为：

$$W(t)=dp(t)/dt$$

也就是说，我们可以求出任一时刻信号的幅值、频率和相位值。取时间、频率和幅值数据{t,f,A}作图，就可以信号的时频分布。但是由于每一个时刻只能求出一个幅值、频率、相位值，因此 x(t)在任一一瞬时只能是单频信号，否则瞬时幅值、频率、相位值数据无意义。

实验设计

生成满足瞬时单频条件的分段不同频率的多频信号、扫频信号、调频调幅波信号，以及不满足条件的双频信号；进行希尔伯特变换，然后计算信号的瞬时幅值、频率，绘制信号的时频分布图；体会希尔伯特变换的特点。

希尔伯特变换和解析信号实验样例代码

```
import numpy as np
import tkinter as tk
from scipy.signal import hilbert
import drvi.drviDSP as dsp
import drvi.drviControlls as dr

def setSignal(v):
    global N,Fs,t,x
    if v==0:
        Fs=4000; N=768; dt=1.0/Fs;
        t=np.arange(256)*dt;
```

```
            x1=np.sin(2*np.pi*200*t)
            x2=np.sin(2*np.pi*400*t)
            x3=np.sin(2*np.pi*800*t)
            x=np.concatenate((x1,x2))
            x=np.concatenate((x,x3))
            t=np.arange(N)*dt;
        if v==1:
            Fs=4000; N=768; dt=1.0/Fs;
            t=np.arange(N)*dt;
            x=np.sin(2*np.pi*(100+3000*t)*t)
        if v==2:
            Fs=4000; N=768; dt=1.0/Fs;
            t=np.arange(N)*dt;
            x1=np.sin(2*np.pi*(100+3000*t)*t)
            x2=0.4*np.sin(2*np.pi*20*t)+0.6
            x=x1*x2
        if v==3:
            Fs=4000; N=768; dt=1.0/Fs;
            t=np.arange(N)*dt;
            x1=np.sin(2*np.pi*50*t)
            x2=np.sin(2*np.pi*1000*t)*0.3
            x=0.8*(x1+x2)
        drawHilbert()

def drawHilbert():
        global Fs,N,t,x
        z=hilbert(x)
        mPlot.setValue2DXM(0,1/Fs,z.real,'#ffff00')
        mPlot.setValue2DXM(1,1/Fs,z.imag,'#00ffff')
        Ainst = np.abs(z)
        Pinst = np.unwrap(np.angle(z))
        Finst =(np.diff(Pinst)/(2.0*np.pi)*Fs)
        #去除野点
        Finst=np.clip(Finst,0,np.max(Finst))
        M=len(Finst)
        mPlot1.setValue2DX(1/Fs,x,c='#00ffff')
        mPlot1.addValue2DX(1/Fs,Ainst,'#ff0000')
        mPlot2.setValue2DX(1/Fs,Finst)
        mP3D.setValue3D(t[0:M],Finst*0+2000,x[0:M],'#007799')
        mP3D.addValue3D(t[0:M],Finst*0+2000,Ainst[0:M],'#990099',s=0)
        ff,AA=dsp.AmplitudeSpetrum(N,Fs,x)
        mP3D.addValue3D(ff*0,ff,AA/np.max(AA),'#009977',s=0)
        mP3D.addValue3D(t[0:M],Finst,Ainst[0:M]*0-1.25,'#000000',s=0)
        mP3D.addValue3D(t[0:M],Finst,Ainst[0:M],'#f00000',s=0)

#==Main Window=======================
win= tk.Tk()
win.geometry("1060x680")
win.config(bg="#ddeeee")
#==Layout of controls=======================
```

```
mL1=dr.DRLabel(win,20,20,500,40,'#440000','#ffffff','希尔伯特变换的作用',20)
mHBut=dr.DRHButtonGroup(win,560,25,480,36,'#003355','#ffffff','多频信号,调频信号,调频调幅信号,瞬时双频信号',12,0)
mTab=dr.DRNoteBook(win,20,75,1020,590,txt='解析信号,瞬时幅值和瞬时频率曲线,时频分布曲线')
mPlot=dr.DRMonitor(mTab.win[0],10,10,998,540,wn=2,T=0.192)
mPlot1=dr.DRPlot(mTab.win[1],10,10,998,270,'','#003355','#000000','#ffffff','#00eeff','#ffff00',0,0,-1.2,1.2,0,0)
mPlot2=dr.DRPlot(mTab.win[1],10,280,998,270,'','#003355','#000000','#ffffff','#00eeff','#ffff00',0,0,0,2000,0,0)
mP3D=dr.DRPlot3D(mTab.win[2],10,10,998,540,xlow=0,xhigh=0.195,ylow=0,yhigh=2000,cb1='#cccccc',cf1='#000000')
#================================
mHBut.addCallBackSingle(setSignal)
setSignal(0)
#==Main Loop=====================
win.mainloop()
```

希尔伯特-黄变换实验

希尔伯特变换可以求出信号的瞬时频率、幅值和相位；但在任一时刻 t，只能求解出一个瞬时频率值，即希尔伯特变换只能处理任何时刻为单一频率的信号，应用颇为受限。为此，美籍华人黄锷教授提出了本征模态函数(Intrinsic Mode Function)概念，认为任何信号都可以分解为一组单一瞬时频率的本征模态函数之合，并设计出求解 IMF 的经验模态分解方法 EMD，然后在其基础上提出了希尔伯特-黄变换。希尔伯特-黄变换实际上由经验模态分解和希尔伯特变换两个部分构成。先用 EMD 将信号分解成若干个只有一个瞬时频率分量的本征模态函数 IMFs；然后对各 IMFs 分量进行希尔伯特变换，计算得到它们的瞬时频率和瞬时幅值，组合在一起显示就得到时变信号的时频分析图。

实验设计

Python 中有开源的 EMD 资源包，实践一下算法并不难。用如下命令安装：

```
pip install EMD-signal
```

然后就可以进行 EMD 分析。实验时生成不满足瞬时单频条件的双频信号、单频+扫频信号、漂移+扫频信号，和双扫频信号；然后进行 EMD 分解：

```
IMFs = EMD().emd(x)
```

最后对各 IMF 信号进行希尔伯特变换，再求它们的瞬时幅值、频率；绘制信号的时频分布图。

希尔伯特-黄变换实验样例代码

```
import numpy as np
import tkinter as tk
from scipy import signal
from scipy.signal import hilbert
from PyEMD import EMD
import drvi.drviDSP as dsp
import drvi.drviControlls as dr

def setSignal(v):
    global N,Fs,t,x
    if v==0:
        Fs=4000; N=768; dt=1.0/Fs;
        t=np.arange(N)*dt;
        x1=np.sin(2*np.pi*100*t)
        x2=np.sin(2*np.pi*600*t)
        x=0.6*x1+0.35*x2
    if v==1:
        Fs=4000; N=768; dt=1.0/Fs;
        t=np.arange(N)*dt;
        x1=np.cos(2*np.pi*1000*t)
        x2=np.cos(2*np.pi*(20+t*1200)*t)
        x=0.35*x1+0.6*x2
    if v==2:
        Fs=4000; N=768; dt=1.0/Fs;
        t=np.arange(N)*dt;
        x1=2*np.cos(2*np.pi*(20+t*1200)*t)
        x2=300*t*t-4.5
        x=0.2*(x1+x2)
```

```
    if v==3:
        Fs=4000; N=768; dt=1.0/Fs;
        t=np.arange(N)*dt;
        f1=50+5000*t*t
        y1=0.5*np.sin(2*np.pi*f1*t)
        f2=250+15000*t*t
        y2=np.sin(2*np.pi*f2*t)
        x=0.6*(y1+y2)
    drawIMF()
    drawEMD()
    drawHilbert()

def drawIMF():
    global Fs,N,t,x
    mPlot1.setValue2DX(1/Fs,x,c='#ffff00')
    aveL,upL,downL,upP,downP=dsp.findIMF(x)
    mPlot2.setValue2DX(1/Fs,upL,c='#00ffff',lw=1)
    mPlot2.addValue2DX(1/Fs,downL,c='#00ff88',lw=1)
    mPlot2.addValue2DX(1/Fs,aveL,c='#ff00ff',lw=1)

def drawEMD():
    global Fs,N,t,x,M,IMFs,IMF1s,res
    emd=EMD()
    emd.emd(x)
    IMFs,res=emd.get_imfs_and_residue()
    IMF1s=IMFs
    M=IMFs.shape[0]
    mPlot.clearFig(wn=M+3)
    mPlot.setValue2DXM(0,1/Fs,x,'#ffff00',ylow=-1.5,yhigh=1.5)
    for i in range(M):
        mPlot.setValue2DXM(i+1,1/Fs,IMFs[i],'#00ff00',ylow=-1.5,yhigh=1.5)
    mPlot.setValue2DXM(M+1,1/Fs,res,'#00ffff',ylow=-1.5,yhigh=1.5)
    mPlot.setValue2DXM(M+2,1/Fs,x,'#ffeeaa',ylow=-1.5,yhigh=1.5)

def setIEMD(v):
    global Fs,N,t,x,M,IMF1s,res
    M=IMF1s.shape[0]
    mPlot.clearFig(wn=M+3)
    mPlot.setValue2DXM(0,1/Fs,x,'#ffff00',ylow=-1.5,yhigh=1.5)
    if (v==0): res=res*0;
    else:
        if v<=M: IMF1s[v-1]=IMF1s[v-1]*0
    x1=res
    for i in range(M): x1=x1+IMF1s[i]
    for i in range(M):
        mPlot.setValue2DXM(i+1,1/Fs,IMF1s[i],'#00ff00',ylow=-1.5,yhigh=1.5)
    mPlot.setValue2DXM(M+1,1/Fs,res,'#00ffff',ylow=-1.5,yhigh=1.5)
    mPlot.setValue2DXM(M+2,1/Fs,x1,'#ffeeaa',ylow=-1.5,yhigh=1.5)

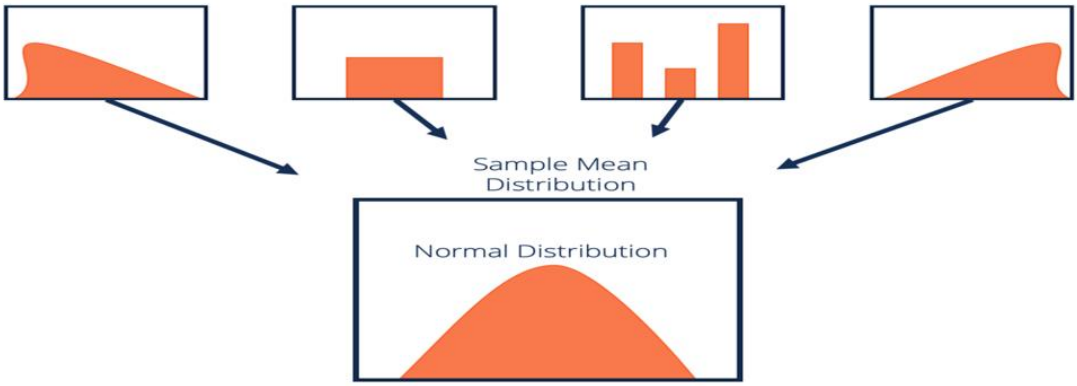
def drawHilbert():
```

```
    global Fs,N,t,x,M,IMFs
    M=IMFs.shape[0]
    for i in range(M):
        z=hilbert(IMFs[i])
        Ainst = np.abs(z)
        Pinst = np.unwrap(np.angle(z))
        Finst =(np.diff(Pinst)/(2.0*np.pi)*Fs)
        #去除野点
        Finst=np.clip(Finst,0,np.max(Finst))
        K=len(Finst)
        if i==0: mP3D.setValue3D(t[0:K],Finst,Ainst[0:K],'#0000f0',s=0)
        if i==1: mP3D.addValue3D(t[0:K],Finst,Ainst[0:K],'#f00000',s=0)
        if i==2: mP3D.addValue3D(t[0:K],Finst,Ainst[0:K],'#f000f0',s=0)
        if i==3: mP3D.addValue3D(t[0:K],Finst,Ainst[0:K],'#00f0f0',s=0)
        if i>=4:    mP3D.addValue3D(t[0:K],Finst,Ainst[0:K],'#f0f000',s=0)
        K=len(Finst)
        if i==0:
            mScat.setValue3DC(t[0:K],Finst[0:K],Ainst[0:K])
        else:
            mScat.addValue3DC(t[0:K],Finst[0:K],Ainst[0:K])

#==Main Window=====================
win= tk.Tk()
win.geometry("1060x680")
win.config(bg="#ddeeee")
#==Layout of controls=====================
mL1=dr.DRLabel(win,20,20,500,40,'#440000','#ffffff','信号的 EMD 分解与重构',20)
mHBut=dr.DRHButtonGroup(win,560,25,480,36,'#003355','#ffffff','双频信号,单频+扫频信号,漂移+扫频信号,双扫频信号',12,0)
mTab=dr.DRNoteBook(win,20,75,1020,590,txt='IMF 求解示意,EMD 分解与重构,时频分布表达,HHT 表达')
mPlot=dr.DRMonitor(mTab.win[1],10,10,998,500,wn=9,T=0.192)
mRBut=dr.DRHButtonGroup(mTab.win[1],10,520,998,30,'#000000','#ffffff','残差,IMF1,IMF2,IMF3,IMF4,IMF5,IMF6',12,0)
mPlot1=dr.DRPlot(mTab.win[0],10,10,998,270,'','#003355','#000000','#ffffff','#00eeff','#ffff00',0,0,-1.5,1.5,0,0)
mPlot2=dr.DRPlot(mTab.win[0],10,280,998,270,'','#003355','#000000','#ffffff','#00eeff','#ffff00',0,0,-1.5,1.5,0,0)
mP3D=dr.DRPlot3D(mTab.win[2],10,10,998,540,xlow=0,xhigh=0.195,ylow=0,yhigh=1200)
mScat=dr.DRScatter(mTab.win[3],10,10,998,540,zlow=0,zhigh=0.5)
#==============================
mHBut.addCallBackSingle(setSignal)
mRBut.addCallBackSingle(setIEMD)
setSignal(0)
#==Main Loop=====================
win.mainloop()
```

中心极限定理实验

无论是什么分布的随机变量，当随机变量足够多时，相互统计独立的随机变量的均值的分布将趋向于高斯分布。



实验设计：

用均匀分布随机数函数产生 1000 个随机数，显示其波形，并求概率密度曲线；然后做 1000 次重复实验，每次生成 1000 个随机数，求均值；最后，显示 1000 次实验均值构成的曲线和直方图曲线。检验中心极限定理。

中心极限定理实验样例代码

```
import tkinter as tk
import numpy as np
import drvi.drviControlls as dr

def getPdf(x):
    a1=np.min(x)
    a2=np.max(x)
    M=128
    A=np.linspace(a1,a2,num=M)
    [mHist,b]=np.histogram(x,bins=A)
    return A[0:M-1],mHist

def myWorking(v):
    N=1000
    y=np.random.rand(N)
    t=np.arange(N)
    mSubPlot.setValue2DX(1,t,y)
    mA,mHist=getPdf(y)
    mSubPlot.setValue2DX(3,mA,mHist)
    K=1000
    z=np.zeros(K)
    for i in range(K):
        yy=np.random.rand(N)
        z[i]=np.mean(yy)
    mSubPlot.setValue2DX(2,t,z)
```

```
    mA,mHist=getPdf(z)
    mSubPlot.setValue2DX(4,mA,mHist)

#==Main Window=======================
win= tk.Tk()
win.geometry("1100x670")
win.config(bg="#ddeeee")
#==Layout of controls=================
mL1=dr.DRLabel(win,20,25,500,40,'#440000','#ffffff','中心极限定理实验',20)
mBut=dr.DRButton(win,980,29,100,40,txt='计算')
mSubPlot=dr.DRSubPlot(win,20,80,1060,580,N=2,M=2)
#==DataStream Binding=================
mBut.addCallBackSingle(myWorking)
#==Main Loop=========================
win.mainloop()
```

了解 FastICA 盲源分离算法实验

信号的盲源分离(Blind Source Separation)是指从多路混合信号(观测信号)中分离出多路源信号的问题，因源信号无法直接观测(盲源)；故该信号分析方法又被称之为盲源分离。

ICA(Independent Component Analysis)是一种利用统计原理来进行盲源问题求解的数学方法，它假设各信号成分是统计独立的，然后按照中心极限定理，通过线性变换把混合后的盲源信号分离成统计独立的非高斯的信号源的线性组合。

Fast-ICA 算法是芬兰赫尔辛基工业大学计算机及信息科学实验室 Hyvarien 等人提出的一种 ICA 算法，它利用最大熵原理来近似负熵，并通过一个合适的非线性函数 g 使负熵达到最优。

实验设计：

生成 1 路正弦波信号和一路锯齿波信号，然后线性混合成 2 路混合信号；组后再用 FastICA 算法从混合信号中分离出 2 路源信号的最佳估计值。FastICA 算法直接引用（https://hbenbel.github.io/blog/fastica/）的简易算法。

了解 FastICA 盲源分离算法实验样例代码

```
import numpy as np
import tkinter as tk
from scipy import signal
import drvi.drviControlls as dr

#==FastICA====================
# Definition of G'
def Gprime(x):
    return np.tanh(x)

# Definition of G"
def Gsecond(x):
    return np.ones(x.shape)-np.power(np.tanh(x),2)

# Center matrix X
def centerMatrix(X, N):
    mean = X.mean(axis=1)
    M = X - (mean.reshape((N, 1)) @ np.ones((1, X.shape[1])))
    return M

# Whiten matrix X with eigenvalue decomposition
def whitenMatrix(X):
    D, E = np.linalg.eigh(X @ X.T)
    DE = np.diag(1/np.sqrt(D + 1e-5)) @ E.T
    return DE @ X

# One-unit algorithm step
def oneUnit(X, wp):
    term1 = np.mean((X @ Gprime(wp @ X).T), axis=1)
    term2 = np.mean(Gsecond(wp @ X), axis=1) * wp
    return term1 - term2
```

```
# Deflationary orthogonalization
def orthogonalize(W, wp, i):
    return wp - ((wp @ W[:i,:].T) @ W[:i,:])

# wp normalization
def normalize(wp):
    return wp / np.linalg.norm(wp)

def fastICA(X, C, max_iter):
    N = X.shape[0]
    X = centerMatrix(X, N)
    X = whitenMatrix(X)
    W = np.zeros((C, N))
    for i in range(C):
        wp = np.random.rand(1, N)
        for _ in range(max_iter):
            wp = oneUnit(X, wp)
            wp = orthogonalize(W, wp, i)
            wp = normalize(wp)
            W[i,:] = wp
    return W @ X
#=========================================

def setSignal(v):
    global N,Fs,t,s1,s2,x1,x2
    N=2000
    t=np.linspace(0,8,N)
    s1=np.sin(5*t)
    s2=signal.sawtooth(2*np.pi*t)
    x1=0.3*s1+0.5*s2
    x2=0.8*s1-0.5*s2
    resetWave(0)

def resetWave(v):
    global N,Fs,t,s1,s2,x1,x2
    mPlot.setValue2DM(0,t,s1,'#ffff00',ylow=-1.5,yhigh=1.5)
    mPlot.setValue2DM(1,t,s2,'#00ffff',ylow=-1.5,yhigh=1.5)
    mPlot.setValue2DM(2,t,x1,'#ffff00',ylow=-1.5,yhigh=1.5)
    mPlot.setValue2DM(3,t,x2,'#00ffff',ylow=-1.5,yhigh=1.5)
    mPlot.setValue2DM(4,t,x1*0,'#ffff00',ylow=-1.5,yhigh=1.5)
    mPlot.setValue2DM(5,t,x2*0,'#00ffff',ylow=-1.5,yhigh=1.5)

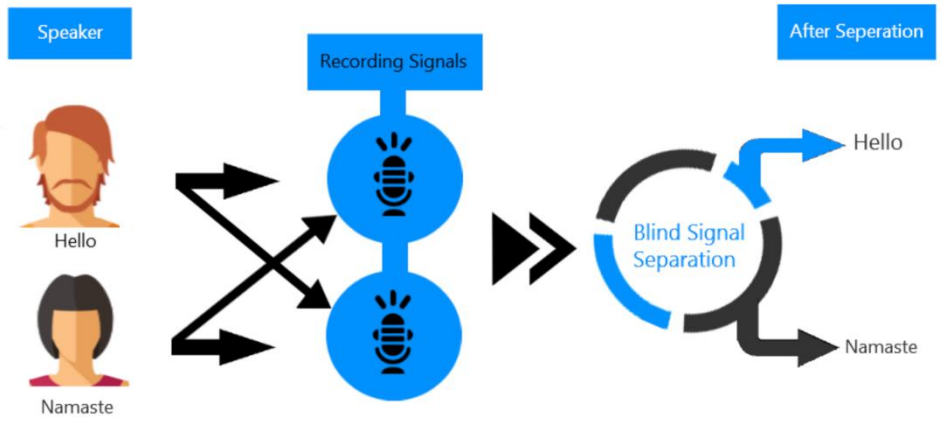
def doCalc(v):
    global N,Fs,t,x1,x2
    X=np.c_[x1[0:N],x2[0:N]]
    X=np.rot90(X,-1)
    S = fastICA(X,2,32)
    y1=S[0,:]/np.max(S[0,:])
    y2=S[1,:]/np.max(S[1,:])
    mPlot.setValue2DM(4,t,y1,'#ffff00',ylow=-1.5,yhigh=1.5)
```

```
        mPlot.setValue2DM(5,t,y2,'#00ffff',ylow=-1.5,yhigh=1.5)


#==Main Window=====================
win= tk.Tk()
win.geometry("1060x680" )
win.config(bg="#ddeeee")
#==Layout of controls=====================
mL1=dr.DRLabel(win,20,20,400,40,'#440000','#ffffff','读懂 FastICA 盲源分离算法',20)
mBut1=dr.DRButton(win,910,25,60,36,txt='计算')
mBut2=dr.DRButton(win,980,25,60,36,txt='重置')
mPlot=dr.DRMonitor(win,20,70,1020,580,wn=6)
#==================================
mBut1.addCallBackSingle(doCalc)
mBut2.addCallBackSingle(resetWave)
setSignal(0)
#==Main Loop=====================
win.mainloop()
```

音频信号的盲源分离实验

在鸡尾酒会等很多场合，人们会同时听到多个声音信号，它们混叠在一起。如果发出声音信号的信号源数量小于等于接收声音信号的观测点数量，则可以用盲源分离的放法将混合在一起的声音信号分离出来。



实验设计：

从 DRVI 数据集中读入 3 个音频文件，然后线性混合成 3 路混合的音频信号；最后再用 FastICA 算法从混合后的音频信号中分离出 3 路源信号的最佳估计值。FastICA 算法采用 Scikit-Learn 机器学习工具库中的 API，因此你需要安装该库（pip install scikit-learn）。

音频信号的盲源分离实验样例代码

```
import numpy as np
from scipy import signal
import tkinter as tk
from tkinter import filedialog
import librosa
from sklearn.decomposition import FastICA
import drvi.drviControlls as dr
import drvi.drviSimpleAudio as sa
import drvi.drviAppendPath as mPath

def getSim(v):
    global Fs,s1,s2,s3;
    Fs=44100; N=Fs*10; dt=1/Fs
    t=np.arange(N)*dt
    s1=0.5*np.sin(2*np.pi*10*t)*(1.2+np.sin(2*np.pi*1*t))
    s2=np.sin(2*np.pi*(1+1*t)*t)
    s3=np.sign(np.sin(2*np.pi*1*t))
    cutWAV()
    getMix(0)
    solveMix()

def getFile(v):
    global Fs,s1,s2,s3;
    if v==3:
        mName1=mPath.getDRVIPath()+'\\dataset\\voice1.mp3'
```

```
            mName2=mPath.getDRVIPath()+'\\dataset\\voice2.mp3'
            mName3=mPath.getDRVIPath()+'\\dataset\\voice3.mp3'
            getWAV(0,mName1);
            getWAV(1,mName2)
            getWAV(2,mName3)
        else:
            mfile=filedialog.askopenfilename(title="选择文件",filetypes=(("MP3 files", "*.mp3"),("WAV files", "*.wav"),("all files", "*.*")))
            if not mfile: return 1
            else: getWAV(v,mfile)
        cutWAV()
        getMix(0)
        solveMix()

def getWAV(v,name):
        global Fs,s1,s2,s3;
        y,Fs=librosa.load(name,mono=True)
        if v==0: s1=y
        if v==1: s2=y
        if v==2: s3=y

def cutWAV():
        global Fs,s1,s2,s3,S;
        N=np.min([len(s1),len(s2),len(s3)])
        S=np.c_[s1[0:N],s2[0:N],s3[0:N]]
        mPlot.setValue2DXM(0,1/Fs,S[:,0],'#ffff00',ylow=-1.25,yhigh=1.25)
        mPlot.setValue2DXM(1,1/Fs,S[:,1],'#00ffff',ylow=-1.25,yhigh=1.25)
        mPlot.setValue2DXM(2,1/Fs,S[:,2],'#00ff00',ylow=-1.25,yhigh=1.25)

def doPlay(v):
        global Fs,s1,s2,s3,S,X,SE;
        if v==0: y=S[:,0]/np.max(S[:,0])
        if v==1: y=S[:,1]/np.max(S[:,1])
        if v==2: y=S[:,2]/np.max(S[:,2])
        if v==3: y=X[:,0]/np.max(X[:,0])
        if v==4: y=X[:,1]/np.max(X[:,1])
        if v==5: y=X[:,2]/np.max(X[:,2])
        if v==6: y=SE[:,0]/np.max(SE[:,0])
        if v==7: y=SE[:,1]/np.max(SE[:,1])
        if v==8: y=SE[:,2]/np.max(SE[:,2])
        sa.play(1,Fs,y,0)

def getMix(v):
        global Fs,A,S,X
        A=np.array([[1,1,1],[1,1,1],[1,1,1]])
        txt1=mEntry1.getValueString()
        txt2=mEntry2.getValueString()
        txt3=mEntry3.getValueString()
        LL1=txt1.split(',');    LL2=txt2.split(',');    LL3=txt3.split(',')
        A[0,0]=float(LL1[0]); A[0,1]=float(LL1[1]); A[0,2]=float(LL1[2])
        A[1,0]=float(LL2[0]); A[1,1]=float(LL2[1]); A[1,2]=float(LL2[2])
```

```
    A[2,0]=float(LL3[0]); A[2,1]=float(LL3[1]); A[2,2]=float(LL3[2])
    X=np.dot(S,A.T)
    X[:,0]=X[:,0]/np.max(X[:,0])
    X[:,1]=X[:,1]/np.max(X[:,1])
    X[:,2]=X[:,2]/np.max(X[:,2])
    mPlot1.setValue2DXM(0,1/Fs,X[:,0],'#ffff00',ylow=-1.25,yhigh=1.25)
    mPlot1.setValue2DXM(1,1/Fs,X[:,1],'#00ffff',ylow=-1.25,yhigh=1.25)
    mPlot1.setValue2DXM(2,1/Fs,X[:,2],'#00ff00',ylow=-1.25,yhigh=1.25)
    print(1)

def solveMix():
    global Fs,A,S,X,SE
    ica = FastICA(n_components=3)
    SE=ica.fit_transform(X)
    SE[:,0]=SE[:,0]/np.max(SE[:,0])
    SE[:,1]=SE[:,1]/np.max(SE[:,1])
    SE[:,2]=SE[:,2]/np.max(SE[:,2])
    mPlot2.setValue2DXM(0,1/Fs,SE[:,0],'#ffff00',ylow=-1.25,yhigh=1.25)
    mPlot2.setValue2DXM(1,1/Fs,SE[:,1],'#00ffff',ylow=-1.25,yhigh=1.25)
    mPlot2.setValue2DXM(2,1/Fs,SE[:,2],'#00ff00',ylow=-1.25,yhigh=1.25)

#==Main Window=====================
win= tk.Tk()
win.geometry("1060x680")
win.config(bg="#ddeeee")
#==Layout of controls=====================
mL1=dr.DRLabel(win,20,20,400,40,'#440000','#ffffff','音频信号的盲源分离',20)
mBut=dr.DRHButtonGroup(win,440,25,330,36,txt='音频 1,音频 2,音频 3,默认音频')
mButMenu=dr.DROptionMenu(win,900,25,140,36,txt='播放源音 1,播放源音 2,播放源音 3,播放混音 1,播放混音 2,播放混音 3,播放分离音 1,播放分离音 2,播放分离音 3')
mTab=dr.DRNoteBook(win,20,75,1020,590,txt='源音频信号,混合的音频信号,盲源分离的音频信号')
mPlot=dr.DRMonitor(mTab.win[0],10,10,998,540,wn=3)
mPlot1=dr.DRMonitor(mTab.win[1],10,10,998,500,wn=3)
mBut1=dr.DRButton(mTab.win[1],10,520,120,36,txt='重新混音')
mEntry1=dr.DREntry(mTab.win[1],140,520,140,36,txt='1.0,1.0,1.0')
mEntry2=dr.DREntry(mTab.win[1],290,520,140,36,txt='0.5,2.0,1.0')
mEntry3=dr.DREntry(mTab.win[1],450,520,140,36,txt='1.5,1.0,0.5')
mPlot2=dr.DRMonitor(mTab.win[2],10,10,998,540,wn=3)
#==================================
mButMenu.addCallBackSingle(doPlay)
mBut.addCallBackSingle(getFile)
mBut1.addCallBackSingle(getMix)
#==Main Loop=====================
win.mainloop()
```

数据的 PCA 降维和可视化处理实验

主成分分析(Principal Component Analysis,PCA)与独立成分 ICA 分析相似，也是一种多变量统计方法，它通过正交变换将一组可能存在相关性的变量数据转换为一组线性不相关的变量，转换后的变量被称为主成分。主成分分解后，将影响不大的主成分分量去掉，就可实现数据降维。



实验设计：

从 DRVI 数据集中读入鸢尾花卉数据集和红酒数据集，然后用 PCA 算法将它们降为 2 维，显示散点图，观察分类效果。鸢尾花卉数据集包含 150 个样本，每行数据包含四个花的特征(花萼长度、花萼宽度、花瓣长度、花瓣宽度)和品种(3 种，每种 50 组数据)。红酒数据集是对意大利同一地区三个不同葡萄酒品种进行化学分析的结果，分析包括葡萄酒中含有的 13 种成分的数量。数据里共有 178 个样本，分别属于三个不同的类别。

数据的 PCA 降维和可视化处理实验样例代码

```
import numpy as np
import tkinter as tk
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.datasets import load_wine
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler
import drvi.drviControlls as dr


def setSignal(v):
    if v==0:
        abc = load_iris()
        M=4
    if v==1:
        abc = load_wine()
        M=13
    X = abc.data
    Y=abc.target
    mPlot.clearFig(wn=M)
    for i in range(M):
        mPlot.setValue2DXM(i,1,X[:,i])
    #PCA==================
    X=StandardScaler().fit(X).transform(X)
    pca = PCA(n_components=2)
    H=pca.fit_transform(X)
```

```
        #Draw Result==========
        mPlot1.setValue2DXM(0,1,H[:,0])
        mPlot1.setValue2DXM(1,1,H[:,1])
        fig, ax = plt.subplots(1,1)
        fig.tight_layout()
        plt.scatter(H[:,0],H[:,1],c=Y)
        mPlot2.setValueFig(fig)

#==Main Window=======================
win= tk.Tk()
winWidthCur=1060
winHeightCur=680
win.geometry("1060x680")
win.config(bg="#ddeeee")
#==Layout of controls=======================
mL1=dr.DRLabel(win,20,20,500,40,'#440000','#ffffff','数据的 PCA 降维和可视化',20)
mHBut=dr.DRHButtonGroup(win,740,25,300,36,'#003355','#ffffff','鸢尾花卉数据集,红酒数据集',12,0)
mTab=dr.DRNoteBook(win,20,75,1020,590,txt='原始数据,降维数据,降维度散点图')
mPlot=dr.DRMonitor(mTab.win[0],10,10,998,540)
mPlot1=dr.DRMonitor(mTab.win[1],10,10,998,540,wn=2)
mPlot2=dr.DRFigure(mTab.win[2],10,10,998,540,cb='#cccccc',cb1='#aaaaaa',cf='#000000',cf1='#000000',cf2='#ffff00')
#================================
mHBut.addCallBackSingle(setSignal)
#==Main Loop=======================
win.mainloop()
```

信号的 PCA 降噪实验

主成分分解后，将影响不大的主成分分量去掉，就可实现数据降维；而从信号分析角度，去掉不重要的主成分，相当于去掉信号中的干扰，也就是实现信号的去噪。

实验设计：

用 PCA 法进行信号分离或降噪时，需要构成观测数据集 A；可以采用下一行数据比上一行数据移位一个点的方式来构造数据集 A。然后，再按奇异值分解法求取出奇异值和左正交矩阵、右正交矩阵；最后按奇异值大小保留重要的奇异值(PCA)进行信号重构，就可以达到去除噪声的目的。

信号的 PCA 降噪实验样例代码

```
import numpy as np
import tkinter as tk
from scipy.linalg import svd
from scipy.misc import electrocardiogram
from scipy.io import loadmat
import drvi.drviControlls as dr
import drvi.drviAppendPath as mPath

def setSignal(v):
    M=128
    N=2500
    if v==0:
        ecg=electrocardiogram()
        x=ecg[256:256+N]
        mPlot.setValue2DXM(0,1,x)
    if v==1:
        mFile=mPath.getDRVIPath()+'\\dataset\\leleccum.mat'
        mat = loadmat(mFile)
        x = mat['leleccum'][0][0:N]
        mPlot.setValue2DXM(0,1,x)
    #PCA Denoise====
    L=N-M
    A=np.zeros((M,L))
    for i in range(M): A[i]=x[i:i+L]
    U,S,Vt=svd(A)
    #==前 k 个奇异值信号重构
    k=8
    EL=np.zeros((U.shape[1],Vt.shape[0]))
    EL[:k, :k]=np.diag(S[:k])
    AA = U @ EL @ Vt
    mPlot.setValue2DXM(1,1,AA[0,:])

#==Main Window=====================
win= tk.Tk()
win.geometry("1060x680")
win.config(bg="#ddeeee")
#==Layout of controls=====================
```

```
mL1=dr.DRLabel(win,20,20,500,40,'#440000','#ffffff','信号的 PCA 降噪实验',20)
mHBut=dr.DRHButtonGroup(win,740,25,300,36,'#003355','#ffffff','ECG 信号,电网电压信号',12,0)
mPlot=dr.DRMonitor(win,20,70,1029,580,wn=2)
#================================
mHBut.addCallBackSingle(setSignal)
#==Main Loop=====================
win.mainloop()
```

信号的 PCA 多尺度分解与重构实验

信号的 PCA 分析完成后，提取出每一个奇异值对应的信号成分，就可以实现 PCA 信号多尺度分解，然后可以按一定规则选择所需成分进行信号重构。

实验设计：

用 PCA 法进行信号的多尺度分解时，需要构成观测数据集 A；可以采用下一行数据比上一行数据移位一个点的方式来构造数据集 A。然后，再按奇异值分解法求取出奇异值和左正交矩阵、右正交矩阵；最后提取出各奇异值对应的信号成分，完成信号的多尺度分解。选择所需分解信号相加，就可完成信号的重构。

信号的 PCA 多尺度分解与重构实验样例代码

```
import tkinter as tk
import matplotlib.pyplot as plt
from scipy.linalg import svd
from scipy.misc import electrocardiogram
from scipy.io import loadmat
import drvi.drviControlls as dr
import drvi.drviAppendPath as mPath

def normSig(x):
    x=x-np.mean(x)
    x=x/np.max(x)
    return x

def setSignal(v):
    global L,x,Y;
    M=128; N=2500; L=N-M
    Y=np.zeros((10,L))
    if v==0:
        ecg=electrocardiogram()
        x=ecg[256:256+N]
    if v==1:
        mFile=mPath.getDRVIPath()+'\\dataset\\leleccum.mat'
        mat = loadmat(mFile)
        x = mat['leleccum'][0][0:N]
    x=normSig(x)
    mPlot.setValue2DXM(0,1,x[0:L],ylow=-1,yhigh=1)
    #PCA Denoise====
    A=np.zeros((M,L))
    for i in range(M): A[i]=x[i:i+L]
    U,S,Vt=svd(A)
    #==前 10 个奇异值分别对应的信号
    D=np.diag(S)
    for k in range(10):
        EL=np.zeros((U.shape[1],Vt.shape[0]))
        EL[k,k]=D[k,k]
```

```
            AA = U @ EL @ Vt
            Y[k]=AA[0,:]
            mPlot.setValue2DXM(k+1,1,Y[k],ylow=-1,yhigh=1)

def setADD(v):
    global L,x,Y;
    txt=mEntry.getValueString()
    ll=txt.split(',')
    NN=len(ll)
    print(NN)
    z=np.zeros(L)
    for i in range(NN):
        k=int(ll[i])
        z=z+Y[k-1]
    fig, axs = plt.subplots(2, 1)
    axs[0].plot(x[0:L])
    axs[0].set_ylim(-1,1)
    axs[1].plot(z)
    axs[1].set_ylim(-1,1)
    plt.show()

#==Main Window=======================
win= tk.Tk()
win.geometry("1060x680")
win.config(bg="#ddeeee")
#==Layout of controls=======================
mL1=dr.DRLabel(win,20,20,400,40,'#440000','#ffffff','信号 PCA 多尺度分解与合成',20)
mHBut=dr.DRHButtonGroup(win,440,25,260,36,'#003355','#ffffff','ECG 信号,电网电压信号',12,0)
mPlot=dr.DRMonitor(win,20,70,1029,580,wn=11)
mBut=dr.DRButton(win,740,25,80,36,'#00aacc','#eeee00','合成',12,101)
mEntry=dr.DREntry(win,825,25,220,36,'#ffffff','#000000','1,2,3,4,5,6,7,8,9,10',16)
#================================
mHBut.addCallBackSingle(setSignal)
mBut.addCallBackSingle(setADD)
#==Main Loop=======================
win.mainloop()
```

时间序列仿真与建模实验

时间序列是指按照时间顺序把某种现象的某一统计指标在不同时间点上的数值，按时间先后顺序排列而形成的序列；而观察、寻找时间序列变化规律，预测其未来发展趋势就是时间序列分析。

实验设计：

用理论模型生成仿真的时间序列数据，然后对仿真数据进行稳定性检验、定阶、建模和预测；全面学习时间序列分析所涉及的主要环节。

时间序列仿真与建模实验样例代码

```
from statsmodels.graphics.tsaplots import plot_pacf
from statsmodels.graphics.tsaplots import plot_acf
from statsmodels.tsa.stattools import adfuller
from statsmodels.tsa.arima_process import ArmaProcess
from statsmodels.tsa.arima.model import ARIMA
import matplotlib.pyplot as plt
import numpy as np
import warnings
warnings.filterwarnings('ignore')

#生成模拟数据===
N=128
#ARMA(1,1)
ar = np.array([1,0.7])
ma = np.array([1,-0.1])
dta=ArmaProcess(ar, ma).generate_sample(nsample=N)
#添加非稳定性项
dta=dta+np.arange(N)*0.01
#显示用于定阶的 acf 和 pacf 图
fig,ax=plt.subplots(nrows=3,ncols=1)
ax[0].plot(dta)
plot_acf(dta,ax=ax[1]);
plot_pacf(dta,ax=ax[2]);
#==稳定性检验
s=adfuller(dta)
print(s)
#==模型拟合
res = ARIMA(dta, order=(1, 1, 1)).fit()
#==预测===
df_test=res.predict(start=0,end=128+32)
fig=plt.figure()
plt.plot(dta)
plt.plot(df_test)
plt.show()
```

典型时间序列 ARIMA 建模实验

ARIMA 模型全称为自回归差分移动平均模型（Autoregressive Integrated Moving Average Model），它主要由三部分构成，分别为自回归模型（AR）、差分过程（I）和移动平均模型（MA）。

实验设计：

使用太阳黑子活动数据集和 AR(1)、MA(1)、ARMA(2,2)理论模型生成的仿真时间序列数据，然后对数据进行稳定性检验、BIC 定阶、建模、预测和模型谱分析；全面了解时间序列分析所涉及的主要环节。

典型时间序列 ARIMA 建模实验样例代码

```
import tkinter as tk
import numpy as np
from scipy import signal
import matplotlib.pyplot as plt
import pandas as pd
import statsmodels.api as sm
from statsmodels.tsa.arima.model import ARIMA
from statsmodels.tsa.stattools import adfuller as ADF
from statsmodels.tsa.arima_process import ArmaProcess
from statsmodels.graphics.tsaplots import plot_pacf
from statsmodels.graphics.tsaplots import plot_acf
import drvi.drviControlls as dr
import drvi.drviAppendPath as mPath
from threading import Thread
import warnings
warnings.filterwarnings('ignore')

#================================================
def getData(v):
    global dta,N
    if v==0:
        data = sm.datasets.sunspots.load_pandas().data
        dta = pd.Series(data['SUNACTIVITY'].values.squeeze())
    if v==1:
        ar = np.array([1,0.9])
        ma = np.array([1])
        dta=ArmaProcess(ar, ma).generate_sample(nsample=300)
        dta=pd.Series(dta)
    if v==2:
        ar = np.array([1])
        ma = np.array([1,0.9])
        dta=ArmaProcess(ar, ma).generate_sample(nsample=300)
        dta=pd.Series(dta)
    if v==3:
        ar = np.array([1,-0.75, 0.25])
        ma = np.array([1,0.65, 0.35])
```

```
        dta=ArmaProcess(ar, ma).generate_sample(nsample=300)
        dta=pd.Series(dta)
    N=len(dta)
    mPlot.setValue2DX(1,dta)
    stable()
    acfpacf()
    initfitData()

def stable():
    global dta,N
    dta1=dta.diff()
    dta2=dta1.diff()
    mPlot1.setValue2DXM(0,1,dta)
    mPlot1.setValue2DXM(1,1,dta1)
    mPlot1.setValue2DXM(2,1,dta2)
    s=sm.tsa.stattools.adfuller(dta.dropna(how='all'))
    s1=sm.tsa.stattools.adfuller(dta1.dropna(how='all'))
    s2=sm.tsa.stattools.adfuller(dta2.dropna(how='all'))
    S='原信号：'+str(s)+'\n 差分信号:'+str(s1)+'\n 二次差分:'+str(s2)
    mText1.setValueString(S)

def acfpacf():
    global dta,N
    fig,ax=plt.subplots(2,1)
    plot_acf(dta,ax=ax[0]);
    plot_pacf(dta,ax=ax[1]);
    mFigure1.setValueFig(fig)

def fitData(v):
    global dta,N,res
    p=mEntry1.getValueSingle()
    d=mEntry2.getValueSingle()
    q=mEntry3.getValueSingle()
    res=ARIMA(dta,order=(p,d,q)).fit()
    dta1=res.predict(start=np.min(dta.index),end=np.max(dta.index)+64)
    S=str(res.params)
    mText2.setValueString(S)
    mText12.setValueString(S)
    fig,ax=plt.subplots(3,1)
    ax[0].plot(dta);
    ax[0].plot(dta1);
    res.resid.plot(ax=ax[1])
    res.resid.plot(kind='kde',ax=ax[2])
    mFigure2.setValueFig(fig)

def initfitData():
    global dta,N,res
    mText2.setValueString('')
    mText12.setValueString('')
    fig,ax=plt.subplots(3,1)
```

```
    ax[0].plot(dta);
    mFigure2.setValueFig(fig)
    fig,ax=plt.subplots(1,1)
    plt.plot(np.zeros(256))
    mFigure3.setValueFig(fig)
    mText3.setValueString('')
    fig,ax=plt.subplots(1,1)
    plt.plot(np.zeros(256))
    mFigure4.setValueFig(fig)

def drawSpectrum(v):
    global dta,N,res
    if len(mText12.getValueString())<2: return;
    ar=res.polynomial_ar
    ma=res.polynomial_ma
    w, h = signal.freqz(ma,ar,512)
    fig,ax=plt.subplots(1,1)
    plt.plot(abs(h))
    mFigure3.setValueFig(fig)

def doBIC():
    global dta,N,res,hl
    d=mEntry22.getValueSingle()
    bic=np.zeros(100)
    K=0
    S=''
    for p in range(10):
        for q in range(10):
            res=ARIMA(dta,order=(p,d,q)).fit()
            bic[K]=res.bic
            K=K+1
            S=str(K)+','+str(p)+','+str(q)+': '+str(res.bic)+'\n'+S
            mText3.setValueString(S)
    fig,ax=plt.subplots(1,1)
    plt.plot(bic)
    mFigure4.setValueFig(fig)
    win.mainloop()

def setBIC(v):
    global dta,N,res,hl
    hl=Thread(target=doBIC)
    hl.start()

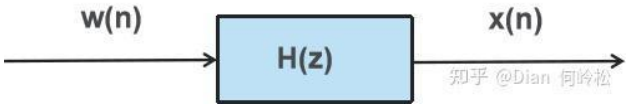
#==Main Window=======================
win= tk.Tk()
winWidthCur=1060
winHeightCur=680
win.geometry("1060x680")
win.config(bg="#ddeeee")
#==Layout of controls=======================
```

```
mL1=dr.DRLabel(win,20,20,400,40,'#440000','#ffffff','ARIMA 时间序列建模分析',20)
mBut=dr.DRHButtonGroup(win,460,25,580,36,txt='太阳黑子,AR1 模拟数据,MA1 模拟数据,ARMA22 模拟数据')
mTab=dr.DRNoteBook(win,20,75,1020,590,txt='原始数据曲线,稳定性检测,ACF/PACF 曲线,BIC 定阶,ARIMA 拟合,模型谱估计')
mPlot=dr.DRPlot(mTab.win[0],10,10,998,540)
mPlot1=dr.DRMonitor(mTab.win[1],10,10,998,430,wn=3)
mText1=dr.DRText(mTab.win[1],10,450,998,100,fsize=10)
mFigure1=dr.DRFigure(mTab.win[2],10,10,998,540,tight=True)
mBut1=dr.DRButton(mTab.win[4],10,10,180,36,txt='模型拟合(p,d,q)')
mEntry1=dr.DREntry(mTab.win[4],10,60,60,30,txt='8')
mEntry2=dr.DREntry(mTab.win[4],70,60,60,30,txt='1')
mEntry3=dr.DREntry(mTab.win[4],130,60,60,30,txt='2')
mText2=dr.DRText(mTab.win[4],10,100,180,440,fsize=10)
mFigure2=dr.DRFigure(mTab.win[4],200,10,798,540,tight=True)
mBut2=dr.DRButton(mTab.win[5],10,10,180,36,txt='模型谱估计')
mText12=dr.DRText(mTab.win[5],10,60,180,480,fsize=10)
mFigure3=dr.DRFigure(mTab.win[5],200,10,798,540,tight=True)
mBut3=dr.DRButton(mTab.win[3],10,10,180,36,txt='BIC 收索(p<10,d,q<10)')
mEntry21=dr.DREntry(mTab.win[3],10,60,60,30,txt='<10')
mEntry22=dr.DREntry(mTab.win[3],70,60,60,30,txt='1')
mEntry23=dr.DREntry(mTab.win[3],130,60,60,30,txt='<10')
mText3=dr.DRText(mTab.win[3],10,100,180,440,fsize=10)
mFigure4=dr.DRFigure(mTab.win[3],200,10,798,540,tight=True)
#================================
mBut.addCallBackSingle(getData)
mBut1.addCallBackSingle(fitData)
mBut2.addCallBackSingle(drawSpectrum)
mBut3.addCallBackSingle(setBIC)
#================================
getData(0)
#==Main Loop=====================
win.mainloop()
```

典型时间序列的高阶 AR 建模谱估计实验

对时间序列模型谱估计来说，建立的模型可以是 AR 模型、MA 模型或 ARMA 模型。从简化问题的角度，理论上我们可以用一个高阶的 AR 模型去逼近任意一个 MA 模型，或 ARMA 模型，这样就把问题统一为高阶 AR 模型的建模和谱估计。而高阶 AR 模型的建模效率比 ARMA 模型快很多倍。

如果将建立的 AR 模型看作系统的传递函数 H(z)，模型拟合后的预报误差 w(n)看作白噪声信输入，而信号 x(n)看作是经 H(z)滤波后的输出，则有如下系统框图。



由于白噪声信号的频谱为常数，信号 x(n)的频谱就等于滤波器 H(z)的频率特性。这样就把信号的谱估计问题转化为 AR 模型的频率特性估计；这种方法又称之为现代谱估计方法。由于是基于数学模型 H(z)来进行谱估计，理论上是一个连续谱，可以克服 FFT 谱估计精度受数据长度影响，在短数据时有优势。

实验设计：

使用太阳黑子活动数据集和三个正弦波组成的复合信号作为分析信号，然后对数据高阶 AR 建模和 AR 模型谱分析。

典型时间序列的高阶 AR 建模谱估计实验样例代码

```
import tkinter as tk
import numpy as np
from scipy import signal
import matplotlib.pyplot as plt
import pandas as pd
import statsmodels.api as sm
from statsmodels.tsa.arima.model import ARIMA
import drvi.drviControlls as dr
from threading import Thread
import warnings
warnings.filterwarnings('ignore')

def getData(v):
    global dta,N,Fs
    if v==0:
        st=0
        Fs=1
        data = sm.datasets.sunspots.load_pandas().data
        dta = pd.Series(data['SUNACTIVITY'].values.squeeze())
    if v==1:
        st=1
        N=300; Fs=5120; dt=1/Fs
        t=np.arange(N)*dt
        x1=np.sin(2*np.pi*150*t)
```

```
            x2=0.4*np.sin(2*np.pi*400*t)
            x3=0.4*np.sin(2*np.pi*800*t)
            x4=0.1*np.random.randn(N)
            x=x1+x2+x3+x4
            dta=pd.Series(x)
        N=len(dta)
        mPlot.setValue2DX(1,dta)
        initfitData()
        drawSpectrum()
        mTab.setPage(0)

def setBIC(v):
        global st,dta,N,res,hl
        e=np.zeros(64)
        b=np.zeros(64)
        S=''
        for p in range(64):
            res,err=sm.regression.linear_model.burg(dta,order=p+1)
            e[p]=err
            b[p]=np.log(err)+(p+1+1)*np.log(N)/N
            S=S+str(p+1)+': '+str(b[p])+'\n'
            mText3.setValueString(S)
        fig,ax=plt.subplots(1,1)
        plt.plot(b)
        mFigure4.setValueFig(fig)

def initfitData():
        global dta,N,Fs
        mText3.setValueString('')
        fig,ax=plt.subplots(1,1)
        plt.plot(np.zeros(256))
        mFigure4.setValueFig(fig)
        mPlot1.setXlim(0,Fs/2)
        mPlot2.setXlim(0,Fs/2)
        Z=np.zeros((512,64))
        tt=np.arange(64)
        ff=np.arange(512)
        mwaterFall.setValue3D(tt,ff,Z)

def drawSpectrum():
        global dta,N,Fs
        fftA=np.abs(np.fft.rfft(dta))
        f=np.arange(len(fftA))*(Fs/N)
        mPlot1.setValue2D(f,fftA)
        res,err=sm.regression.linear_model.burg(dta,order=64)
        ar=np.append(1,-res)
        w, h = signal.freqz(1,ar,5000)
        w=(Fs/2)*(w/np.pi)
        mPlot2.setValue2D(w,np.abs(h))
```

```
def setView(v):
    global dta,N,Fs
    Fs2=Fs/2;
    v1=mHSlider1.getValueSingle()/100
    v2=mHSlider2.getValueSingle()/100
    if v1>v2: v1=v2-0.01*v2
    mPlot1.setXlim(v1*Fs2,v2*Fs2)
    mPlot2.setXlim(v1*Fs2,v2*Fs2)

def setOrderSpec(v):
    global st,dta,N,res,hl
    M=64
    Z=np.zeros((512,M))
    for p in range(M):
        res,err=sm.regression.linear_model.burg(dta,order=p+1)
        ar=np.append(1,-res)
        w, h = signal.freqz(1,ar,512)
        Z[:,p]=np.abs(h)
    ff=(Fs/2)*(w/np.pi)
    tt=np.arange(M)
    mwaterFall.setValue3D(tt,ff,Z)

#==Main Window=====================
win= tk.Tk()
win.geometry("1060x680")
win.config(bg="#ddeeee")
#==Layout of controls=====================
mL1=dr.DRLabel(win,20,20,400,40,'#440000','#ffffff','高阶 AR 模型频谱估计',20)
mBut=dr.DRHButtonGroup(win,740,25,300,36,txt='太阳黑子,三频模拟数据')
mTab=dr.DRNoteBook(win,20,75,1020,590,txt='原始数据曲线,FFT 谱和 AR 模型谱,AR 模型 BIC 定阶,AR 阶次谱阵')
mPlot=dr.DRPlot(mTab.win[0],10,10,998,540)
mPlot1=dr.DRPlot(mTab.win[1],10,10,998,250)
mPlot2=dr.DRPlot(mTab.win[1],10,260,998,250)
mHSlider1=dr.DRHSlider(mTab.win[1],30,520,480,20,val=0)
mHSlider2=dr.DRHSlider(mTab.win[1],530,520,480,20,val=100)
mBut3=dr.DRButton(mTab.win[2],10,10,180,36,txt='BIC 收索(p<64)')
mText3=dr.DRText(mTab.win[2],10,60,180,480,fsize=10)
mFigure4=dr.DRFigure(mTab.win[2],200,10,798,540,tight=True)
mwaterFall=dr.DRwaterFall(mTab.win[3],10,10,998,540,'#113344','#000000','#ffffff','#00eeff',0.3,0.1,0.1,0,0)
mBut4=dr.DRButton(mTab.win[3],890,25,100,30,txt='计算阶次谱')
#==================================
mBut.addCallBackSingle(getData)
mBut3.addCallBackSingle(setBIC)
mBut3.addCallBackSingle(setOrderSpec)
mHSlider1.addCallBackSingle(setView)
mHSlider2.addCallBackSingle(setView)
mBut4.addCallBackSingle(setOrderSpec)
#==================================
getData(0)
#==Main Loop=====================
```

```
win.mainloop()
```